



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ
НАУКА У НОВОМ САДУ



Лука Бурсаћ

**Управљање датотекама,
трансакцијама и
интерпретација упита у
релационим базама података**

Нови Сад, 2026

	УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА 21000 НОВИ САД, Трг Доситеја Обрадовића 6	Број:
	ЗАДАТАК ЗА ЗАВРШНИ РАД	Датум:

(Податке уноси предметни наставник - менџор)

Студијски програм:	Софтверско инжењерство и информационе технологије		
Студент:	Лука Бурсаћ	Број индекса:	SV 22/2022
Степен и врста студија:	Основне академске студије		
Област:	Електротехничко и рачунарско инжењерство		
Ментор:	Бранко Милосављевић		
НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА: - проблем – тема рада; - начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;			

НАСЛОВ ЗАВРШНОГ РАДА:

Управљање датотекама, трансакцијама и интерпретација упита у релационим базама података
--

ТЕКСТ ЗАДАТКА:

Пројектовати и имплементирати систем за управљање релационим базама података у програмском језику Java. Систем треба да обухвати управљање датотекама и блоковима на диску, баферовање страница у радној меморији, трансакционе механизме са гарантовањем ACID особина, управљање метаподацима, као и парсирање, планирање и извршавање подскупа SQL језика кроз стабло релационих оператора. Систем реализовати у клијентско-серверској архитектури са сопственим протоколом за мрежну комуникацију. Исправност решења проверити одговарајућим скупом аутоматизованих тестова.
--

Руководилац студијског програма:	Ментор рада:

Примерак за: ☐ - Студента; ☐ - Ментора
--

	УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА 21000 НОВИ САД, Трг Доситеја Обрадовића 6	
	КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА	

Редни број, РБР:		
Идентификациони број, ИБР:		
Тип документације, ТД:		Монографска документација
Тип записа, ТЗ:		Текстуални штампани материјал
Врста рада, ВР:		Дипломски - бечелор рад
Аутор, АУ:		Лука Бурсаћ
Ментор, МН:		Др Бранко Милосављевић, редовни професор
Наслов рада, НР:		Управљање датотекама, трансакцијама и интерпретација упита у релационим базама података
Језик публикације, ЈП:		српски/ћирилица
Језик извода, ЈИ:		српски/енглески
Земља публиковања, ЗП:		Република Србија
Уже географско подручје, УГП:		Војводина
Година, ГО:		2026
Издавач, ИЗ:		Ауторски репринт
Место и адреса, МА:		Нови Сад, трг Доситеја Обрадовића 6
Физички опис рада, ФО: (поглавља/страна/ цитата/табела/слика/графика/прилога)		11/85/12/11/48/0/2
Научна област, НО:		Електротехничко и рачунарско инжењерство
Научна дисциплина, НД:		Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, ПО:		Релационе базе података, трансакције, датотеке, слогови, упити
УДК		
Чува се, ЧУ:		У библиотеци Факултета техничких наука, Нови Сад
Важна напомена, ВН:		
Извод, ИЗ:		Имплементација једног система за управљање релационим базама података у програмском језику Java
Датум прихватања теме, ДП:		
Датум одбране, ДО:		17.07.2026
Чланови комисије, КО:	Председник:	Др Горан Сладић, редовни професор
	Члан:	Др Милан Стојков, доцент
	Члан:	
	Члан, ментор:	Др Бранко Милосављевић, редовни професор
		Потпис ментора

	UNIVERSITY OF NOVI SAD • FACULTY OF TECHNICAL SCIENCES 21000 NOVI SAD, Trg Dositeja Obradovića 6	
	KEY WORDS DOCUMENTATION	
Accession number, ANO :		
Identification number, INO :		
Document type, DT :	Monographic publication	
Type of record, TR :	Textual printed material	
Contents code, CC :		
Author, AU :	Luka Bursać	
Mentor, MN :	Branko Milosavljević, Phd., full professor	
Title, TI :	Management of files, transactions and query interpretation in relational databases	
Language of text, LT :	Serbian	
Language of abstract, LA :	Serbian/English	
Country of publication, CP :	Republic of Serbia	
Locality of publication, LP :	Vojvodina	
Publication year, PY :	2026	
Publisher, PB :	Author's reprint	
Publication place, PP :	Novi Sad, Dositeja Obradovica sq. 6	
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	11/85/12/11/48/0/2	
Scientific field, SF :	Electrical and Computer Engineering	
Scientific discipline, SD :	Applied computer science and informatics	
Subject/Key words, S/KW :	Relational databases, transactions, files, records, queries	
UC		
Holding data, HD :	The Library of Faculty of Technical Sciences, Novi Sad	
Note, N :		
Abstract, AB :	Implementation of a system for relational database management written in Java	
Accepted by the Scientific Board on, ASB :		
Defended on, DE :	17.07.2026	
Defended Board, DB :	President:	Goran Sladić, Phd., full professor
	Member:	Milan Stojkov, Phd., asist. professor
	Member:	
	Member, Mentor:	Branko Milosavljević, Phd., full professor
		Menthor's sign



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

*Хвала мојим родитељима, мом куму и свим пријатељима који
су ми пружили невероватне животне прилике и учинили ме
оним што јесам*

Садржај

1	Увод	1
1.1	О систему	1
1.2	Клијентско-серверска архитектура	2
1.3	Систем за обраду упита	2
2	Управљање датотекама	3
2.1	Управљање блоковима	3
2.1.1	Интерфејс ка <i>file</i> систему оперативног система	3
2.1.2	Странице	3
2.1.3	Управљање <i>log</i> датотеком	4
2.2	Управљање баферима	5
2.2.1	Алгоритми избора бафера за смену	6
2.2.1.1	<i>Naive</i>	6
2.2.1.2	<i>FIFO</i>	6
2.2.1.3	<i>LRU</i>	6
2.2.1.4	<i>Clock</i>	6
2.2.1.5	<i>First unmodified</i>	6
2.2.1.6	<i>LRM</i>	6
2.3	Слогови	6
2.3.1	Механизми дефинисања структуре слога	7
2.3.1.1	Шема	7
2.3.1.2	Распоред поља	7
2.4	Примена структуре слогова на блокове	8
2.4.1	Страница слогова	8
3	Управљање трансакцијама	9
3.1	Реализација трансакционих механизма у систему	9
3.1.1	Трансакције као главно место приступа вредностима	9
3.1.2	Систем за опоравак	10
3.1.2.1	Систем <i>log</i> -овања	10
3.1.2.2	Алгоритми опоравка система	12
3.1.2.3	Алгоритам поништавања трансакције	13
3.1.3	Безбедан вишенитни приступ	13
3.1.3.1	Изолациони нивои трансакција	14
3.1.3.2	Каталог катанаца	15
3.1.3.3	Управљање катанцима на нивоу индивидуалних трансакција	16

3.2	Трансакције и клијенти	16
3.2.1	Алгоритам записа мирне контролне тачке	16
4	Метаподаци	17
4.1	Каталожке табеле	17
4.2	Статистички подаци	18
4.2.1	Рачунање статистичких података	18
4.3	Пристап метаподацима	18
5	Релациони оператори	19
5.1	Виртуелна машина	19
5.1.1	Константе	19
5.1.2	Изрази	20
5.1.3	Чланови	21
5.1.4	Предикати	21
5.1.5	Генерализација евалуације	21
5.2	Структура релационих оператора у систему	22
5.2.1	Проточна обрада	22
5.2.2	Хијерархија имплементације релационих оператора	22
5.2.2.1	Scan	23
5.2.2.2	UpdateScan	24
5.2.2.3	UnaryScan	24
5.2.2.4	UnaryUpdateScan	24
5.2.2.5	BinaryScan	24
5.2.2.6	DiffSchemaJoinContextScan	24
5.2.2.7	TableScan	24
5.2.2.8	DummyTableScan	25
5.2.2.9	SelectScan	25
5.2.2.10	ExtendProjectScan	25
5.2.2.11	RenameScan	25
5.2.2.12	ProductScan	25
5.2.2.13	UnionAllScan	26
5.2.3	Примери	26
5.2.3.1	Пример изгледа компоненти виртуелне машине	26
5.2.3.2	Пример позива <code>getValue()</code> методе	27
5.2.3.3	Пример позива <code>next()</code> методе	28
6	Парсирање	29
6.1	Токенизатор	29
6.2	Парсер	30
6.2.1	Искази	30
6.2.2	Синтаксичке категорије <i>SQL</i> операција	30

6.2.2.1	Parse	30
6.2.2.2	ParseUpdate	31
6.2.2.3	ParseSelect	31
6.2.2.4	ParseInsert	32
6.2.2.5	ParseDelete	32
6.2.2.6	ParseCreateTable	33
6.2.2.7	ParsePredicate	33
6.2.2.8	ParseExpression	34
6.2.2.9	ParseEvaluable	36
7	Планирање	37
7.1	Структура класа планова у систему	37
7.1.1	Хијерархија имплементације планова	37
7.1.1.1	Plan	39
7.1.1.2	TablePlan	39
7.1.1.3	DummyTablePlan	39
7.1.1.4	SelectPlan	39
7.1.1.5	ExtendProjectPlan	41
7.1.1.6	RenamePlan	42
7.1.1.7	ProductPlan	43
7.1.1.8	UnionAllPlan	44
7.2	Планер	44
7.2.1	Евалуација израза током планирања	44
7.2.2	Улазна тачка креирања и извршавања планова	45
7.2.3	Планирање <i>read-only</i> наредби	46
7.2.3.1	Алгоритам планирања SELECT наредби	47
7.2.3.2	Аутоматско генерисање описа планова	51
7.2.4	Планирање модификационих наредби	52
7.2.4.1	Алгоритам планирања INSERT наредбе	53
7.2.4.2	Алгоритам планирања UPDATE наредбе	53
7.2.4.3	Алгоритам планирања DELETE наредбе	53
7.2.4.4	Алгоритам планирања CREATE TABLE наредбе	54
8	Клијентско-серверска архитектура	55
8.1	Комуникација са клијентима	55
8.1.1	Одговори система	55
8.1.2	Протокол серијализације	56
8.2	Серверски слој	57
8.2.1	Руковођење клијентима	57
8.2.1.1	Обрада једног клијента	58
8.2.2	Писање контролних тачака	59

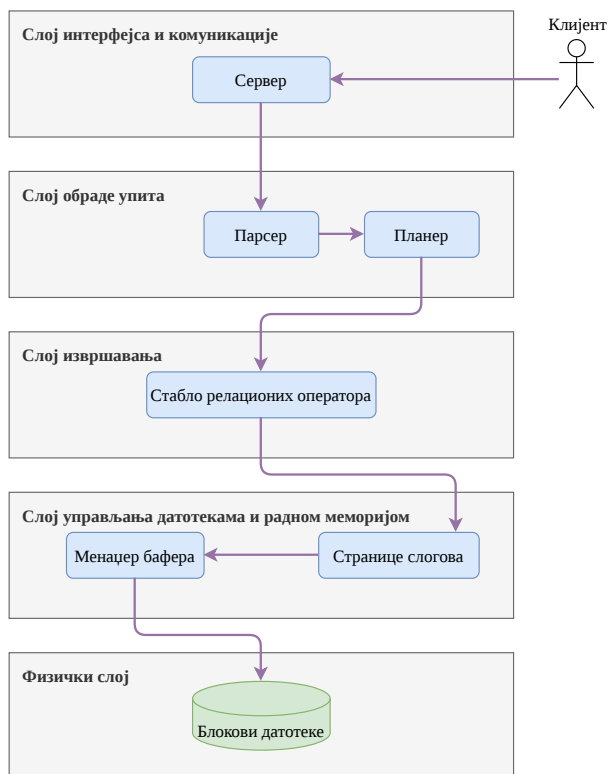
8.2.3	Гашење система	59
8.3	Клијентски слој	60
8.3.1	Штампање табела	60
8.3.2	Масовно покретање наредби	60
9	Тестови	61
9.1	Организација тестова	61
9.1.1	Тестно окружење	61
9.1.2	Систем изолације директоријума на диску	62
9.1.3	Систем изолације директоријума у радној меморији	62
9.2	Функционални тестови	62
9.3	Тестирање перформанси	63
9.3.1	Тестови перформансе алгоритама смене бафера	63
9.3.2	Тестови перформансе <i>RBO</i> технике у планирању <i>SELECT</i> наредби ..	66
10	Преглед система	67
10.1	Изградња и покретање	67
10.1.1	Руковођење зависностима	67
10.1.1.1	Зависности сервера	67
10.1.1.2	Зависности клијента	68
10.1.1.3	Зависности тестног окружења	68
10.2	Системска конфигурација	68
10.3	Интеграција са <i>GitHub</i> платформом	69
10.4	Пример функционисања целокупног система	69
11	Закључак	71
11.1	Мотивација	71
11.2	Ограничења система	71
11.2.1	Имплементиран је само подскуп <i>SQL</i> -а	71
11.2.2	Ограничење на слоге фиксне дужине	72
11.2.3	Споро рачунање статистичких метаподатака	73
11.2.4	Недостајућа имплементација индексних структура података	73
11.2.5	Неоптимални планер	74
11.2.6	Уланчавање чланова предиката је могуће само коњункцијом	74
11.3	Разлике између основне имплементације и <i>LBDB</i> система	74
	Списак слика	75
	Списак листинга	77
	Списак табела	79
	Списак коришћених скраћеница	81
	Списак коришћених појмова	83
	Литература	85

Глава 1

Увод

1.1 О систему

LBDB је вишекориснишки систем са трансакцијама за управљање релационим базама података. Његова сврха је да прима наредбе добијене од клијената, интерпретира их по стандарду *SQL* програмског језика и да врати резултат тим клијентима. Резултат може бити број погођених слогова за случај модификационих операција или резултујућа табела за случај упита. Зарад ефикасног интерпретирања, потребно је обезбедити алгоритме и структуре података за следеће модуле: управљање датотекама, рад са трансакцијама, рад са метаподацима, парсирање упита, прављење планова извршавања упита, извршавање упита и за мрежну комуникацију. Модули овог система су распоређени тако да се сваки брине о једној групи алгоритама и структура података кроз коју упит пролази.



Слика 1: Упростио дијаграм слојевите архитектуре система

Сваки слој из упрошћене архитектуре система је посебно детаљно објашњен у наставку, а уз те слојеве се додатно објашњавају и трансакције, које су испреплетене кроз цео систем. Сличан дијаграм који описује целу архитектуру система, али много детаљније, се може пронаћи на слици 33.

Основна структура и алгоритми су изведени из књиге *Database Design And Implementation* [1], а њихова унапређења су део овог рада. Књига дефинише задатке на крају сваког модула и ти задаци су основа за унапређивање система. Преглед знатних разлика између основне имплементације и *LBDB* система се може пронаћи у секцији 11.3, а детаљан списак урађених задатака и њихових белешки се може пронаћи у оквиру репозиторијума¹.

1.2 Клијентско-серверска архитектура

LBDB систем се покреће као сервер и њему се приступа преко мреже и специјалног протокола. Постоји имплементација клијентске апликације која имплементира овај протокол. Детаљи се могу пронаћи у поглављу 8 које описује клијентско-серверску архитектуру система.

1.3 Систем за обраду упита

LBDB класа представља највиши апстрактциони ниво система обраде упита на који се сервер ослања. Служи за оркестрацију управљача главних подсистема: менаџер трансакција (секција 3.2), менаџер метаподатака (секција 4.3) и планер (секција 7.2.2). Такође, класа *LBDB* је одговорна за логику иницијализације и опорављања система од неочекиваног гашења, за одређени директоријум базе података.

¹<https://github.com/lukascobbler/lbdb>

Глава 2

Управљање датотекама

Управљање датотекама се врши кроз више слојева у систему, где је сваки слој одговоран за организацију датотека на различитом апстракционом нивоу. Основна премиса је да све операције са датотекама, то јест диском морају да се извршавају у јединицама блокова, јер је оперативни систем, а и хардвер (диск) оптимизован за рад са њима [1]. Због тога што је блок најмања јединица интеракције са диском и датотекама, сва читања, писања и модификације података се раде заједно са целим блоком где се ти подаци налазе, а не директно.

2.1 Управљање блоковима

Систем за управљање блоковима је примарно задужен за добављање блока са диска где се налазе тражени подаци и за коректно записивање *log* података. Сваки блок има свој уникатни идентификатор, који је представљен *Java record* структуром. Сваки блок је везан за датотеку и има своју блок позицију у тој датотеци.

2.1.1 Интерфејс ка *file* систему оперативног система

Најнижи ниво апстракције представља менаџер датотека (*FileManager* класа) који има функцију интерфејса ка *file* систему оперативног система и нема представу шта се налази у самим датотекама *LBDB* система. Менаџер датотека чува показиваче на све датотеке којима систем управља и омогућава вишенитни безбедан приступ истим, употребом *Java synchronized* кључне речи. Вишенитни безбедан приступ омогућава систему да подржи више различитих клијената у исто време, али није довољан само на овом слоју, већ је детаљно обрађен у оквиру трансакција (секција 3.1.3).

Могуће је подесити систем да користи произвољну величину једног блока, зависно од природе података којима ће база података бити попуњена и то је главни параметар менаџера датотека.

2.1.2 Странице

Страница (енг. *page*) представља сирове бајтове једног блока учитане у радну меморију. Иако није нужно потребно, све вредности из странице заједно са њиховим типом се претварају у еквивалентне *Java* објекте (користећи *ByteBuffer* стандардну *Java* класу) да би се омогућио приступ операцијама из *Java* стандардне библиотеке за те типове. Када менаџер датотека добавља одређени блок, бајтови тог блока се смештају у радну меморију у објекат странице. Странице су на апстракционом нивоу испод система баферовања и користе се као потпора тог система.

Систем подржава следеће просте и композитне типове података: *String*, *Boolean* и *Integer*. Код простих типова, из странице се само читају њихови бајтови и претварају у одређени *Java* објекат тог типа, док код композитних типова као што је *String* потребно је знати дужину, саме бајтове и кодирање да би се конструисао *Java* објекат *String* типа.

2.1.3 Управљање *log* датотеком

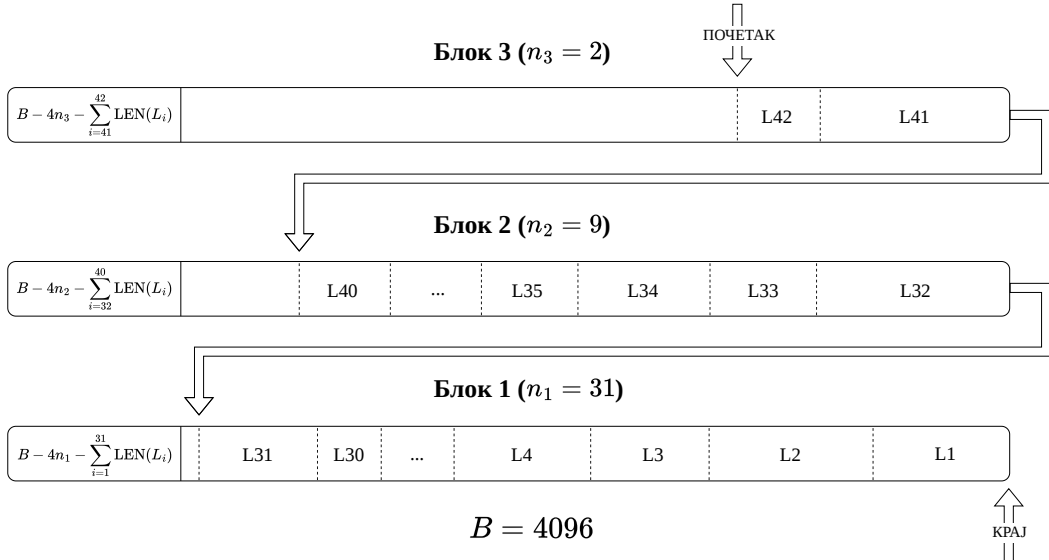
Један *log* је низ бајтова чије интерпретирање указује на то како се променила нека вредност у неком блоку.

Log датотека је специјална врста датотеке у којој се чува низ *log*-ова. На апстракционом нивоу управљања блоковима, није битан садржај ове датотеке, већ само алгоритми неопходни за коректно пролажење кроз њу и алгоритми за додавање нових *log*-ова. Ови алгоритми се налазе у менаџеру *log*-ова (*LogManager* класа) и *log* итератору (*LogIterator* класа).

Log датотека је организована тако да се новији *log*-ови налазе у блоковима ближим крају датотеке, а унутар једног блока *log* датотеке новији *log*-ови се налазе при почетку блока. Ако нема места да се упише нови *log*, прелази се у следећи блок *log* датотеке.

Менаџер *log*-ова обезбеђује алгоритме управљања *log* датотеком тако што чува страну последњег блока *log* датотеке. Управљање *log* датотеком подразумева додавање нових *log*-ова, упис *log*-ова на диск и архивирање *log* датотеке.

Итератор *log*-ова обезбеђује читање *log* датотеке у коректном редоследу и имплементиран је помоћу *Java Iterator* интерфејса.



Слика 2: Изглед *log* датотеке и смер итерације

Сваки блок *log* датотеке на нултој позицији садржи четворобајтни број који представља локацију првог *log*-а у том блоку. Сваки *log* се састоји из садржаја и своје дужине која је исто четворобајтни број.

Приликом додавања новог *log*-а, систем израчунава *log* секвенцу (енг. *log sequence number*) која јединствено идентификује записан *log* и користи се у даљим подсистемима за форсирање писања претходних *log*-ова, гарантовајући редослед операција.

2.2 Управљање баферима

Чишћење странице из меморије након завршетка операције која ју је користила може бити јако неефикасно јер се за поновни приступ том блоку мора одлазити до диска, поготово у вишекориснишким контекстима и ситуацијама када се истим блоковима често приступа. Због тога се уводи концепт бафера (енг. *buffer*) који, уз алгоритме у менаџеру бафера (*BufferManager* класа), омогућава страницама да остану у меморији прилагодљиву количину времена. Последица овог система је да директан приступ подацима са диска више није могућ, већ се све операције обављају кроз бафере.

Buffer класа енкапсулира учитану страницу и садржи додатне податке који се могу искористити за имплементацију разних алгоритама убрзања система:

- када је страница учитана
- када је било последње приступање страници
- који је редни број бафера у листи бафера
- која трансакција је последња изменила ту страницу и *log* секвенца задње операције
- колико трансакција тренутно употребљава ту страницу (број пинова бафера)

Пошто је количина радне меморије ограничена, и количина бафера у систему исто мора бити ограничена. Менаџер бафера има листу бафера са којима располаже и потребно је да повећа степен искоришћености бафера из те листе што је више могуће. У идеалном, хипотетичком сценарију, менаџер бафера би предвидео будућност и знао којим баферима би се следеће приступало и избацио из меморије оне који су временски најдаље од приступа. Овај сценарио је очигледно немогућ, па је потребно искористити алгоритме који најбоље “предвиђају будућност” на основу реалних података. Када се бафер избаци из меморије, његов садржај се пише у блок за који је везан и на тај начин се подаци перзистирају. Постоје и друге ситуације (секција 3.1.2.2.2) када се садржај бафера одмах записује на диск.

Да би се бафер избацио из меморије, не сме да буде део ни једне актуелне трансакције, то јест број пинова му мора бити нула. Постоје два сценарија када страница која до сада није била у меморији треба да се мапира на бафер:

- У случају да не постоји ниједан бафер са нула пинова, нова страница чека одређени временски период да се ослободи неки бафер и ако се ниједан бафер не ослободи, враћа се грешка клијенту (детаљније у секцији 3.1.3.2).

- У случају да постоји више бафера са нула пинова, потребно је изабрати који ће бити избачен из радне меморије помоћу алгоритма избора.

2.2.1 Алгоритми избора бафера за смену

У оптицају је неколико алгоритама [1] за избор бафера који ће бити смењен. Потврда свих тврдњи о перформансама се може пронаћи у [1] и у секцији 9.3.1.

2.2.1.1 *Naive*

Наивни алгоритам, како му и име каже, нема никакву логику избора бафера којег ће сменити већ само узима први бафер који има нула пинова. Очекиване су лоше перформансе јер је велика шанса да ће се сменити бафер који ће се ускоро опет користити.

2.2.1.2 *FIFO*

FIFO (енг. *First In First Out*) алгоритам смењује бафер који је најраније ушао у листу бафера. Перформансе су боље од наивног алгоритма али *FIFO* алгоритам ће сменити и јако често коришћене бафере иако су најраније ушли у систем, на пример бафери где се чувају блокови метаподатака система.

2.2.1.3 *LRU*

LRU (енг. *Least Recently Used*) алгоритам смењује најдавніје коришћен бафер. Перформансе су одличне јер се неактивни бафери ретко користе у блиској будућности [1].

2.2.1.4 *Clock*

Clock алгоритам претпоставља да бафери имају фиксне позиције у листи бафера и смењује први бафер који има нула пинова, али претрагу почиње од претходног смењеног бафера. Перформансе су одличне јер је битан бафер вероватно пинован [1].

2.2.1.5 *First unmodified*

First unmodified алгоритам смењује први бафер који пронађе да није модификован и да има нула пинова, или ако је сваки модификован први који има нула пинова. Перформанса може бити боља од наивног алгоритма, али може се десити да модификован бафер неће дуго бити коришћен што значи да није изабран оптималан бафер за смену.

2.2.1.6 *LRM*

LRM (енг. *Least Recently Modified*) алгоритам смењује последње измењен бафер који има нула пинова. Претпоставка је да модификовани бафер неће поново бити коришћен неко време јер је трансакција већ завршила. Може бити бољи од наивног алгоритма у специфичним ситуацијама, а често је лошији од њега.

2.3 Слогови

Подсистем за управљање баферима је генералан и не пружа никакву структуру података унутар самих бафера, односно блокова. На нивоу релационе базе података,

најмања јединица интеракције није један блок, већ један слог неке табеле и потребно је блокове организовати тако да се то омогући. Један слог неке табеле је исто што и један ред те табеле што је детаљније објашњено у поглављу 5.

2.3.1 Механизми дефинисања структуре слога

Да би се подржало креирање перзистентне структуре једног слога нове табеле, потребно је дефинисати механизме којима ће клијенти описивати ту структуру. На апстракционом нивоу управљања датотекама, систем се само брине о томе да је теоретска и физичка структура испоштована, а перзистирање и само креирање структуре је задатак виших подсистема.

2.3.1.1 Шема

Сваки слог једне табеле се састоји од истих метаподатака, то јест истих колона. Свака колона је описана својим типом, својом дужином и тиме да ли може имати *NULL* вредности.

Сваки од типова је дефинисан у *SQL Java* стандардној библиотеци, али коришћење тих вредности директно може довести до некомплетности на разним местима где су типови коришћени у систему, па је због тога уведена енумерација која стриктно дефинише подржане типове, заједно са њиховом подразумеваном дужином у бајтовима. Тип *VARCHAR*, то јест *String* нема подразумевану дужину јер је различита за свако поље. Додатно постоји и *NULL* тип који означава одсуство вредности за то поље.

```
import java.sql.Types;
public enum DatabaseType {
    INT(Types.INTEGER, 4), BOOLEAN(Types.BOOLEAN, 1),
    VARCHAR(Types.VARCHAR, -1), NULL(Types.NULL, 0);
}
```

Листинг 1: Подржани типови колона

Шема једне табеле је скуп података о колонама те табеле заједно са именима тих колона. Битно је напоменути да табеле у својој основној дефиницији представљају податке које се налазе у датотекама, али то није увек случај. Постоје и виртуелне табеле које су резултати упита и могу, али не морају да се директно мапирају на табеле које се налазе у датотекама. Могуће је комбиновати више табела у једну табелу и додати виртуелне колоне (секција 5.2.2.10) које се не налазе у датотеци већ су њихове вредности изведене на основу неке калкулације.

2.3.1.2 Распоред поља

Шема представља теоретски изглед једне табеле, али то није довољно да би се тај изглед перзистирао и могао поново рекреирати. Због тога је потребно увести механизам памћења и физичких карактеристика колона табеле (постоји само за неvirtуелне табеле). Тај механизам се реализује преко распореда поља (енг. *layout*).

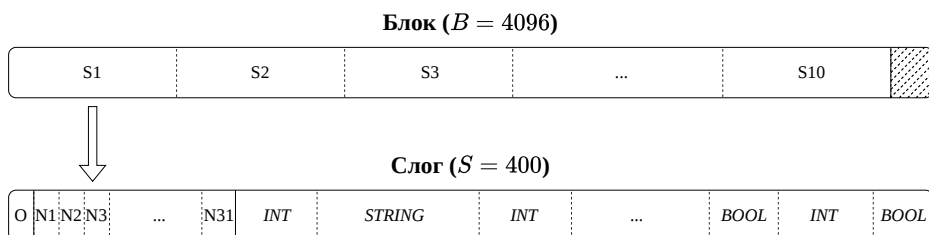
За сваку колону памте се следеће физичке особине: позиција почетка вредности те колоне, максимална дужина вредности те колоне и позиција те колоне у шеми. Памти се и целокупна дужина целог слога. Колоне се идентификују помоћу њиховог назива.

2.4 Примена структуре слогова на блокове

Након дефинисања физичке структуре слога табеле, потребно је применити ту физичку структуру на блокове датотека. У *LBDB* систему, један блок садржи фиксни број слогова који су сви из исте табеле и не постоје вредности променљиве дужине (више о овом ограничењу у секцији [11.2.2](#)).

Пошто су сви слогови исте дужине, $\frac{B}{S}$ слогова стаје у један блок, где B представља дужину блока у систему, S представља дужину једног слога те табеле, а $B - S * \lfloor \frac{B}{S} \rfloor$ простора остаје неискоришћено (све вредности су у бајтовима). Слогови у блоковима чувају само вредности колона, али не и метаподатке тих колона. За чување метаподатака постоји посебан подсистем који је описан у поглављу [4](#).

Ипак, у оквиру једног слога се чувају метаподаци о томе које вредности нису присутне, то јест имају *NULL* вредност и то да ли је слог обрисан. Резервише се четворобјитно заглавље на почетку сваког слога и његови битови представљају ове метаподатке. Да ли је слог означен као обрисан се представља првим битом (O), док осталих 31 битова (N_i) означавају да ли поље на тој позицији има *NULL* вредност. Због овога постоји ограничење на број поља по табели, максимално 31 поље.



Слика 3: Изглед једног блока попуњеног слоговима

2.4.1 Страница слогова

RecordPage класа енкапсулира сву логику одржавања структуре индивидуалног блока тако што пружа интерфејс за постављање вредности само на основу имена колоне и броја слога у том блоку. Такође, пружа интерфејс за постављање *NULL* вредности и претрагу слободних или заузетих слогова у блоку за који је повезана. За приступ свим блоковима једне табеле, потребно је sukcesивно конструисати објекте *RecordPage* класе, што је посао подсистема релационих оператора (секција [5.2.2.7](#)).

Битно је напоменути да логика странице слогова за постављање вредности не ради само постављање вредности, већ рачуна где та вредност треба бити постављена. Постављање вредности делегира систему трансакција.

Глава 3

Управљање трансакцијама

Трансакције су основни механизам за гарантовање различитих особина отпорности система за управљање релационим базама података. Омогућавају систему да врши атомичне операције, да се увек одржи у конзистентном стању, да изолије независне конкурентне операције и да гарантује перзистентност података. Генерално, ове особине се зову *ACID* особине (енг. *atomicity, consistency, isolation, durability*) [2].

Свака операција у систему мора бити извршена у оквиру једне трансакције, али се једна трансакција може састојати и од више операција које се све морају или успешно извршити или се морају све поништити. Свака трансакција има почетак и крај. Крај може бити потврда (енг. *commit*) или поништавање (енг. *rollback*). Ако је трансакција успешно потврђена, од тог тренутка па на даље све извршене промене морају бити одмах видљиве другим трансакцијама.

3.1 Реализација трансакционих механизма у систему

Вредност је низ бајтова (одређеног типа), која добија семантички значај тек на апстракционим нивоима изнад нивоа трансакција, наиме на нивоу организовања блокова у слоге. На нивоу трансакција, вредности немају семантичко значење, али да би систем обезбедио висок степен усклађености са *ACID* особинама, операције над вредностима се обављају искључиво кроз трансакције које енкапсулирају сву потребну логику тих особина.

3.1.1 Трансакције као главно место приступа вредностима

На нивоу слогова, операције се извршавају у целинама једног слога, али градивни елементи тих слогова су ипак вредности. Страница слогова управља позицијама вредности, а сам приступ њима делегира трансакцији за коју је везана.

Пошто блокови где се вредности налазе морају бити у меморији да би се њима приступило, потребно је учитати их у системске бафере. Подсистем за управљање баферима је свестан само начина мапирања блокова на бафере, али нема никакве гаранције о редоследу приступа, не чува од конфликтујућих операција и не прати историју измена. То је посао трансакција.

Сви бафери у радној меморији система се групишу у трансакције, тако што свака трансакција чува листу својих бафера и ради операције само над њима. Бафер улази у листу неке трансакције тако што га трансакција пинује, и тиме означава да се садржај тог бафера не сме вратити на диск док трансакција не заврши са њим (док не прекине

баферов пин). Страница слогова и трансакција раде у пару: страница слогова зна којем блоку жели да приступи, а трансакција зна како да обезбеди да операције над бафером тог блока испоштују *ACID* особине.

3.1.2 Систем за опоравак

Систем за опоравак је подсистем у оквиру трансакција који садржи први део логике због којег се сав приступ вредностима ради кроз трансакције. Примарно се брине о опоравку система, али се брине и о поништавању операција једне трансакције (енг. *rollback*). Обухвата **ACID** (енг. *atomicity, consistency, durability*) особине.

LBDB систем је софтверско решење које ради у контексту неког хардвера и оперативног система. Сваки од слојева на које се *LBDB* систем ослања, има могућност да закаже због неког фактора који је изван контроле *LBDB* система. Примери су нестајање струје, убијање *LBDB* серверског процеса или неуспешно писање на диск. Ако се у тренутку заказивања извршава нека операција која мења податке у систему, ти подаци ће бити изгубљени, а систем ће бити остављен у неконзистентном стању. Коришћење система који је у неконзистентном стању може довести до лошег тумачења података, али може бити и опасно у појединим ситуацијама. Због овога се уводи подсистем који опоравља систем од неконзистентног стања.

Конзистентно стање се може дефинисати са следеће две особине [1], [2]:

- све недовршене трансакције су поништене,
- све модификације потврђених (енг. *committed*) трансакција морају бити на диску

3.1.2.1 Систем *log*-овања

Log-ови су у *log* датотеци увек поређани редом којим су се извршили у оквиру једне трансакције. Писањем *log*-ова се постиже високо гранулирана историја измена. Основа алгоритама опоравка је пролажење кроз историју измена почевши од последње измене, да би се те измене поништиле или поново примениле тачним редоследом.

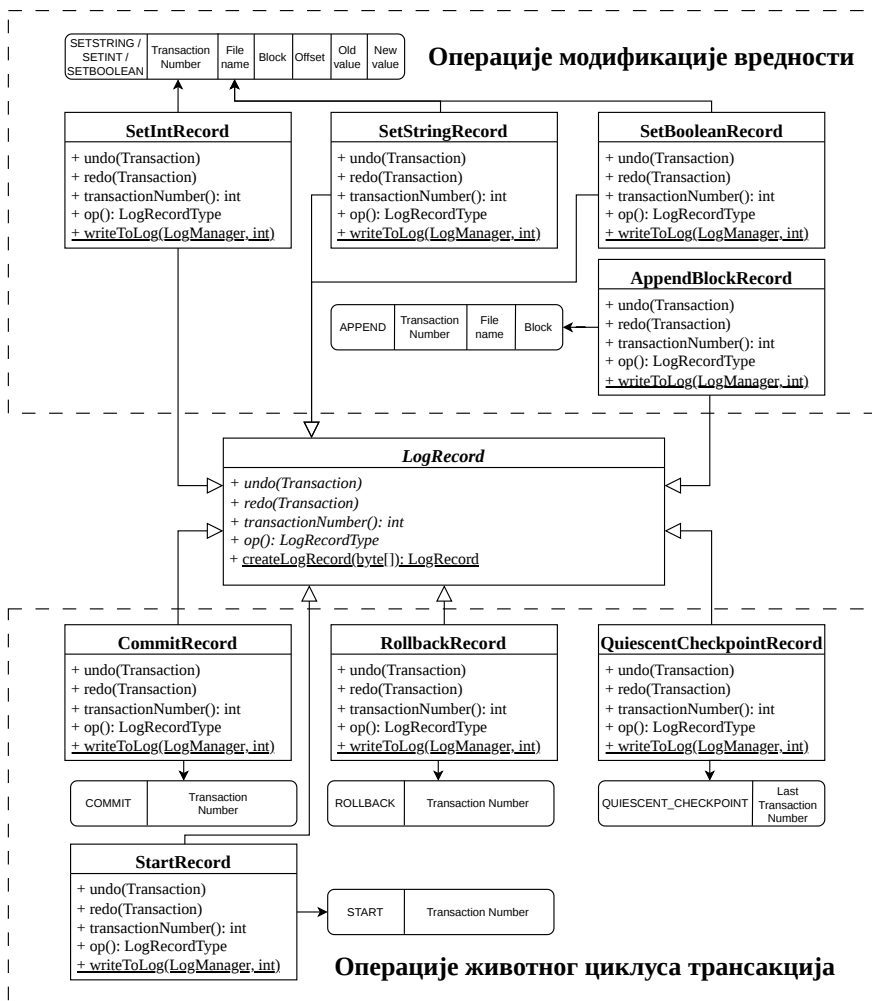
У систему постоје две главне групе операција: операције модификације вредности и операције животног циклуса трансакција. За сваку операцију се дефинише структура *log*-а. За операције модификације вредности, у *log*-у стоје сви подаци неопходни да се та операција поништи или поново примени, док у *log*-у операције животног циклуса трансакције стоје сви неопходни подаци за праћење рада трансакције.

Пошто у *log*-овима операција модификације вредности увек стоје и стара и нова вредност, поништавање или поновна примена операције је тривијална. Специјална операција модификације је операција уметања новог блока на крају датотеке, која не мења никакву вредност али је и даље потребно пратити њене позиве због одржавања коректне величине датотека. Алгоритам поништавања уметања новог блока није тривијалан јер није само замена вредности, већ је потребно означити бафер тог блока као

немодификован и скратити датотеку за један блок. У *log* датотеку се увек записује нови *log* за позвану операцију модификације пре саме обраде те операције.

Операције животног циклуса трансакција је потребно пратити да би систем опоравка знао које трансакције су се успешно и неуспешно извршиле и на основу тога реаговао на операције модификације. Маркер контролне тачке је специјални запис који означава када није потребно даље пролазити кроз *log* датотеку.

Хијерархија *log*-ова почиње од главног интерфејса *LogRecord* који дефинише неопходне операције које ће алгоритми опоравка звати. Операције животног циклуса трансакција остављају празну имплементацију *undo* и *redo* метода јер не модификују податке. Додатно, сваки тип *log*-а има пратећу статичку методу *writeToLog* која енкапусуира логику записа *log*-а преко менаџера *log*-ова за неки број трансакције.



Слика 4: Операције, њихови *log* типови и структура *log* типова

3.1.2.2 Алгоритми опоравка система

Алгоритам опоравка система је процедура која прати кораке дефинисане стратегијом опоравка коју систем користи и заједно са подацима из *log* датотеке враћа систем у конзистентно стање. Сваки алгоритам опоравка мора бити идемпотентан, јер систем може нагло престати са радом и док је у процесу опорављања [1]. Процес опоравка се врши у оквиру подизања система.

Постоје три генерална алгоритма опоравка система [1]:

- поновна примена успешних трансакција и поништавање неуспешних трансакција (енг. *undo redo recovery*),
- само поништавање неуспешних трансакција (енг. *undo only recovery*),
- само поновна примена успешних трансакција (енг. *redo only recovery*).

Избор алгоритма опоравка утиче на то када ће садржај бафера бити уписан на диск.

У *LBDB* систему, имплементирани су само *undo redo* и *undo only* алгоритми опоравка.

3.1.2.2.1 *Undo redo recovery*

Алгоритам се дели на два корака: фаза поништавања и фаза поновне примене.

Фаза поништавања (енг. *undo*):

- За сваки *log* запис (читајући уназад од краја *log* датотеке):
 - Ако је текући запис *commit* запис, додај ту трансакцију у листу потврђених.
 - Ако је текући запис *rollback* запис, додај ту трансакцију у листу поништених.
 - Ако је текући запис *update* запис (модификациона операција), и трансакција није ни у листи потврђених ни у листи поништених, врати стару вредност на задатој локацији.

Фаза поновне примене (енг. *redo*):

- За сваки *log* запис (читајући унапред од почетка *log* датотеке):
 - Ако је текући запис *update* запис и трансакција је у листи потврђених трансакција, упиши нову вредност на задатој локацији.

Главна предност овог алгоритма је то што не утиче на редослед писања бафера на диск, али мана је то што је процес опоравка спорији. Ако се претпостави да су изненадна гашења система ретка, предности овог алгоритма побеђују остале алгоритме.

3.1.2.2.2 *Undo only recovery*

Undo only recovery алгоритам ради само фазу поништавања јер је сигуран да су сви бафери *committed* трансакција већ записани на диску. Ово се постиже модификацијом *commit* алгоритма тако да се прво уписује садржај бафера на диск пре писања *commit log*-а. Главна предност овог алгоритма је брзина, јер се кроз *log* датотеку пролази само једном, али мана је то што *commit* операција постаје много спорија.

3.1.2.2.3 *Redo only recovery*

Redo only recovery алгоритам ради само фазу поновне примене јер је сигуран да бафери трансакција које нису ни потврђене ни поништене сигурно нису записани на диск. Ово се постиже модификацијом трансакција тако да су бафери у њиховим листама пиновани докле год се трансакција не заврши. Главна предност овог алгоритма је брзина јер се кроз *log* датотеку пролази само једном, али мана је то што бафери остају пиновани много дуже, драстично успоравајући систем.

3.1.2.2.4 Мирна контролна тачка

Што се систем дуже користи, то ће његов *log* датотека постајати обимнији јер увек садржи комплетну историју измена података. Пролазак кроз целу *log* датотеку приликом сваког покретања система може трошити непотребно много ресурса, а у једном тренутку ће и бити немогуће.

Тренутак када алгоритам опоравка не мора да чита *log* датотеку дубље се може окарактерисати са две особине [1]:

- сви претходни *log*-ови су написани од завршених трансакција (*undo* фаза алгоритма)
- бафери тих трансакција су написани на диск (*redo* фаза алгоритма)

Контролна тачка представља запис у *log* датотеци после којег се не мора читати даље јер је систем проверено потврдио да обе особине важе. Систем тривијално ово може да обезбеди након што је завршио са опоравком, а алгоритам који ово обезбеђује у осталим ситуацијама се може пронаћи у секцији 3.2.1. *LBDB* имплементира само логику за мирну контролну тачку (енг. *quiescent checkpoint*), али постоји и немирна контролна тачка (енг. *nonquiescent checkpoint*) која је флексибилнија, али и алгоритамски захтевнија.

Након опоравка, уместо писања контролне тачке, систем ради архивирање *log* датотеке. Архивирање је еквивалентна операција, а омогућава прегледније одржавање система тако што се *log* датотеке којима није потребан даљи приступ померају у засебан директоријум.

3.1.2.3 Алгоритам поништавања трансакције

Систем ради поништавање трансакције тако што пролази кроз *log* датотеку од краја ка почетку и поништава сваку модификациону операцију те трансакције, а стаје са проласком *log* датотеке када наиђе на операцију која означава почетак те трансакције.

3.1.3 Безбедан вишенитни приступ

Систем за безбедан вишенитни приступ је подсистем у оквиру трансакција који садржи други део логики због којег се сав приступ вредностима ради кроз трансакције. Примарно се брине о решавању конфликта конкурентног приступа истим блоковима. Обухвата *ACID* особину (енг. *isolation*).

Природа вишенитних приступа уводи недетерминистичан редослед операција који имплицира конфликте. Конфликт је када резултат две операције зависи од њиховог редоследа извршавања. Постоје две врсте конфликта: *write-write* конфликт и *read-write* конфликт. Код *write-write* конфликта, једна операција модификује неку вредност, па друга модификује исту вредност. Код *read-write* конфликта, једна операција чита неку вредност, а друга модификује исту ту вредност. Конфликти не могу да се десе код *read-read* операција или ако операције раде над вредностима које се налазе у различитим блоковима [1].

Спречавање конфликтујућих операција у систему се постиже преко система катанаца, који омогућавају коректан редослед приступа вредностима за читање и писање. Протокол закључавања у *LBDB* систему дефинише следећа правила рада са катанцима:

- пре читања вредности из блока, потребно је стећи *дељени* катанац за тај блок,
- пре писања вредности у блок, потребно је стећи *ексклузивни* катанац за тај блок,
- након завршетка трансакције, потребно је пустити све катанце за све блокове које је та трансакција закључала.

Поштовање оваквог протокола закључавања увек гарантује тачност рада са вредностима, али знатно смањује конкурентност система. Повећање конкурентности система се ради избором изолационог нивоа индивидуалних трансакција. Изолациони нивои трансакција повећавају конкурентност али жртвују тачност прочитаних вредности тако што управљају животним циклусом *дељених* катанца на различите начине [1].

Различити изолациони нивои су описани у секцији 3.1.3.1 због шире слике разумевања трансакција и потенцијалних проширења, али подсистем безбедног вишенитног приступа имплементира само серијализујући изолациони ниво трансакција.

3.1.3.1 Изолациони нивои трансакција

Табела 1: Различити изолациони нивои трансакција

Изолациони ниво	Проблеми	Пуштање <i>дељених</i> катанаца	<i>EOF</i> маркер
Серијализујући (енг. <i>serializable</i>)	не постоје	<i>дељени</i> катанци се држе до завршетка трансакције	постоји <i>дељени</i> катанац
Поновно читање (енг. <i>repeatable read</i>)	фантомске вредности	<i>дељени</i> катанци се држе до завршетка трансакције	не постоји <i>дељени</i> катанац
Читање само потврђених вредности (енг. <i>read committed</i>)	фантомске вредности, промењене вредности	<i>дељени</i> катанци се пуштају одмах након читања	не постоји <i>дељени</i> катанац
Читање и непотврђених вредности (енг. <i>read uncommitted</i>)	фантомске вредности, промењене вредности, “прљави” читања	не користе се <i>дељени</i> катанци	не постоји <i>дељени</i> катанац

Различити изолациони нивои се могу имплементирати и преко *MVCC* (енг. *multi version concurrency control*) приступа [3].

Изолациони нивои трансакција су конципирани тако да сваки ниво изолације решава све проблеме нивоа испод њега.

- Проблем “прљавог” читања је када трансакција *A* може да чита вредности која је трансакција *B* изменила, пре него што је трансакција *B* завршила.
- Проблем промењивих вредности је када трансакција *A* прочита неку вредност, трансакција *B* је модификује, трансакција *A* поново прочита исту вредност и добије вредност коју је трансакција *B* модификовала уместо вредности коју је трансакција *A* првобитно прочитала.
- Проблем фантомских вредности је када трансакција *A* прочита све вредности неког блоковског опсега, трансакција *B* дода нову вредност и прошири тај блоковски опсег са новим блоком, и уместо да трансакција *A* при поновном читању свих вредности добије исту листу првобитних вредности, добије и нову вредност из новог блока коју је трансакција *B* додала.

Пошто је ниво грануларности закључавања на нивоу блока, фантомска читања у оквиру трансакције *A* и датотеке *D* се могу десити само ако трансакција *B* на крају *D* дода нови блок пре завршетка трансакције *A*. Спречавање овога се реализује добијањем виртуелног *дељеног* катанца над *EOF* (енг. *end of file*) маркером датотеке *D* од стране трансакције која чита податке (трансакција *A*) и добијањем виртуелног *ексклузивног* катанца над *EOF* маркером датотеке *D* од стране трансакције која додаје нови блок (трансакција *B*). *B* неће моћи да дода нови блок у *D* док *A* држи виртуелни *дељени* катанац над *EOF* маркером.

Споменути изолациони нивои трансакција се односе само на операције које читају вредности. Операције које модификују вредности увек морају поштовати коректно добијање *ексклузивних* катанаца. Трансакције на индивидуалном нивоу могу толерисати непрецизне податке, али када би се добијање *ексклузивних* заобишло, цела база података би постала неупотребљива [1].

3.1.3.2 Каталог катанаца

Инстанца класе `LockTable` је дељена за све трансакције у систему. У њој се реализују механизми праћења постојања катанаца за све блокове. *Дељени* катанац за неки блок је могуће стећи без обзира на постојање других дељених катанаца, али *ексклузивни* катанац за неки блок је могуће стећи само када не постоји ниједан други катанац било које врсте.

Ако се деси да трансакција није успела да закључа *жељени* блок (било са *дељеним* или *ексклузивним* катанцем) враћа се грешка клијенту. Систем нема механизам за детекцију застоја, већ чека да прође одређени временски период, после ког сматра да

је немогуће закључати жељени блок. Такође, систем не може да спречи ситуацију где трансакције чекају једне на друге у кругу (енг. *deadlock*).

3.1.3.3 Управљање катанцима на нивоу индивидуалних трансакција

Менаџер конкурентности (*ConcurrencyManager* класа) садржи скуп једноставних алгоритама који интерагују са каталогом катанца да би трансакцијама на индивидуалном нивоу омогућили управљање катанцима. Свака трансакција има свој менаџер конкурентности.

3.2 Трансакције и клијенти

Пошто свака операција у систему мора бити извршена у оквиру неке трансакције, клијентима *LBDB* система је потребно доделити коректне трансакционе објекте. Менаџер трансакција управља животним циклусом трансакција и везује их за клијенте. Менаџер трансакција је један од три главна подсистема *LBDB* система обраде упита.

Нова трансакција почиње чим се претходна трансакција за коју је клијент везан заврши. Прва трансакција се додељује клијенту приликом његовог повезивања на систем. Подразумевани режим завршетака трансакција је аутоматско завршавање (енг. *autocommit*) у ком се свака трансакција састоји од тачно једне операције. Други режим завршетака трансакција је ручни режим, у ком се трансакција може састојати од више операција, и у том случају крај трансакције означава операција потврде или поништавања која се добија од клијента.

Да би се постигло перзистирање истог трансакционог објекта кроз више операција, потребно је пратити све актуелне трансакционе објекте у систему и јединствено описати сваког клијента. За истог клијента се операције морају извршавати у оквиру његовог трансакционог објекта докле год трансакцији не дође крај. Јединствени идентификатор клијента представљен је његовом сесијом. На апстракционом нивоу управљања трансакцијама, сесија није ништа више него број, али на вишим слојевима је потребно генерисати ту сесију коректно. Детаљи генерисања сесије се налазе у секцији [8.2.1.1](#).

3.2.1 Алгоритам записа мирне контролне тачке

Да би мирна контролна тачка била записана, ниједна трансакција не сме бити активна у систему, а ово је евидентно због прве особине мирне контролне тачке. Коректан запис мирне контролне тачке се извршава следећим алгоритмом [1]:

- менаџер трансакција престаје да прихвата нове трансакције од клијената
- менаџер трансакција чека да сви клијенти заврше са својом трансакцијом, било она ручна или аутоматска
- менаџер трансакција записује све бафере на диск
- менаџер трансакција додаје запис мирне контролне тачке у *log* датотеку
- менаџер трансакција поново прихвата нове трансакције од клијената

Глава 4

Метаподаци

Метаподаци су подаци који описују друге податке. Иако су подаци организовани преко слогова (поглавље 2), систем им не може приступити ако се не побрине о перзистирању структуре тих слогова. Праћење дистрибуције вредности је корисно приликом прављења ефикасног начина добављања слогова. Подаци који дефинишу структуру слогова и подаци о расподели вредности су метаподаци којима систем барата.

4.1 Каталогске табеле

Метаподатаке које систем чува да би омогућио рад са табелама су подаци о постојећим табелама и како колоне тих табела изгледају. Ти подаци се чувају у оквиру системских табела и оне се називају каталогске табеле. Каталогска табела *tablecatalog* чува податке о постојећим табелама, док каталогска табела *fieldcatalog* чува податке о физичкој структури слога неке табеле.

Табела 2: Шема *tablecatalog* табеле

Колона	Тип	Опис
<i>tableid</i>	<i>Integer</i>	јединствени идентификатор табеле
<i>tablename</i>	<i>String</i>	име табеле
<i>slotsize</i>	<i>Integer</i>	величина једног слога те табеле у бајтовима

Табела 3: Шема *fieldcatalog* табеле

Колона	Тип	Опис
<i>type</i>	<i>Integer</i>	тип колоне записан као нумеричка вредност
<i>runtime length</i>	<i>Integer</i>	максимална физичка дужина вредности овог поља
<i>offset</i>	<i>Integer</i>	позиција вредности те колоне
<i>tableid</i>	<i>Integer</i>	јединствени идентификатор табеле у којој се колона налази
<i>fieldname</i>	<i>String</i>	име колоне
<i>nullable</i>	<i>Boolean</i>	да ли вредности колоне могу бити <i>NULL</i> вредност

Каталогске табеле се креирају приликом иницијализације система. Битно је напоменути да се каталогске табеле перзистирају на исти начин као и све остале табеле у систему, што значи да ће оне садржати и слоге које описују њих саме. Тиме што се каталогске табеле перзистирају исто као и корисничке, системским табелама се може приступити путем стандардних механизма релационих оператора (поглавље 5).

Сви идентификатори у систему (имена колона, табела, ...) се имплицитно конвертују тако да садрже само мала слова.

4.2 Статистички подаци

Приступ истим слоговима табела се често може извршити на више различитих начина, али неки начини могу бити знатно мање ефикасни од осталих. Апстрактциони ниво управљања метаподацима је дужан да обезбеди статистичке метаподатке који помажу при процени времена извршавања одређених начина приступа. Сам посао конструисања ефикасног начина приступа је брига подсистема планирања (поглавље 7).

Статистички метаподаци неке табеле укључују:

- број блокова табеле
- број слогова у табели
- број различитих вредности колона табеле
- број *NULL* вредности колона табеле

4.2.1 Рачунање статистичких података

Приликом иницијализације система, рачунају се статистички метаподаци за сваку табелу у систему, а сваких 100 позива добављања метаподатака за било коју табелу се освежавају статистички метаподаци за све табеле. Овај начин освежавања није идеалан јер паузира систем док се рачунање статистичких метаподатака не заврши и ово ограничење је детаљније описано у секцији 11.2.3.

Број блокова и број слогова табеле се тривијално добијају проласком кроз сваки слог. Број различитих вредности колоне табеле није могуће израчунати прецизно, јер је за то потребно чување свих јединствених вредности те колоне у радној меморији. Мале непрецизности неће утицати на процену времена извршавања операција, па је искоришћена пробабилистичка структура података *HyperLogLog* [4] која решава *count distinct* проблем и она не чува све јединствене вредности у радној меморији. Једна таква структура се додељује за сваку колону. Бројање *NULL* вредности колона табеле се своди на чување бројача за сваку колону.

4.3 Приступ метаподацима

Менаџер метаподатака је главно место приступа свим метаподацима. Састоји се из менаџера метаподатака табела и менаџера статистичких метаподатака. Менаџер метаподатака је један од три главна подсистема *LBDB* система обраде упита.

Глава 5

Релациони оператори

SQL програмски језик је језик декларативног типа. То значи да се преко њега специфицира шта треба да се уради са подацима (добављање, филтрирање, модификација, ...), али за разлику од процедуралних програмских језика, не специфицира се и како. Мост између декларативне природе SQL језика и потребе дефинисања начина приступа подацима је решен имплементацијом *relacione algebre* [5]. LBDB систем преводи код SQL програмског језика у стабло оператора релационе алгебре.

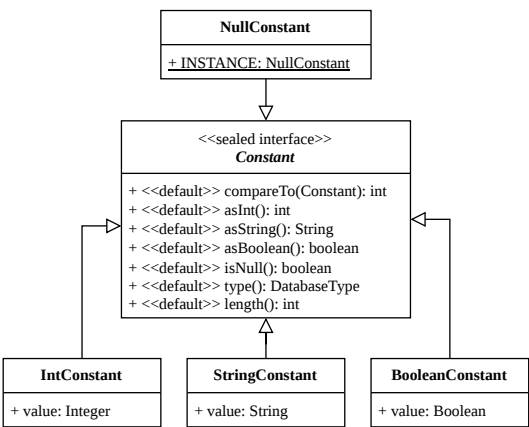
Релација R је скуп торки облика $(d_1, d_2, \dots, d_j)$ где за сваку компоненту d_k торке d_j важи $d_k \in D_k$, где је D_k домен који дефинише скуп свих дозвољених вредности за ту компоненту. У релационим базама података, табела се моделује као релација, док оператори релационе алгебре пресликавају једну или више релација у нову релацију као резултат примењене трансформације. Торке се мапирају на слогове табела.

5.1 Виртуелна машина

Виртуелна машина LBDB система представља окружење извршавања логичких и аритметичких операција и састоји се од класа које моделују компоненте тих операција.

5.1.1 Константе

Све вредности са којима релациони оператори баратају представљене су Constant класом. Она омогућава систему да дефинише генеричко понашање за све различите типове подржане у систему. Релациони оператори увек враћају Constant објекте на свом излазу. Имплементира Comparable<Constant> интерфејс зарад лаког поређења.

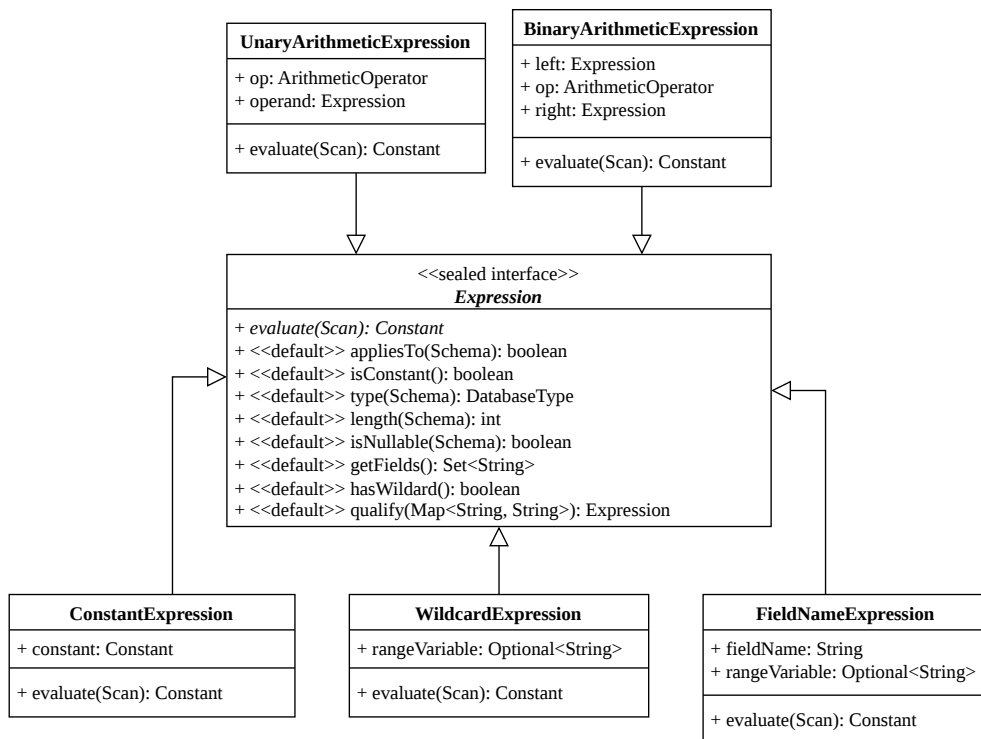


Слика 5: Хијерархија константи

Интерфејс константе је дефинисан *sealed interface Java* конструктом, због његове одличне компатибилности са *switch* синтаксом. Конкретне константе су представљене *Java record* структуром. `NullConstant` нема никакве податке инстанце, јер су све `NULL` вредности идентичне у систему, па се на свим местима користи иста инстанца.

5.1.2 Изрази

Све аритметичке операције које систем евалуира су представљене `Expression` класом. Евалуација израза увек производи `Constant` објекте. Изрази се састоје од произвољне комбинације аритметичких оператора (+, -, *, /, ^), заграда, константи и идентификатора колона табеле. Интерфејс израза је дефинисан *sealed interface Java* конструктом, због његове одличне компатибилности са *switch* синтаксом. Конкретни изрази су представљени *Java record* структуром.



Слика 6: Хијерархија израза

Евалуација израза представља процес рачунања константе која представља вредност тог израза за неки слог табеле.

`BinaryArithmeticExpression` је израз који у себи садржи изразе и омогућава креирање стабла израза, где се прво евалуирају његови чланови, па онда он. `UnaryArithmeticExpression` има исту функцију, али за префиксне операторе (+, -) и има само један члан.

`WildcardExpression` суштински не представља израз, али због начина парсирања изрази (секција 6.2.2.8), третира се као један. Служи као заменски члан (енг. *placeholder*) свих колона упитаних табела. Евалуација је забрањена операција и онемогућена је изузетком.

`FieldNameExpression` је израз који идентификује виртуелну колону неке табеле упита, са опционим преименовањем табеле и евалуира се на вредност те колоне. `ConstantExpression` се евалуира директно на константу за коју је везан и независан је од табела.

5.1.3 Чланови

Члан (енг. *term*) енкапсулира логику за поређење константи два евалуирана изрази. Поређење две константе зависи од оператора поређења који могу бити `=`, `!=`, `>`, `>=`, `<`, `<=`, `IS`, `IS NOT`. Разлика између `=` и `IS` (исто тако и између `!=` и `IS NOT`) је у томе како се понашају са `NULL` вредностима. `IS` омогућава поређење са `NULL` вредностима, док `=` то не подржава. Евалуација члана над неким релационим оператором, за разлику од евалуације изрази, враћа само да ли члан важи или не.

5.1.4 Предикати

Предикати уланчавају чланове логичким операторима. Евалуација предиката над неким релационим оператором функционише исто као и евалуација једног члана, али између тих чланова стоје различите логичке операције. *LBDB* систем подржава само `AND` логичке операторе између чланова и ово ограничење је детаљније описано у секцији 11.2.6.

5.1.5 Генерализација евалуације

Изрази и предикати имплементирају `Evaluatable` интерфејс који дефинише све операције потребне за њихову евалуацију и каснију проверу валидности. Овај интерфејс омогућава систему да се не брине о томе шта се евалуира, већ само о константи коју ће добити, што даље омогућава да се вредности пројектованих колона (секција 5.2.2.10) добијају и преко евалуације изрази и преко евалуације предиката.

```
public interface Evaluatable {
    Constant evaluate(Scan scan);
    boolean isConstant();
    DatabaseType type(Schema schema);
    int length(Schema schema);
    boolean isNullable(Schema schema);
    Set<String> getFields();
    boolean hasWildcard();
    Evaluatable qualify(Map<String, String> aliases);
}
```

Листинг 2: `Evaluatable` интерфејс

5.2 Структура релационих оператора у систему

За извршавање наредби дефинисаних *SQL* језиком, често је потребно применити више релационих оператора. Примена више релационих оператора се ради њиховим уланчавањем у стабловску структуру података и због овога се каже да систем извршава “стабло” релационих оператора.

5.2.1 Проточна обрада

Релациони оператори подржани у систему имају две заједничке особине [1]:

- генеришу слоге по један
- не чувају генерисане слоге и не чувају никакве међурезултате

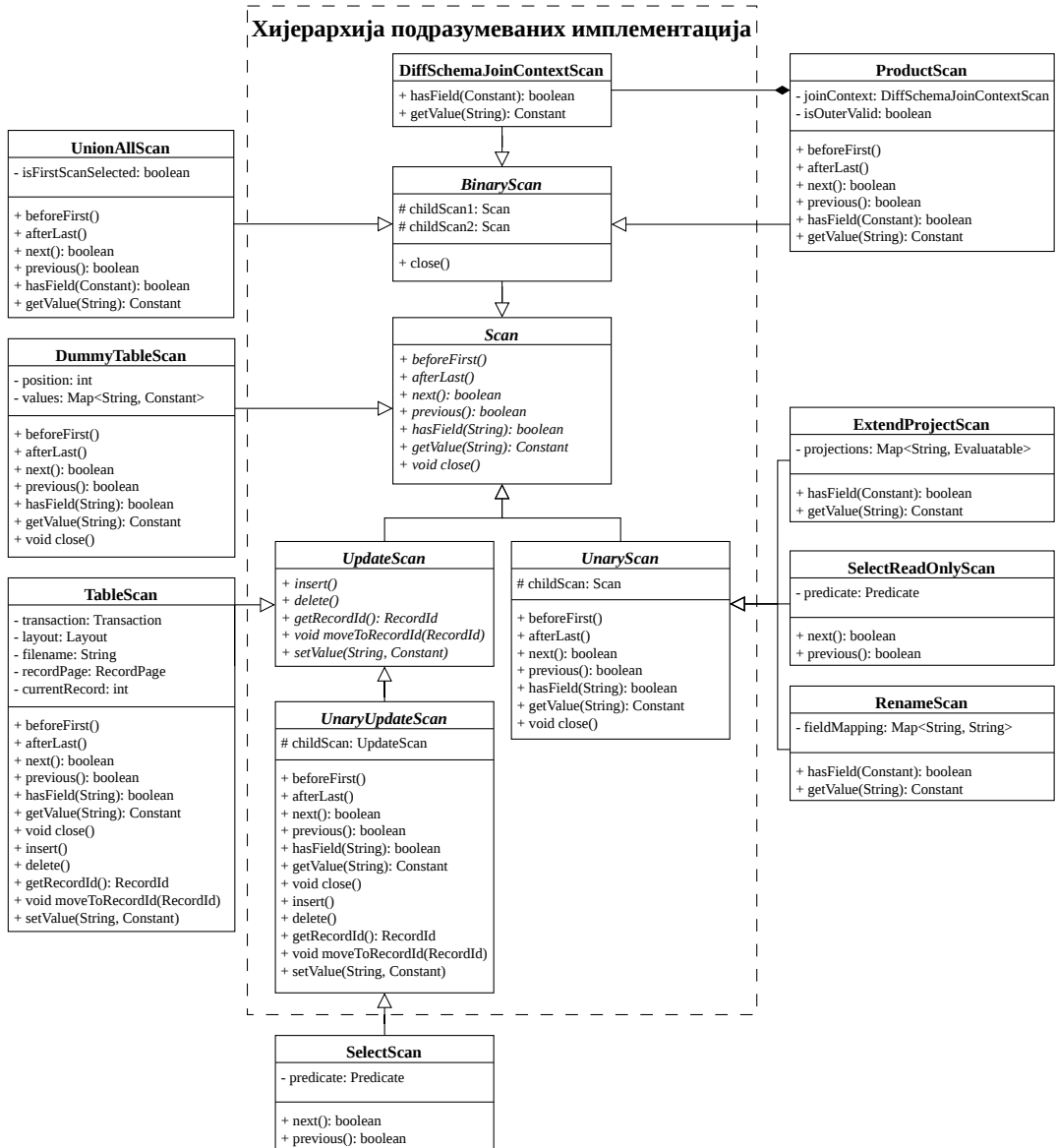
Захтев за извршавање неке операције над стаблом оператора почиње од корена стабла, који формира резултат уз помоћ чворова испод њега, али некад и директно. Овим начином, захтев пролази кроз цело стабло оператора. Враћа се грешка клијенту уколико барем један чвор није успео да формира резултат због грешке приликом извршавања.

Комбинација две наведене особине уз делегацију операција се зове проточна обрада (енг. *pipelined processing*). Коришћење проточне обраде у многим сценаријима не додаје никаквно додатно *U/I* оптерећење, па ју је погодно користити.

Предност проточне обраде што не чува међурезултате је управо и њена мана за одређене операције. Материјализована обрада (енг. *materialization, materialized processing*) омогућава и чување међурезултата па може решити ову ману, али долази са својим проблемима. Такође, неке операције попут груписане агрегације, враћање само јединствених слогова и произвољно сортирање није могуће обавити без материјализоване обраде. Потребно је користити оба начина обраде у систему за најбоље резултате, али *LBDB* систем имплементира само проточну обраду.

5.2.2 Хијерархија имплементације релационих оператора

Најопштија подела релационих оператора је на оне који само читају податке (енг. *read-only*) и на оне који могу да модификују податке. Ова дистинкција је направљена да се обезбеди сигурност од погрешне примене оператора у времену компајлирања кода (енг. *compile time*). Хијерархија подразумеваних имплементација се састоји од разних костурских имплементација које се користе за лако дефинисање новог релационог оператора. Остале класе ван хијерархије подразумеваних имплементација представљају различите релационе операторе подржане у систему и објашњене су испод.



Слика 7: Хијерархија имплементације релационих оператора

5.2.2.1 Scan

Scan апстрактна класа дефинише операције неопходне за пролазак кроз све вредности резултујуће виртуелне табеле. Сваки релациони оператор имплементира бар ове операције. Враћање вредности се ради искључиво кроз Constant објекте. Scan се може, али не мора, мапирати на физичку табелу. Имплементира AutoCloseable интерфејс који омогућава *RAII* (енг. *Resource Acquisition Is Initialization*) шаблон, али не пружа имплементацију логике ослобађања ресурса.

5.2.2.2 UpdateScan

Резултујуће виртуелне табеле релационих оператора који дозвољавају модификацију вредности се морају мапирати на физичке табеле, јер нема смисла мењати виртуелне вредности. То значи да за сваки виртуелни слог r у стаблу модификујућих оператора, мора да постоји r' који има идентичну структуру, позицију и вредности у датотеци табеле. UpdateScan класа дефинише операције промене вредности колона неког слога, уметања новог слога и брисања слога.

5.2.2.3 UnaryScan

UnaryScan садржи подразумеване имплементације произвољног релационог оператора који трансформише **једну** релацију, то јест једну табелу. Сваки позив подразумеване имплементације је само прослеђен оператору испод. Оваква структура омогућава да оператори који наслеђују ову класу редефинишу само методе које су потребне за функционисање тог оператора и избегава се дуплирање истог кода.

5.2.2.4 UnaryUpdateScan

UnaryUpdateScan садржи подразумеване имплементације произвољног релационог оператора који може да модификује вредности физичке табеле. Функционише исто као и UnaryScan и има све подразумеване имплементације из њега.

5.2.2.5 BinaryScan

BinaryScan садржи подразумеване имплементације произвољног релационог оператора који трансформише **две** релације, то јест две табеле. Пошто оператори који раде над две табеле немају толико међусобних преклапања, BinaryScan имплементира само `close()` методу која ослобађа ресурсе оба детета.

5.2.2.6 DiffSchemaJoinContextScan

DiffSchemaJoinContextScan има сличну улогу као BinaryScan, али само за операторе који врше мултипликативно обједињавање две табеле и енкапсулира коректан приступ колонама из две шеме које немају преклапање.

5.2.2.7 TableScan

TableScan није прави релациони оператор зато што његова имплементација не ради трансформације табеле, већ пружа логику за добављање и модификацију физичких вредности. Служи као омотач око објеката странице слогова и задужен је за конструисање нових повезаних објеката странице слогова. Пошто пружа иницијалне вредности које ће даље бити трансформисане, увек се налази на дну стабла релационих оператора.

5.2.2.8 DummyTableScan

DummyTableScan није прави релациони оператор зато што његова имплементација не ради трансформације табеле, већ пружа логику за добављање и навигацију јединог слога виртуелне табеле која се конструише у SQL упитима који не раде ни са једном физичком табелом.

5.2.2.9 SelectScan

SelectScan имплементира оператор генерализоване селекције $\sigma_{\varphi}(R)$ из релационе алгебре, где је σ име селекционог оператора, φ је формула филтера, а R је релација над којом се оператор примењује. Редифинише само методе за пролазак кроз слоге, јер је то једино неопходно да се укину слогови који не пролазе филтер. Садржи Predicate објекат помоћу ког филтрира. Може да се користи и у контекстима модификујућих стабала оператора и у контекстима *read-only* стабала оператора и због тога постоји и SelectReadOnlyScan варијанта која има исту функцију.

5.2.2.10 ExtendProjectScan

ExtendProjectScan имплементира оператор пројекције $\Pi_{a_1, \dots, a_n}(R)$ из релационе алгебре, где је Π име пројекционог оператора, а a_n представља израз чија евалуација производи вредност компоненте n . Оператор пројекције који класа имплементира није стриктно оператор пројекције формално дефинисан у релационој алгебри, зато што дозвољава креирање нових колона са изразима који ће бити израчунати у тренутку извршавања стабла, а не повучене директно из физичке табеле. Дозвољава и додељивање произвољног имена изразима колона, али се то не треба помешати са оператором преименовања. Редифинише методе добављања вредности и провере постојања колоне неког имена.

5.2.2.11 RenameScan

RenameScan имплементира оператор преименовања $\rho_{a_n/b_n}(R)$ из релационе алгебре, где је ρ име оператора преименовања, а b_n представља ново име компоненте a_n . Разликује се од формалне дефиниције оператора преименовања из релационе алгебре јер дозвољава n преименовања одједном да би се избегло уланчавање истих оператора. Битно је напоменути да оператор преименовања омогућава приступ колони са именом a кроз име b , за разлику од оператора пројекције који само додељује име неком изразу. Редифинише методе добављања вредности и провере постојања колоне неког имена.

5.2.2.12 ProductScan

ProductScan имплементира операцију декартовог производа релација R и S : $R \times S$. Резултујућа релација представља виртуелну табелу која има $|R| * |S|$ торки, а свака торка се састоји од свих компоненти обе релације. Операција декартовог производа је мултипликативна, а ProductScan захтева да табеле немају колоне са истим именом,

па се користи `DiffSchemaJoinContextScan`. Итерација кроз резултујућу табелу након `ProductScan` оператора се врши тако што се за сваки слог прве табеле, пролази кроз све слоге друге табеле. Редифинише методе добављања вредности, провере постојања колоне неког имена и све навигационе методе.

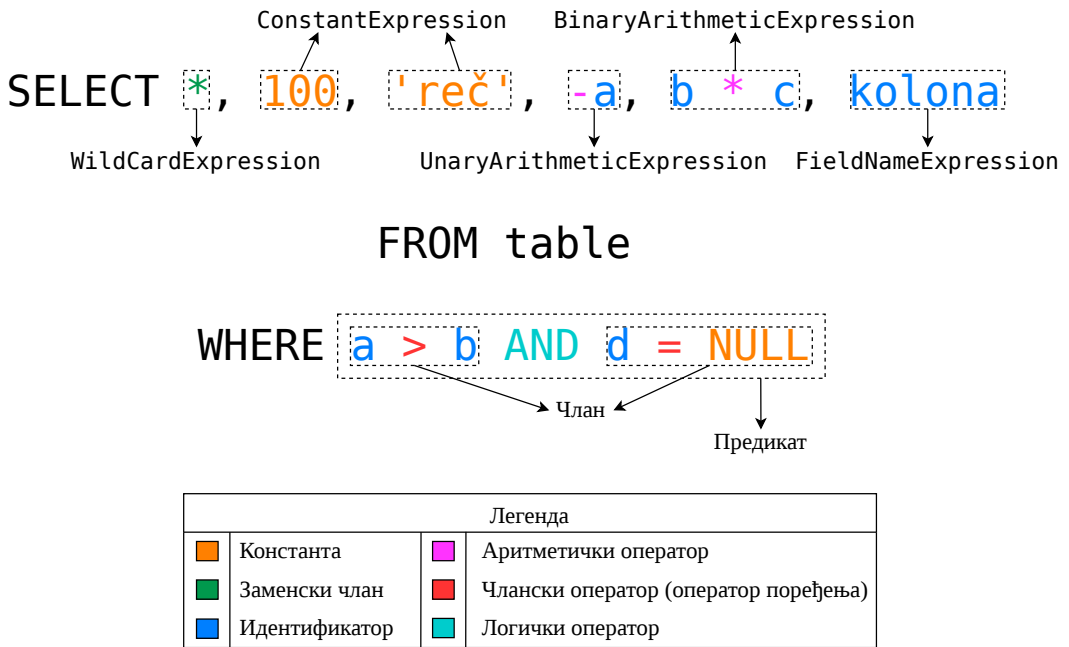
5.2.2.13 UnionAllScan

`UnionAllScan` имплементира операцију креирања релације које садржи све торке релација R и S . Не брише дубликате. Захтева да су компоненте торки обе релације истог типа и да обе релације имају исти број компонената по торки. По `SQL` стандарду, колонама друге табеле се приступа по именима прве. Операција уније је адитивна. Итерација кроз резултујућу табелу након `UnionAllScan` оператора се врши тако што се прво пролази кроз све слоге прве табеле, па се пролази кроз све слоге друге табеле. Редифинише методе добављања вредности, провере постојања колоне неког имена и све навигационе методе.

5.2.3 Примери

Следећи примери визуелно показују различите делове стабла релационих оператора и како се неке функционалности понашају. Примери не осликавају стабла релационих оператора које би `LBDB` систем направио, већ служе да покажу уклапање компонената виртуелне машине, проточну обраду и како се `Scan` објекти ослањају једни на друге.

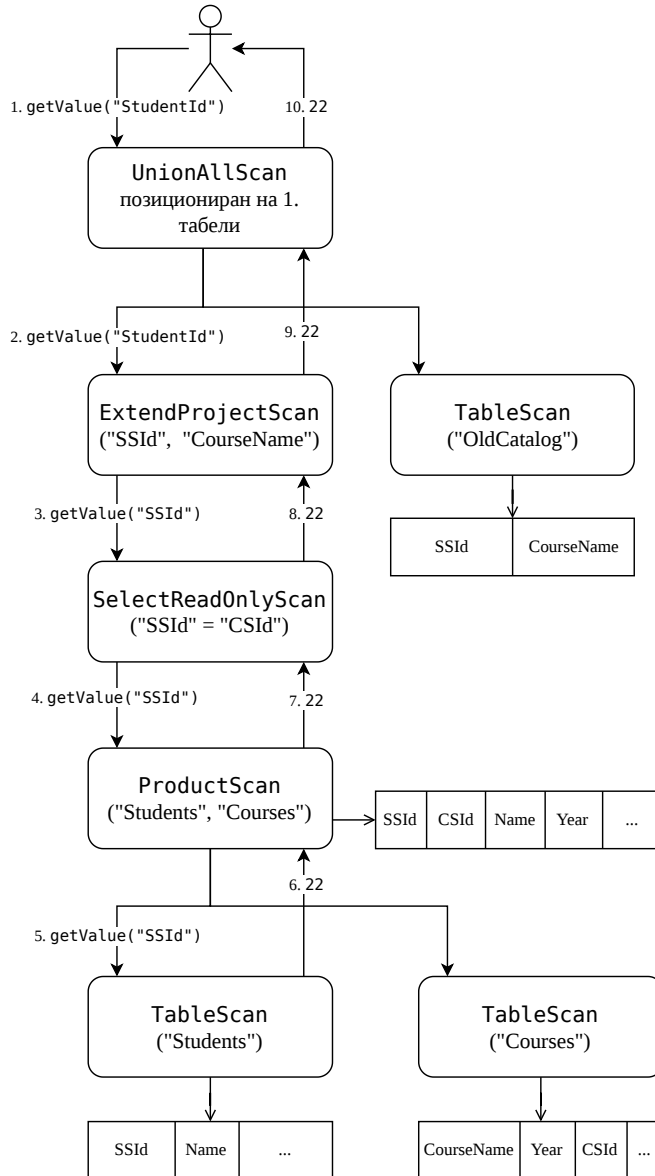
5.2.3.1 Пример изгледа компоненти виртуелне машине



Слика 8: Пример наредбе у `SQL` језику где се виде све компоненте виртуелне машине

5.2.3.2 Пример позива `getValue()` методе

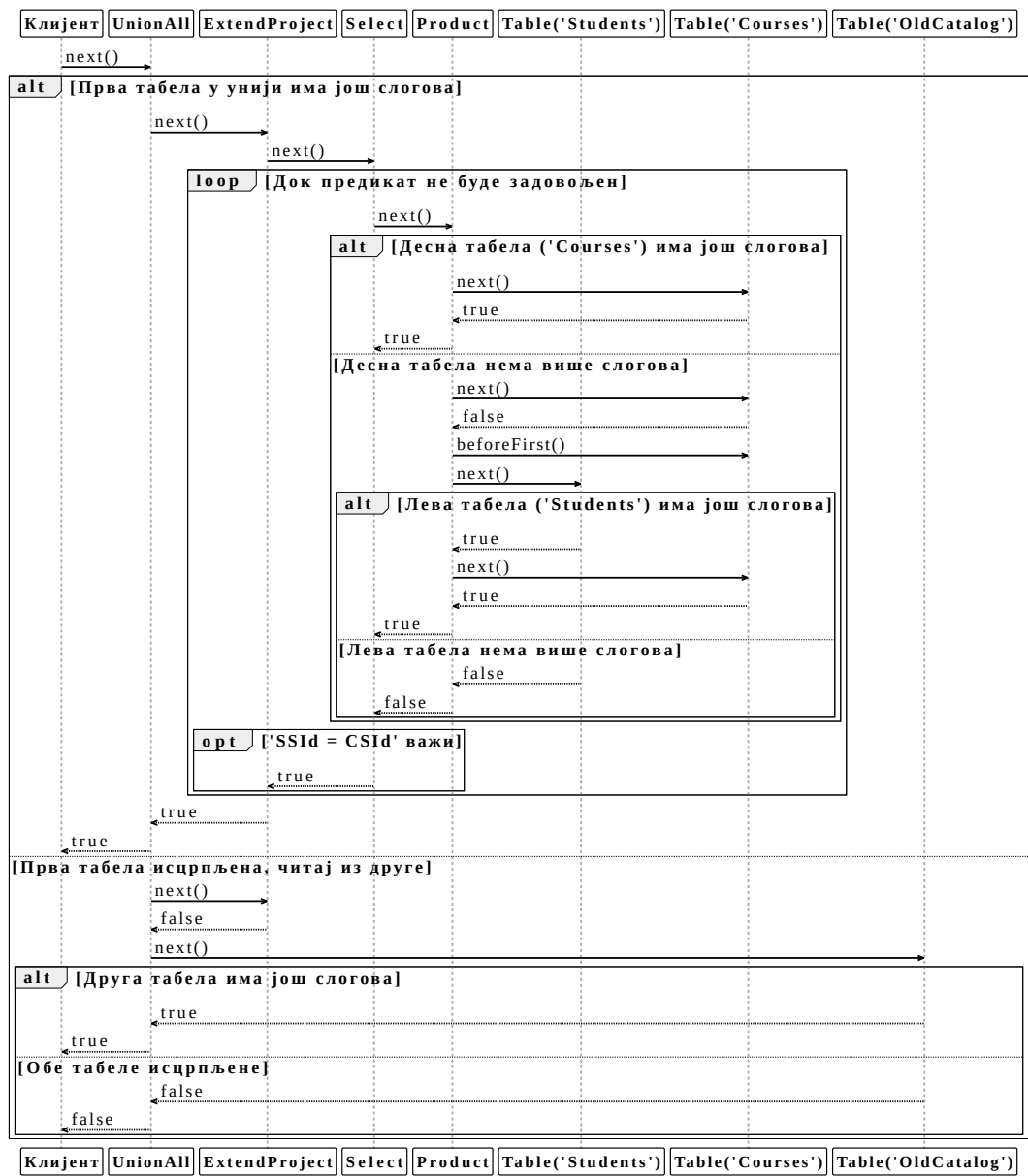
Прате се кораци позива `getValue()` методе, за виртуелну колону "StudentId". Унион оператор, који је скроз на врху, спаја две табеле: табелу која моделује застарели начин вођења евиденције и виртуелну табелу која је резултат подстабла оператора који пројектују исте колоне као и у табели застареле евиденције. `ExtendProjectScan` је преименовао "SSId" у "StudentId". Уз `TableScan` и `ProductScan` операторе стоји и упрошћена шема.



Слика 9: Пример позива `getValue()` у конкретном стаблу релационих оператора

5.2.3.3 Пример позива next() методе

Овај пример користи исто релационо стабло као и претходни пример. Помоћу дијаграма секвенце се описује процедура итерације кроз стабло са свим врстама оператора који мењају подразумевану итерацију.



Слика 10: Дијаграм секвенце next() у конкретном стаблу релационих оператора

Глава 6

Парсирање

Клијенти шаљу *SQL* наредбе у текстуалном формату, али систему комад текста нема никакво инхерентно значење. Подсистем за екстракцију информација из текста наредбе се назива парсер.

Није сваки комад текста валидна *SQL* наредба, али његова валидност се може поделити на два слоја [1]:

- синтаксичка валидност, где синтакса представља скуп правила која дефинишу могуће операције по некој граматици,
- семантичка валидност, која је испуњена ако је нека операција валидна у контексту података које користи (имена табела, имена колона, ...)

Парсирање конструише апстрактно синтаксичко стабло (енг. *abstract syntax tree*, *AST*) које се мапира на подржане операције и служи за проверу **само** синтаксичке валидности операције. Провера семантичке валидности је део планера (секција 7.2).

6.1 Токенизатор

Блокови текста се састоје од индивидуалних карактера. Већина индивидуалних карактера садржи јако мало значења када се обрађују независно и због тога се уводи систем који групише повезане карактере. Скуп груписаних карактера који заједно имају висок степен значења се зову токени. Карактери који се обрађују сами су исто описани као токени, јер је битно да је сваки токен именован.

`Tokenizer` класа садржи логику претварања блока текста у токене одговарајућег типа и имплементирана је преко `Java Iterator` интерфејса. `Token` је дефинисан *sealed interface Java* конструктом, због његове одличне компатибилности са `switch` синтаксом. Сви токени у систему су груписани у следеће категорије:

- кључне речи *SQL* језика,
- идентификатори,
- симболи,
- бројеви,
- *String*-ови,
- невалидни карактери,
- EOF токен.

Свака категорија токена имплементира `Token` интерфејс и представљена је *record* или *enum Java* структуром.

6.2 Парсер

Граматика неког језика представља скуп правила које описују све легалне комаде текста, које неки систем подржава. Синтаксичке категорије су концепти са којима граматика барата. Представљају чворове синтакстичког стабла и садрже се од других синтаксичких категорија и токена.

Врста парсирања која је имплементирана се зове *recursive descent* парсирање. У *recursive descent* парсирању, граматика се проверава од горе ка доле. Улазна тачка је корен синтаксичког стабла и за свако подстабло, то јест граматичко правило, постоји функција која обрађује то правило. Функције се често позивају рекурзивно да би обрадили веће целине, па је тако овај начин парсирања и добио име. Свака функција обраде правила, на било ком нивоу, се мапира на једну синтаксичку категорију.

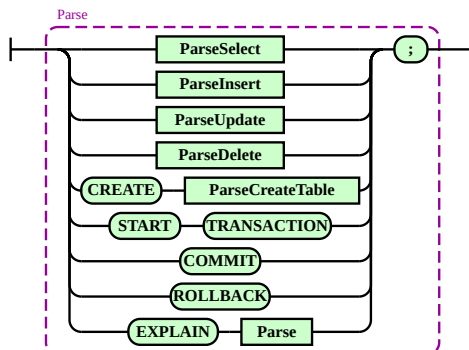
6.2.1 Искизи

Успешно парсирање неког блока текста који представља *SQL* операцију резултује у исказу, који садржи све неопходне податке да се та операција изврши. Исказ (Statement класа) је дефинисан *sealed interface Java* конструктом, због његове одличне компатибилности са *switch* синтаксом. Сваки исказ је представљен *Java record* структуром и наслеђује *Statement*.

6.2.2 Синтаксичке категорије *SQL* операција

Функције које генеришу исказе представљају синтаксичке категорије највишег апстракционог нивоа и груписане су у различите *Java* датотеке. Свака датотека садржи све синтаксичке категорије нижег апстракционог нивоа потребне да се исказ успешно обради.

6.2.2.1 Parse

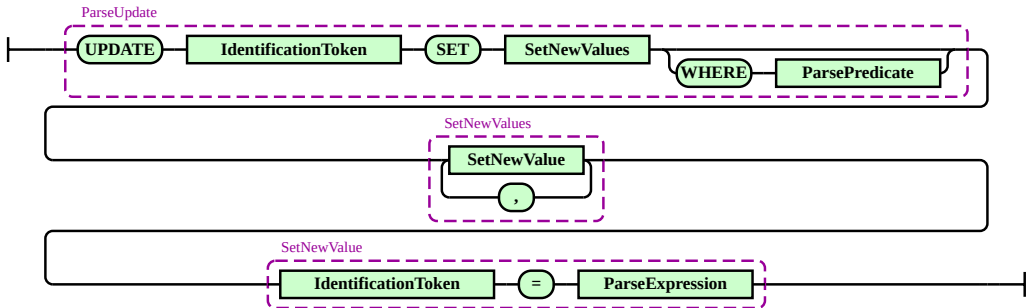


Слика 11: Граматика Parse синтаксичке категорије

Parse представља главну синтаксичку категорију и групише све остале синтаксичке категорије. Омогућава и EXPLAIN наредбу, која генерише опис наредбе која ће се извр-

шити. Искизи управљања животним циклусима трансакција се исто парсирају овде јер су превише једноставни да би се правила посебна синтаксичка категорија за њих. Враћа Statement објекат.

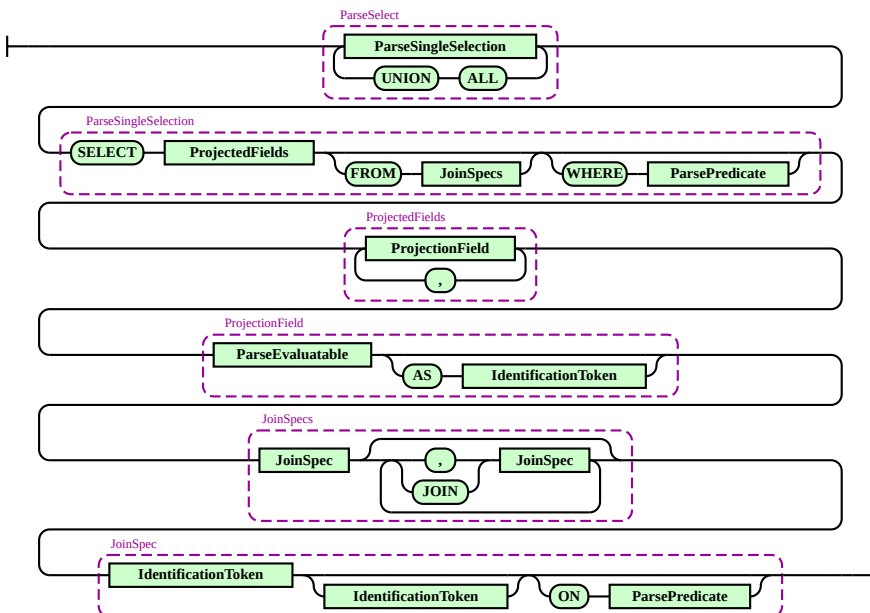
6.2.2.2 ParseUpdate



Слика 12: Граматика ParseUpdate синтаксичке категорије

ParseUpdate представља наредбу модификовања података неке табеле. Може садржати произвољан број нових додела вредности, али свако поље у свим доделама вредности мора постојати у споменутој табели. Могуће је модификовати само подскуп слогова, тако што се дефинише услов претраге. Враћа UpdateStatement објекат.

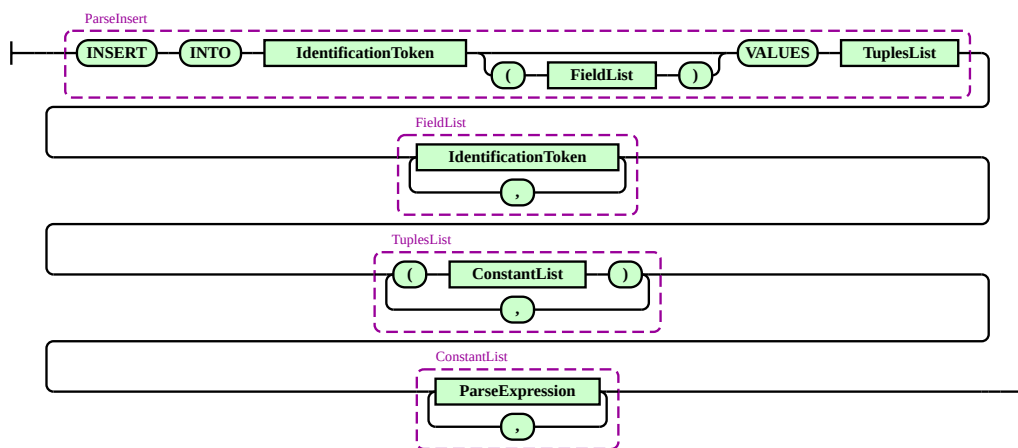
6.2.2.3 ParseSelect



Слика 13: Граматика ParseSelect синтаксичке категорије

ParseSelect представља наредбу упита (енг. *query*) података. Може садржати произвољан број пројектованих колона, где је свака колона представљена као било шта што може да се евалуира и којој се може доделити ново име (уз AS кључну реч). Подржава филтрирање на основу услова претраге. Упит може бити над правим табелама или над виртуелном табелом која садржи један слог. Уланчавање табела се може радити на два начина: само навођење табела одвојене зарезом или преко JOIN кључне речи где се услов уланчавања уписује одмах. Услов уланчавања написан у JOIN секцији се само додаје на услов претраге. Враћа SelectStatement објекат.

6.2.2.4 ParseInsert



Слика 14: Граматика ParseInsert синтаксичке категорије

ParseInsert представља наредбу уметања нових слогова у неку табелу. Листа поља не мора бити дефинисана, узима се подразумевани редослед који је направљен током креирања те табеле. Изрази морају бити константни, то јест њихова евалуација не сме зависити од вредности које не могу да се израчунају без приступа табелама. Враћа InsertStatement објекат.

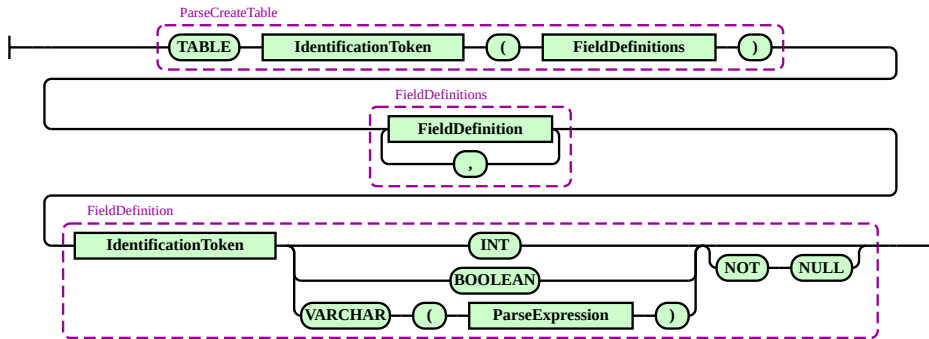
6.2.2.5 ParseDelete



Слика 15: Граматика ParseDelete синтаксичке категорије

ParseDelete представља наредбу брисања слогова из табеле. Могуће је обрисати само подскуп слогова, тако што се дефинише услов претраге. Враћа DeleteStatement објекат.

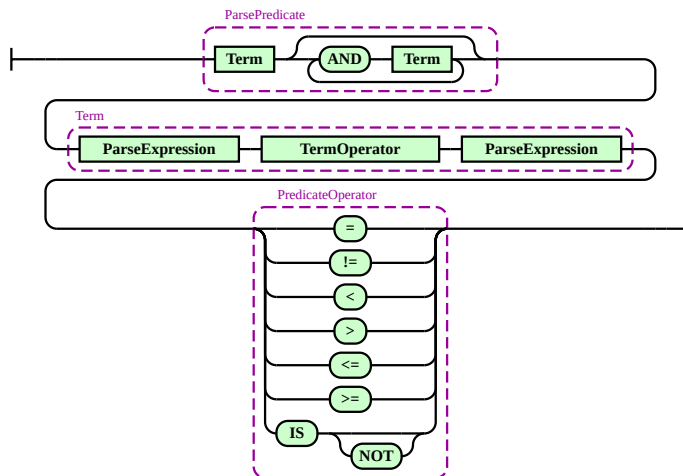
6.2.2.6 ParseCreateTable



Слика 16: Граматика `ParseCreateTable` синтаксичке категорије

`ParseCreateTable` представља наредбу креирања нове табеле. Поље може бити једно од типова подржаних у систему (листинг 1), а за *String* (`VARCHAR`) тип се мора дефинисати и максимална дужина, која мора бити константан израз. Ограничење да слогови за неку колону не смеју имати *NULL* вредности је опционо и дефинише се након типа колоне. Враћа `CreateTableStatement` објекат.

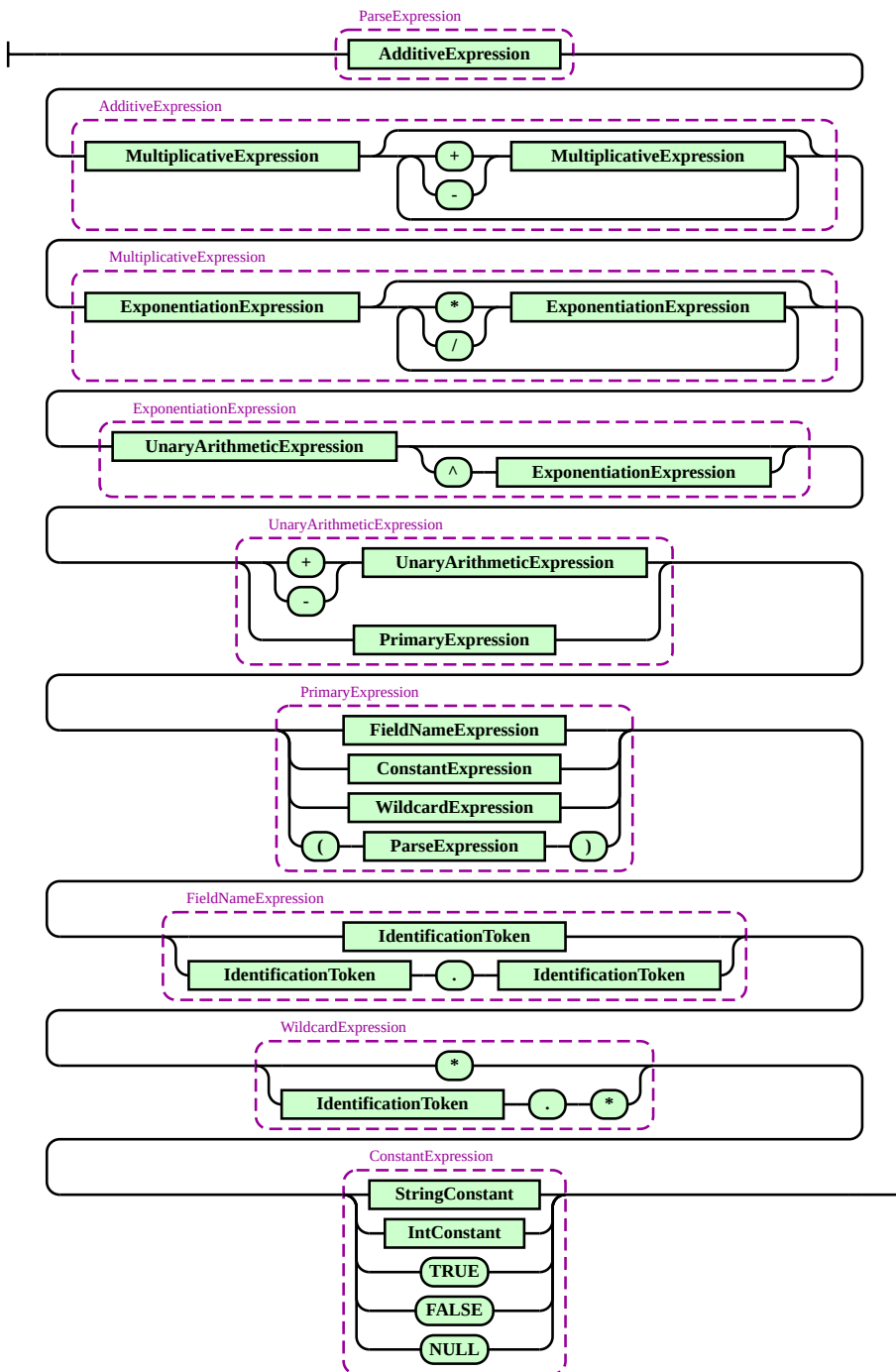
6.2.2.7 ParsePredicate



Слика 17: Граматика `ParsePredicate` синтаксичке категорије

`ParsePredicate` је специјална врста синтаксичке категорије која не производи исказ, већ служи за креирање синтаксичких стабала предиката. Парсирају се изрази и операције поређења од којих се чланови састоје, а затим се уланчавају чланови да би се формирао предикат. Враћа `Predicate` објекат. Не подржава чланове без оператора поређења.

6.2.2.8 ParseExpression

Слика 18: Граматика `ParseExpression` синтаксичке категорије

`ParseExpression` је специјална врста синтаксичке категорије која не производи исказ, већ служи за креирање синтаксичких стабала израза. Парсирање израза је урађено специјалном техником *recursive descent* парсирања која се зове *Pratt parsing* [6]. *Pratt parsing* дефинише технике обраде приоритета операција, заграда, препознавања идентификатора и заменских чланова. Враћа `Expression` објекат.

Прво се парсира префиксни израз, који може бити литерал различитог типа, идентификатор, заменски члан, израз са унарном операцијом или израз у заградама. Резултат овог корака постаје почетни леви операнд. Затим се улази у петљу која се извршава све док је приоритет следећег (инфиксног, $+$, $-$, $*$, $/$, $\wedge$) оператора строго већи од тренутног приоритета.

Унутар петље, оператор се конзумира, а десни операнд се добија рекурзивним позивом функције за парсирање израза којој се прослеђује приоритет тог новог оператора (или приоритет умањен за један, уколико је операција десно-асоцијативна, попут степеновања). Од левог операнда, оператора и десног операнда креира се ново стабло бинарног израза, које затим постаје нови леви операнд за наредну итерацију петље.

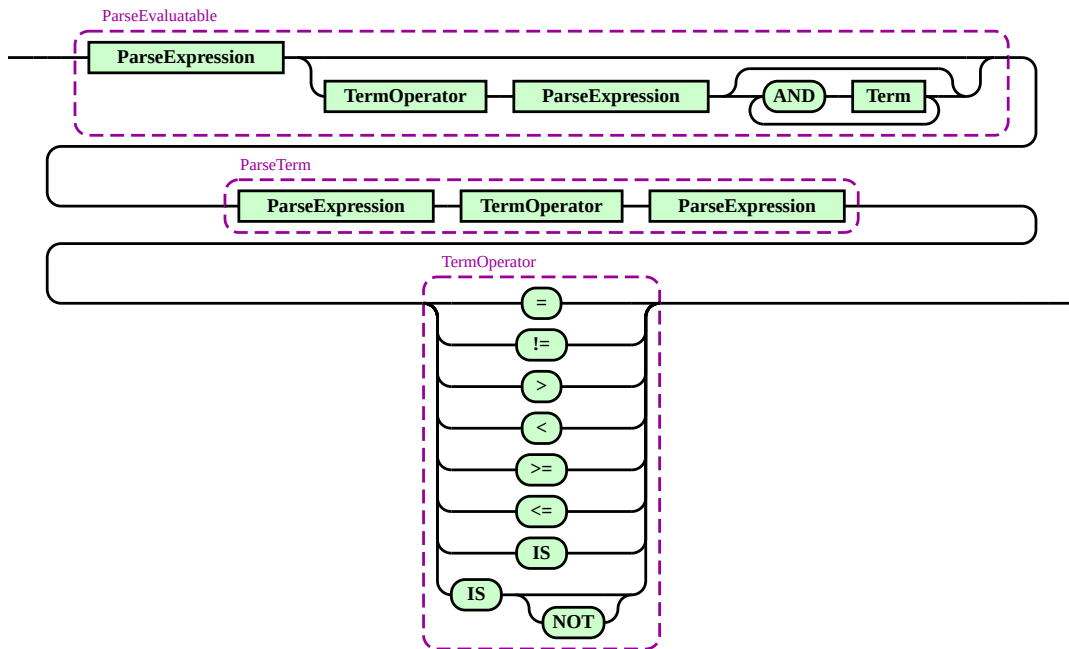
У исечку кода испод се могу видети различити приоритети операција на основу токена који их описују. Токен `STAR` представља и оператор заменског члана, па се зове `STAR` уместо `MULTIPLY`. Префиксне операције имају највећи приоритет.

```
private static final int PREFIX_PRECEDENCE = 100;

private int getPrecedence(Token opToken) {
    return switch (opToken) {
        case SymbolToken.CARET -> 30;
        case SymbolToken.STAR, SymbolToken.DIVIDE -> 20;
        case SymbolToken.PLUS, SymbolToken.MINUS -> 10;
        default -> 0;
    };
}
```

Листинг 3: Приоритети аритметичких операција у систему

6.2.2.9 ParseEvaluatable

Слика 19: Граматика `ParseEvaluatable` синтаксичке категорије

`ParseEvaluatable` је специјална врста синтаксичке категорије која не производи исказ, већ служи за агностично креирање објеката који се могу евалуирати на вредност. Личи на `ParsePredicate`, али са додатном могућношћу да препозна када израз није део члана или предиката. Враћа објекат који имплементира `Evaluatable` интерфејс (секција 5.1.5). Не подржава чланове без оператора поређења.

Подсистем планирања је један од три главна подсистема *LBDB* система обраде упита. У оквиру животног циклуса обраде једне *SQL* наредбе, подсистем планирања се ослања на подсистем парсирања зарад конструкције стабала релационих оператора.

Подсистем планирања се састоји од две целине:

- скуп *Java* класа појединачних планова. Свака класа појединачног плана моделује примену **једног** релационог оператора без извршавања тог оператора у оквиру виртуеле машине
- планер, чији је задатак да конструише стабло појединачних класа планова које се преводи у стабло релационих оператора; ово стабло често има и назив **план**, али га не треба помешати са класом плана

7.1 Структура класа планова у систему

Класа плана релационог оператора описује шему виртуелне табеле након примене оператора и омогућава рачунање статистичких метаподатака (секција 4.2) за ту виртуелну табелу. Ови статистички метаподаци могу бити искоришћени за ефикасније извршавање наредби.

Статистичке метаподатке које класа плана може да израчуна су исти као и статистички метаподаци који се прате за физичке табеле, а они су:

- број блокова потребних за пролазак кроз све слоге
- број слогова
- број јединствених вредности за сваку колону
- број *NULL* вредности за сваку колону

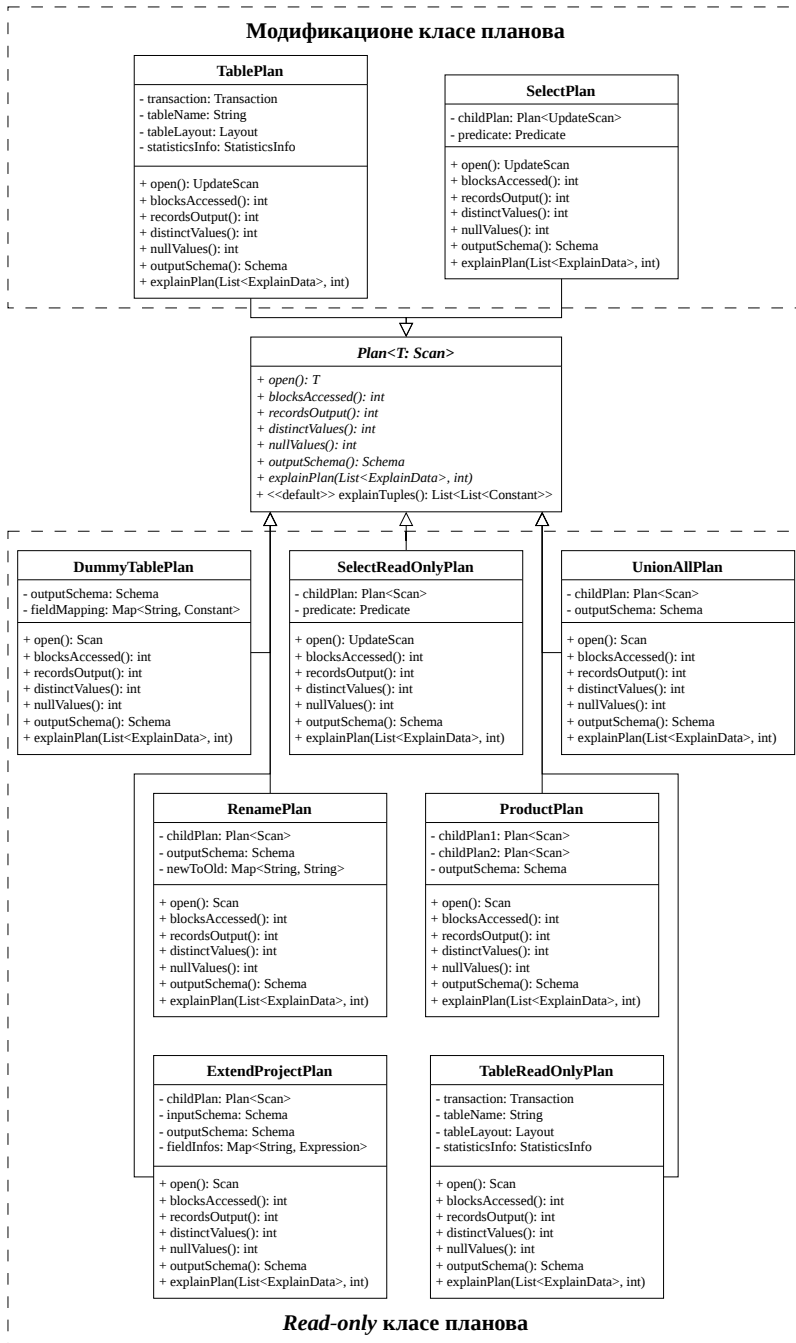
Постоје два ограничења везана за ове статистичке метаподатке:

- нису 100% прецизни, али без обзира на то, могу помоћи планерима (више о овом ограничењу у секцији 11.2.3)
- имплементација планера *LBDB* система их не користи у пуном потенцијалу (више о овом ограничењу у секцији 11.2.5)

7.1.1 Хијерархија имплементације планова

Најопштија подела класа планова је на оне који само читају податке (енг. *read-only*) и на оне који могу да модификују податке. За разлику од хијерархије релационих оператора, не постоји хијерархија подразумеваних имплементација јер класе планова немају толико заједничких особина. Подела на *read-only* и модификационе класе

планова је одрађена употребом *generics Java* конструкта, уместо дељења главног интерфејса на два подтипа. Свака класа плана оперише над једним или над двоје деце, а дете плана се такође зове и продређена класа плана.



Слика 20: Хијерархија имплементације планова

7.1.1.1 Plan

Plan интерфејс дефинише операције неопходне за рачунање свих статистичких података, добијање шеме резултујуће табеле, претварање стабла планова у стабло релационих оператора и добијање слогова за аутоматски опис плана (секција 7.2.3.2). Све класе планова су моделоване преко њега.

7.1.1.2 TablePlan

TablePlan описује конкретну физичку табелу, уместо да врши трансформације виртуелне табеле. Излазна шема је једнака шеми физичке табеле. Извлачи статистичке податке директно из менаџера метаподатака за табелу за коју је везан. Израчунати статистички метаподаци свих планова у стаблу планова на крају зависе од вредности статистичких метаподатака ове класе плана. Због имплементационих детаља Java језика, може да се користи само у модификујућим стаблима оператора. Постоји и TableReadOnlyPlan варијанта која има исту функцију за *read-only* стабла оператора.

7.1.1.3 DummyTablePlan

DummyTablePlan описује виртуелну табелу која се састоји од једног слога у упитима који не раде са физичким табелама. Излазна шема се одређује на основу константних вредности у упиту, а статистички подаци су прецизни јер се лако рачунају пошто је број конкретних вредности јако мали.

7.1.1.4 SelectPlan

SelectPlan описује виртуелну табелу након примене услова филтрирања. Излазна шема је једнака шеми подређене класе плана. Број блокова остаје непромењен јер да би знали који све слогови испуњавају услов филтера, потребно је проћи кроз све слокове, па са тиме и кроз све блокове подређеног плана.

Редукциони фактор представља колико пута ће број слогова на излазу бити смањен. Рачуна се на основу услова филтрирања, који је представљен предикатом. Сваки члан предиката представља један део филтера и има свој редукциони фактор. Пошто систем подржава само уланчавање чланова AND логичким оператором, редукциони фактор предиката се рачуна као производ свих редукционих фактора чланова од ког се предикат састоји.

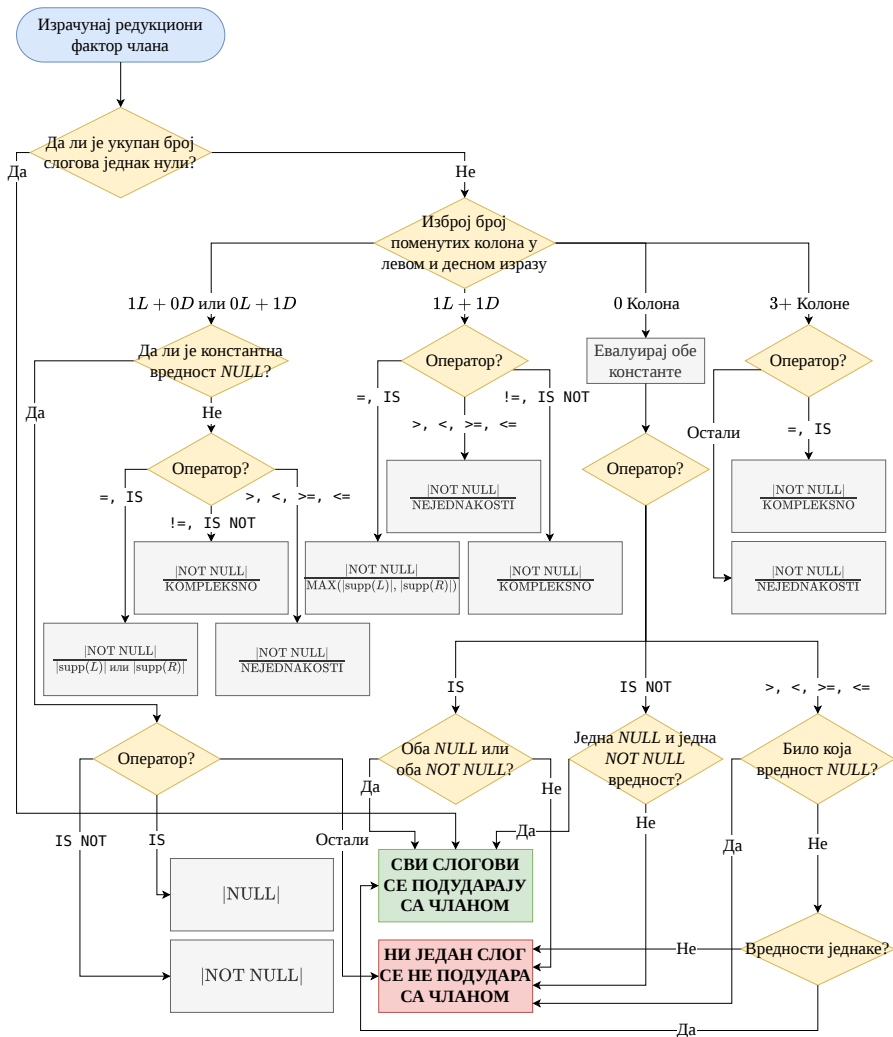
Алгоритам рачунања редукционог фактора једног члана је представљен на фигури 21. Пошто се број слогова након филтрирања добија дељењем броја слогова подређене класе плана и редукционог фактора, специјални случајеви се могу представити различитим константама.

Специјални случај неједнакости је представљен константом *NEJEDNAKOSTI* која има вредност 3.0 и означава процењену вредност редукције у случају коришћења операција неједнакости.

Специјални случај када постоје сувише комплексни изрази је представљен константом *KOMPLEKSNO* која има вредност 10.0 и означава процењену вредност редукције у случају постојања изрази који има или више од две колоне или изрази који комбинује колоне са операцијама поређења на нетривијалан начин.

Случај када се ниједан слог не подудара са чланом је представљен константом максималне вредности Double типа и означава максималну редукцију. Случај када сви слогови подударају неки члан је представљен константом 1.0 и представља одсуство редукције.

Избор вредности ових константи је описан у *SystemR* истраживачком раду [7] о путањама приступа.



Слика 21: Flowchart дијаграм рачунице редукционог фактора члана

Процена броја јединствених вредности за излазну колону врши се анализом предиката и проналажењем услова једнакости и тај број је једнак:

- нула – уколико предикат изједначава тражену колону са две или више различитих константи. У овом случају, услов је контрадикторан и ниједан слог неће задовољити филтер, па самим тим неће бити ни јединствених вредности;
- један – уколико предикат изједначава тражену колону са тачно једном константом. Сви слогови који прођу филтер имаће исту вредност за ту колону;
- минимуму између броја јединствених вредности те колоне и свих колона са којима је изједначена, уколико колона није изједначена ни са једном константом, али јесте са једном или више других колона. У овом случају, број јединствених вредности тражене колоне не може бити већи од њеног оригиналног броја јединствених вредности из подређене класе плана, али не може бити већи ни од броја јединствених вредности најрестриктивније колоне са којом је изједначена.

Процена броја *NULL* вредности за сваку излазну колону врши се анализом предиката и његовог односа према *NULL* константама и тај број је једнак:

- укупном броју излазних слогова овог плана, уколико предикат експлицитно изједначава тражену колону са *NULL* вредношћу, а излазна шема дозвољава *NULL* вредности за ту колону. У овом случају, сви слогови који прођу филтер имаће *NULL* вредност;
- нула – уколико предикат изједначава тражену колону са *NULL* вредношћу, али излазна шема не дозвољава *NULL* вредности (није *nullable*). У овом случају, услов је немогуће испунити;
- нула – уколико предикат садржи услов који експлицитно искључује *NULL* вредности за тражену колону. Сви слогови са *NULL* вредностима неће проћи овакав филтер;
- пропорцији броју *NULL* вредности из подређеног плана, уколико предикат не спомиње *NULL* константу у вези са траженом колоном. У овом случају, претпоставља се да услов филтера равномерно смањује укупан број слогова и број *NULL* вредности, па се број *NULL* вредности из подређене класе плана дели са фактором редукције целокупног предиката.

Због имплементационих детаља *Java* језика, може да се користи само у модификујућим стаблима оператора. Постоји и *SelectReadOnlyPlan* варијанта која има исту функцију за *read-only* стабла оператора.

7.1.1.5 **ExtendProjectPlan**

ExtendProjectPlan описује виртуелну табелу са свим пројектованим колонама. Излазна шема има све додате колоне, а из ње су избрисане непројектоване колоне. С обзиром да операција пројекције не додаје нове слокове, број блокова и број слогова остају непромењени и директно се преузимају од подређене класе плана.

Процена броја јединствених вредности за сваку излазну колону врши се на основу сложености израза или предиката који ту колону дефинише и тај број је једнак:

- нула – уколико се тражи процена за колону која се не налази у пројекцији;
- један – уколико је израз или предикат константа (не референцира ниједну колону);
- два – уколико је у питању предикат, који ће највероватније имати обе могуће вредности;
- броју јединствених вредности те колоне из подређене класе плана, уколико израз референцира тачно једну колону. Претпоставка је да већина трансформација над једном колоном (нпр. аритметичке операције) задржава сличну дистрибуцију вредности;
- укупном броју слогова, уколико израз референцира више од једне колоне. У овом случају, претпоставља се да комбинација више поља резултује јединственом вредношћу за сваки слог.

Процена броја *NULL* вредности за сваку излазну колону врши се на основу сложености израза или предиката који ту колону дефинише и тај број је једнак:

- нула – уколико се тражи процена за колону која се не налази у пројекцији;
- нула – уколико је у питању предикат, јер се предикат може евалуирати само на тачно и нетачно;
- нула – уколико шема гарантује да израз не може имати *NULL* вредности (није *nullable*);
- нула – уколико је израз једнак било којој константи сем *NULL* константе;
- један – уколико је израз једнак *NULL* константи;
- броју јединствених вредности те колоне из подређене класе плана, уколико израз референцира тачно једну колону. Претпоставка је да већина трансформација над једном колоном задржава сличну дистрибуцију *NULL* вредности;
- максималаном броју *NULL* вредности међу свим референцираним колонама из подређене класе плана, уколико израз референцира више од једне колоне. Претпоставља се да ће израз бити *NULL* уколико је барем један од операнада *NULL*, па је максималан број *NULL* вредности песимистична процена.

7.1.1.6 RenamePlan

RenamePlan описује виртуелну табелу са свим примењеним преименовањима колоне. Излазна шема садржи све колоне са новим именом и ниједну колону са старим именом. С обзиром да операција преименовања не додаје нове слокове, број блокова и број слогова остају непромењени и директно се преузимају од подређене класе плана. Процена броја јединствених вредности колоне је једнака подређеној класи плана уколико се тражи ново име старе колоне или уколико је колона непреименована, а једнака је нули ако се тражи старо име преименоване колоне. Процена броја *NULL* вредности функционише исто.

7.1.1.7 ProductPlan

ProductPlan описује виртуелну табелу која је производ две табеле. Излазна шема садржи све колоне од обе табеле. За демонстрацију рачунице броја блокова дефинишемо табеле T_1 и T_2 :

Табела 4: Пример карактеристика табела за рачунање броја блокова плана производа

T_i	$B(T_i)$	$R(T_i)$	$RPB(T_i)$
T_1	5	1000	$\frac{1000}{5} = 200$
T_2	100	500	$\frac{500}{100} = 5$

- $B(T_i)$ представља број блокова неке табеле,
- $R(T_i)$ представља број слогова неке табеле,
- $RPB(T_i)$ представља колико слогова може да стане по једном блоку за неку табелу.

Овај пример се односи на рад са конкретним физичким табелама, али у генералном случају, табеле нису физичке, већ су представљене класама планова са којима класа плана производа барата.

Да би се прошло кроз сваки слог резултујуће табеле, потребно је да се за сваки слог леве табеле прође кроз сваки слог десне табеле. Формула која описује број блокова потребан да се ово изврши је следећа [1]:

$$B(T_r) = B(T_l) + (R(T_l) * B(T_d))$$

Ако ставимо конкретне вредности табела T_1 и T_2 у ову формулу, добијамо различите резултате у односу на то која табела је лева, а која десна:

- $(T_l = T_1, T_d = T_2) \Rightarrow B(T_r) = 5 + (1000 * 100) = 100005$
- $(T_l = T_2, T_d = T_1) \Rightarrow B(T_r) = 100 + (500 * 5) = 2600$

Види се да ако ставимо да табела T_1 буде десна, а T_2 лева, добијамо мањи број блокова резултујуће табеле, а са тиме и ефикаснију операцију производа. Еквивалентна формула [1]:

$$B(T_r) = B(T_l) + (RPB(T_l) * B(T_l) * B(T_d))$$

даје бољи увид због чега рачуница броја блокова резултујуће табеле није симетрична у односу на две табеле које учествују у производу. Сабирак $RPB(T_l) * B(T_l) * B(T_d)$ знатно више утиче на финални резултат у односу на $B(T_l)$.

Што је слог мањи, један блок може да их садржи више. У супротном, што је слог већи, један блок може да их садржи мање. У табелама где је слог већи, потребно је приступити више блокова да би се прошло кроз исти број слогова као у табелама где је слог мањи. У операцијама производа боље је ставити табелу где је слог већи (то јест

где је RPB мањи) на леву страну, а табелу где је слог мањи (то јест где је RPB већи) на десну страну јер се слоговима десне табеле приступа знатно више него слоговима леве табеле.

Број слогова је производ броја слогова обе подређене класе плана, а број јединствених и број *NULL* вредности се прослеђује подређеној класи плана у ком се налази тражена колона.

7.1.1.8 UnionAllPlan

UnionAllPlan описује виртуелну табелу која је збир две табеле. Излазна шема је једнака излазној шеми леве подређене класе плана. Број блокова је збир броја блокова обе подређене класе плана, а број слогова је збир броја слогова обе подређене класе плана. Процена броја јединствених вредности колоне је једнака збиру процена јединствених вредности обе подређене класе плана за ту колону. Процена *NULL* вредности колоне је једнака збиру процена *NULL* вредности обе подређене класе плана.

7.2 Планер

Већина наредби дефинисаних SQL стандардом захтева пропратно стабло релационих оператора. Конструкција стабла класа планова које се даље преводи у стабло релационих оператора и провера семантичке валидности наредби су послови планера.

Главна подела техника планирања у релационим базама података је на технике праћења стриктних правила прављења планова (енг. *rule-based optimisation*, *RBO*; *heuristics-based optimisation*, *HBO*) и технике планирања који раде са ценама (енг. *cost-based optimisation*, *CBO*). Цена представља комбинацију статистичких метаподатака релационих оператора са хардверским особинама који ти релациони оператори користе.

Ранији системи управљања базама података попут *INGRES* система су користили *RBO* технике планирања [8], док модерни системи користе *CBO* технике планирања^{2,3} које су постале популарне након *SystemR* истраживачког рада о путањама приступа [7].

Еволуција техника планирања, која се може видети кроз ову главну поделу, постоји јер је кроз историју било потребно обезбедити све ефикасније планере који раде са све већим скуповима података.

7.2.1 Евалуација израза током планирања

Евалуација израза и предиката у стаблу релационих оператора је најскупље место евалуације, јер се операције извршавају у оквиру виртуелне машине система, где се не користе процесорске инструкције директно. *PartialEvaluator* пружа обраду опера-

²<https://www.postgresql.org/docs/current/planner-optimizer.html>

³<https://www.postgresql.org/docs/current/planner-stats-details.html>

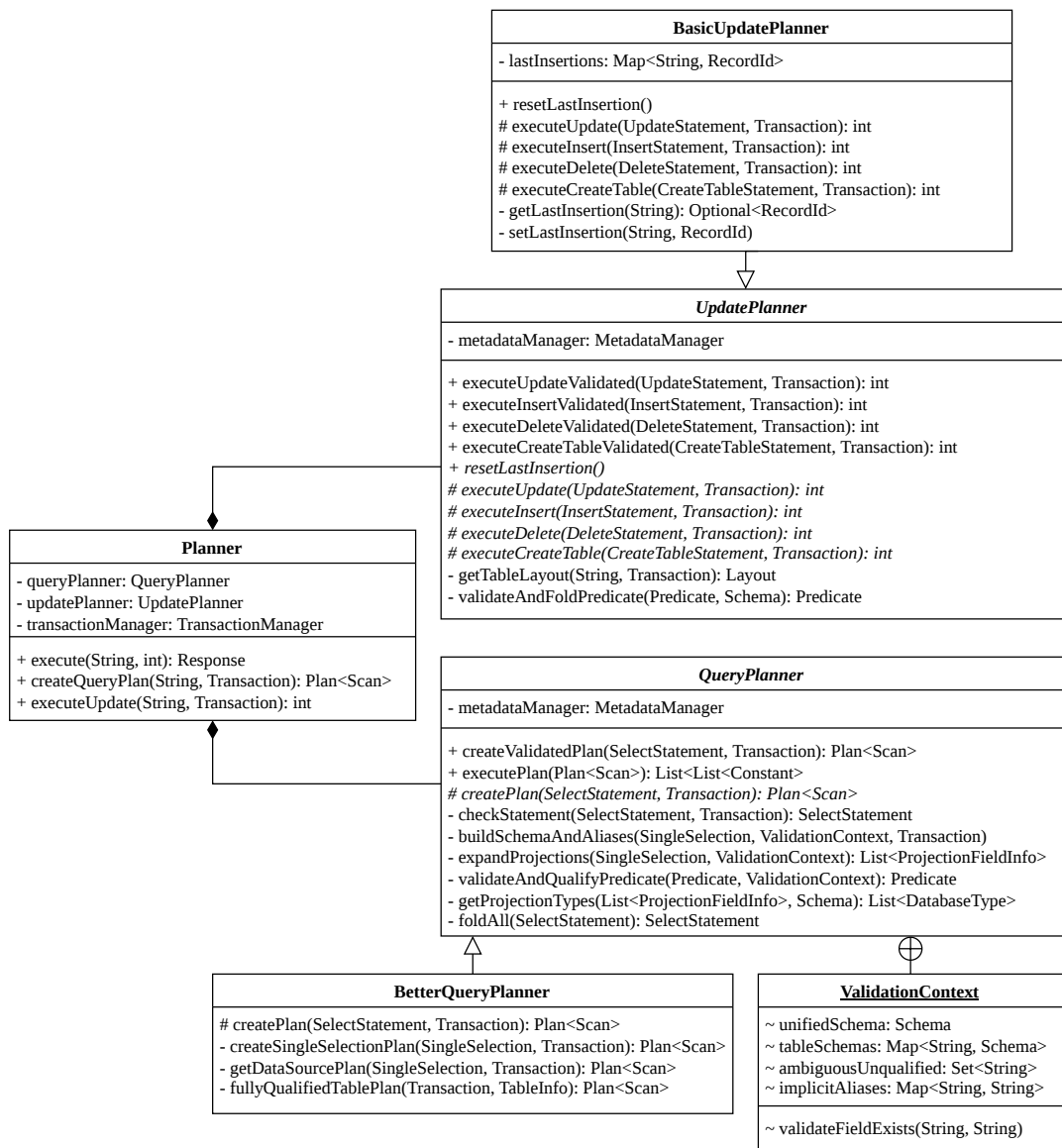
ција у тренутку планирања, што знатно повећава перформансу упита јер се тривијалне операције не извршавају за сваки слог.

Тривијалне операције које се редукују су: аритметичке операције које не трансформишу податке, аритметичке операције између две константе, операције поређења које су увек тачне и у случају да постоји контрадикторна операција поређења она скраћује (енг. *short-circuit*) цео предикат, чинећи га увек нетачним без обзира на његове остале компоненте.

7.2.2 Улазна тачка креирања и извршавања планова

Свака *SQL* наредба, која је првобитно низ карактера, се прослеђује *Planner* класи, која прво врши парсирање и претварање низа карактера у објекат исказа (*Statement*). Класа *Planner* дефинише две групе функција које су прилагођене различитим *API* (*Application Programming Interface*) интерфејсима:

- *createQueryPlan* и *executeUpdate* које су прилагођене *JDBC* (*Java Database Connectivity*) *API* интерфејсу. *JDBC* дефинише генеричко понашање за интеракцију са системима за управљање базама података (не постоји конкретна имплементација за *LBDB*, али дефинисањем ових метода ју је лако додати). *createQueryPlan* креира план за *read-only* наредбу, али га не извршава, док се *executeUpdate* ослања на то да су модификационе наредбе дизајниране да се одмах изврше и враћа број промењених слогова,
- *execute* која је прилагођена клијентско-серверској архитектури (поглавље 8) *LBDB* система, у оквиру које се брине о аутоматском или ручном потврђивању трансакција, креирању и извршавању плана. Враћа инстанцу *Response* објекта (секција 8.1.1), која представља све могуће врсте одговора на неку наредбу.



Слика 22: Структура планера

7.2.3 Планирање *read-only* наредби

SQL наредбе се деле на *read-only* и модификационе. Главни пример *read-only* наредбе је SELECT наредба, која служи за структурирано упитивање (енг. *query*) базе података.

Свака SELECT наредба прво мора проћи семантичку проверу пре прављења самог плана. Семантичка провера се састоји од следећих корака:

- провера постојања физичких табела споменутих у наредби
- проширење заменских чланова на конкретне колоне

- провера да се заменски чланови не користе у изразима
- провера постојања колона споменутих у пројекцијама и предикату
- провера двосмислених имена колона (у случају да две табеле имају исти назив колоне и не може да се тривијално разуме на коју колону је корисник мислио)
- провера да ли аритметичке операције могу да се изврше за тип колоне
- провера да ли колоне у унијама имају исте типове

QueryPlanner апстрактна класа пружа имплементацију семантичке провере, а конкретни алгоритми планирања SELECT наредбе који је наслеђују могу да подразумевају да су наредбе које добију сигурно семантички валидне. QueryPlanner такође редукује све изразе и предикат помоћу PartialEvaluator класе.

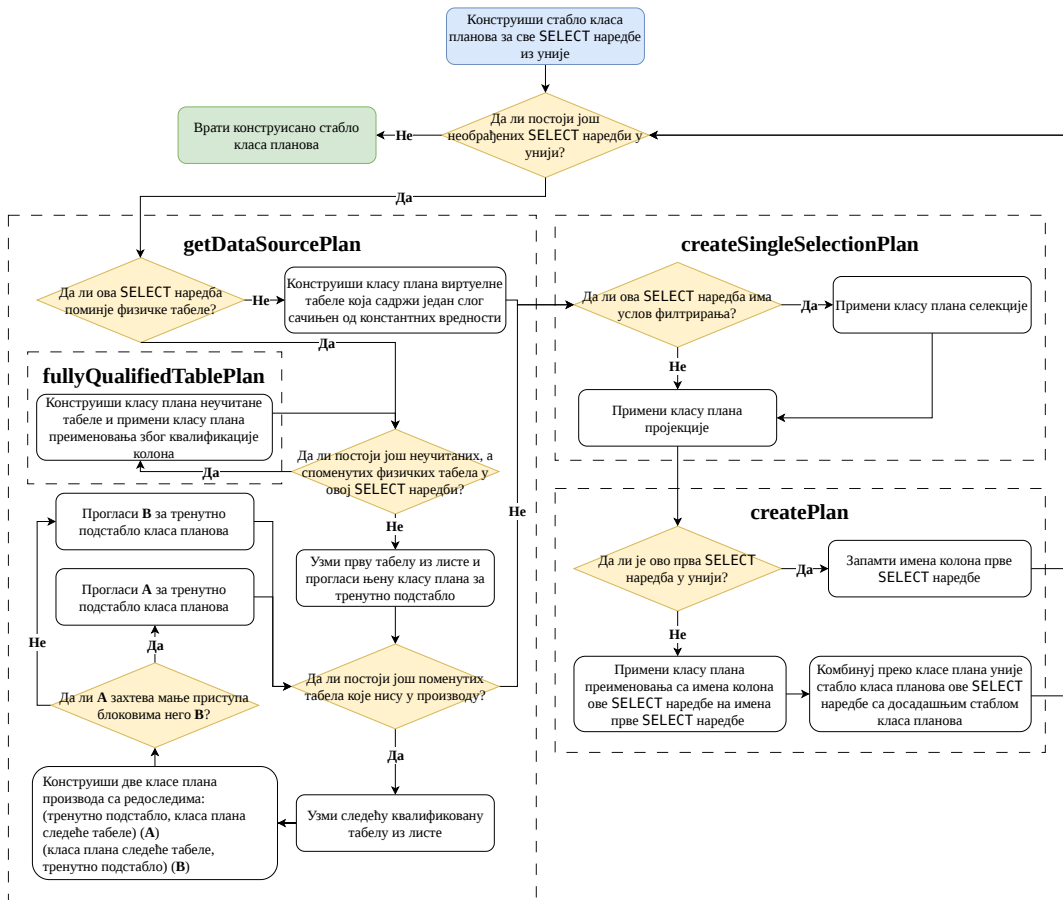
7.2.3.1 Алгоритам планирања SELECT наредби

BetterQueryPlanner класа наслеђује QueryPlanner и представља имплементацију основног планера који подржава све алтернативе SELECT наредбе представљене у њеној граматичи (секција [6.2.2.3](#)).

Стабло релационих оператора је коректно (енг. *sound*), ако сви слогови које оно производи испуњавају све услове релационих операција дефинисаних неком SELECT наредбом. Стабло планова, које се преводи у стабло релационих оператора, које BetterQueryPlanner конструише је увек коректно.

Проблем BetterQueryPlanner имплементације је ниска ефикасност конструисаних табала планова, јер не користи ни технике *RBO* планирања, ни технике *CBO* планирања, већ извршава само минимални скуп корака који су неопходни да се обезбеди коректност.

Алгоритам планирања се може видети на фигури [23](#).

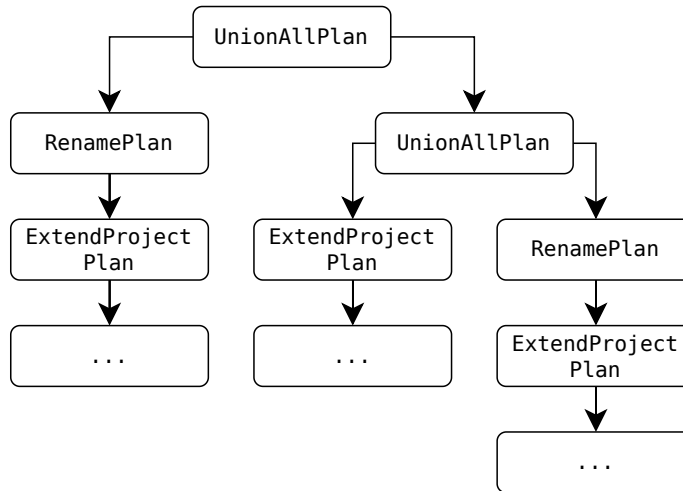


Слика 23: Flowchart дијаграм BetterQueryPlanner алгоритма планирања

Креира финално стабло планова кроз четири функције које се међусобно позивају, имају растући приоритет и задужене су за различите операције:

- `createPlan` функција је улазна тачка алгоритма, дефинисана у `QueryPlanner` класи и има задужење креирања унија више `SELECT` наредби, у случају постојања `UNION ALL` граматичке алтернативе.

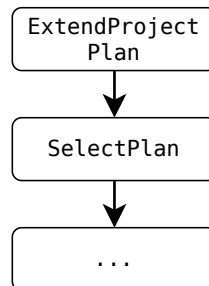
Уније `SELECT` наредби захтевају да се колонама сваке `SELECT` наредбе приступа помоћу имена датих у првој `SELECT` наредби. Због овога се додаје оператор преименовања испред сваког подстабла планова сваке наредбе у унији, сем прве. Операција уније има најмањи приоритет, па се последња извршава. Уколико не постоји `UNION ALL` граматичка алтернатива, функција враћа подстабло планова једине `SELECT` наредбе.



Слика 24: Пример стабла планова након уније 3 наредбе

- `createSingleSelectionPlan` функција обезбеђује коректност филтрирања и пројекције.

Додаје оператор филтрирања само ако филтер постоји; додаје нове виртуелне колоне и брише колоне које нису споменуте.



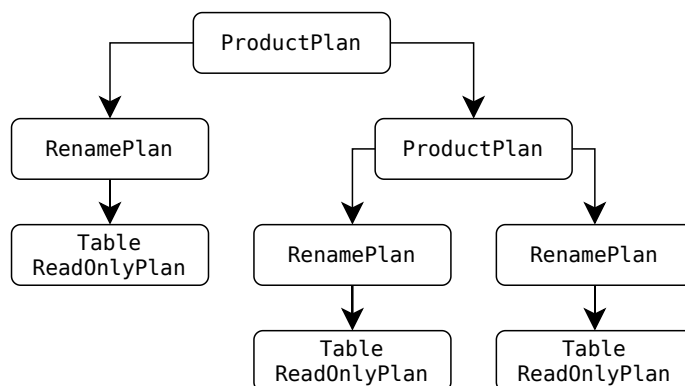
Слика 25: Пример стабла планова након филтрирања и пројекције

- `getDataSourcePlan` функција се брине о томе одакле ће доћи слонови и има две путање извршавања.

Прва путања извршавања се дешава када се у `SELECT` наредби не спомиње ниједна физичка табела, већ се ради упит виртуелне табеле која има један слог који се састоји само од константи. У том случају, само враћа `DummyTablePlan` класу плана.

Друга путања извршавања се дешава када се спомиње једна или више физичких табела. Ако се спомиње једна физичка табела, њен план бива враћен. Ако се спомиње више од једне физичке табеле, потребно је урадити операцију производа. Производ табела се врши тако што се прва споменута табела прогласи да буде почетна, па се пролази кроз све остале споменуте табеле и понавља се поступак: креирају се два

производ плана, један где је досадашње подстабло планова на левом месту, а план следеће табеле на десном и један где је редослед обрнут. План који има мање приступа блоковима се узима као следећи корен подстабла планова и поступак се понавља док се не прође кроз све споменуте табеле. Тиме се добија оформљено подстабло планова где су производи табела задовољени. Оваква провера броја приступаних блокова није оптимална, али може помоћи у отклањању веома неефикасних стабала планова и представља једино место где се примењује *RBO* техника планирања.



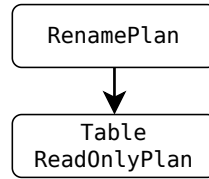
Слика 26: Пример стабла планова након производа 3 табеле

- `fullyQualifiedTablePlan` обезбеђује давање пуноквалификујућих имена колонама физичких табела.

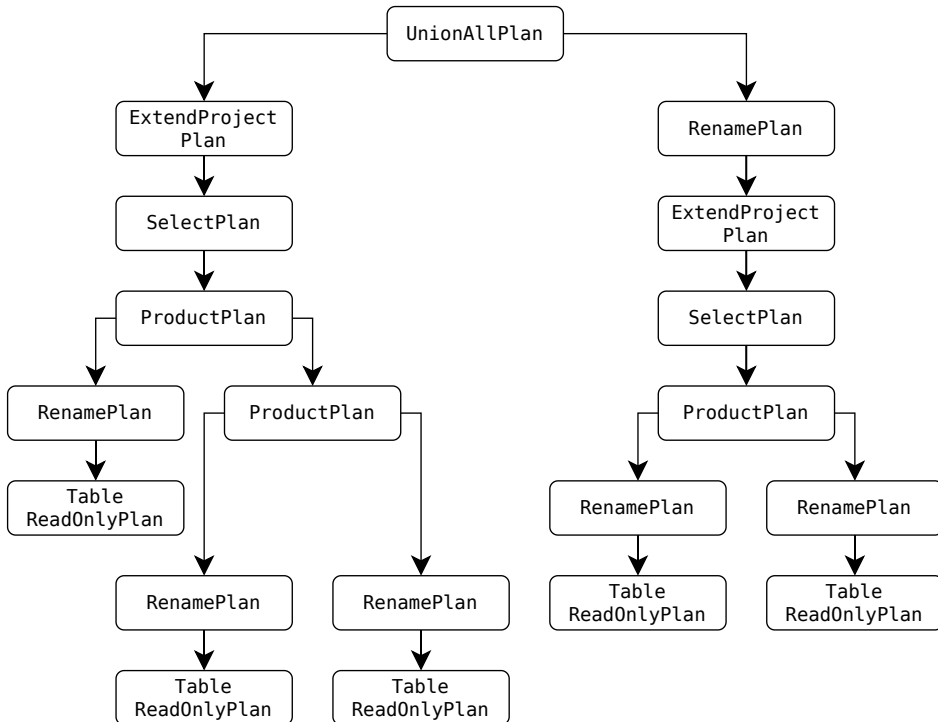
У случајевима где се врши производ две или више табеле, постоји могућност да те табеле имају истоимену колону. Ово је чест случај јер повећава читљивост упита и структуре табела, па је за њега потребно пружити адекватну подршку.

У претходној функцији која врши производе, речено је да се директно барата са плановима табела. Ово није прецизно, јер се не барата директно са табелама, већ са подстаблом планова које представља табелу са пуноквалификованим именима колона. Квалификација имена колона се врши преко оператора преименовања, тако што се пре самог имена колоне дода име табеле и тачка (`ime_kolone` постаје `ime_tabele.ime_kolone`). Дозвољава и преименовање табела у случају да вршимо производ две (или више) исте табеле.

Ако се производу налазе само табеле са са пуноквалификованим колонама, не постоји могућност да систем не може да препозна којој табели колона припада, сем ако корисник није задао семантички неисправну наредбу.



Слика 27: Пример стабла планова након квалификације колона табеле



Слика 28: Пример комплетног стабла планова

7.2.3.2 Аутоматско генерисање описа планова

Други пример *read-only* наредбе је EXPLAIN наредба. Њена улога у систему је табеларно исписивање комплетних стабла планова. EXPLAIN наредба се позива тако што се дода кључна реч EXPLAIN испред SELECT наредбе. Није подржана за модификационе наредбе.

EXPLAIN наредба је имплементирана тако да генерише слоге који прате шему специјалне табеле која има следеће колоне: име релационог оператора, процена кроз колико блокова ће тај релациони оператор проћи да генерише све слоге, процена броја слогова и специјални детаљи. Сваки слог представља чвор резултујућег стабла релационих оператора. Иако је поента наредбе табеларни приказ стабла, наредба само генерише ове слоге и не брине се о форматирању табеле (секција 8.3.1).

Scan	Block est.	Record est.	Details
ExtendProjectScan	5	52	
└─ SelectScan	5	52	student.isactive = TRUE
└─ RenameScan	5	103	
└─ TableScan	5	103	'student'

Листинг 4: Пример резултујуће табеле EXPLAIN наредбе

7.2.4 Планирање модификационих наредби

Модификационе наредбе су разне, а UpdatePlanner има исту улогу за њих, као што QueryPlanner има за SELECT наредбу, а то је само семантичка провера. Конкретни алгоритми планирања модификационих наредби могу да подразумевају да је наредба семантички валидна и да су изрази и предикати редуковани помоћу PartialEvaluator класе. За сваку модификациону наредбу су описани кораки за семантичку проверу.

INSERT наредба служи за уметање нових слогова. Семантичка провера INSERT наредби се састоји од следећих корака:

- да ли постоји физичка табела у коју се умећу нови слогови
- да ли се број колона нових слогова подудара са бројем дефинисаним у шеми табеле
- провера типова колона нових слогова са типовима колона дефинисаних у шеми табеле
- да ли су дозвољене *NULL* вредности за колоне где је вредност нових слогова *NULL*

UPDATE наредба служи за ажурирање вредности постојећих слогова на основу неког услова филтрирања. Семантичка провера UPDATE наредби се састоји од следећих корака:

- да ли постоји физичка табела чији се слогови ажурирају
- да ли постоје колоне споменуте у предикату и изразима ажурирања
- да ли су дозвољене *NULL* вредности за колоне где је нова вредност *NULL*
- провера типова колона ажурираних слогова са типовима колона дефинисаних у шеми табеле

DELETE наредба служи за брисање постојећих слогова на основу неког услова филтрирања. Семантичка провера DELETE наредби се састоји од следећих корака:

- да ли постоји физичка табела чији се слогови бришу
- да ли постоје колоне споменуте у предикату

CREATE TABLE наредба служи за креирање нових табела. Семантичка провера CREATE TABLE наредби се састоји од следећих корака:

- да ли табела са тим именом већ постоји

- да ли је величина слога прешла максималну величину (ограничење описано у секцији [11.2.2](#))

7.2.4.1 Алгоритам планирања INSERT наредбе

Стабло планова за INSERT наредбе је увек исто и састоји се само од `TablePlan` класе плана. Тај чвор се претвара у свој пратећи релациони оператор над којим се врше уметања нових слогова. Враћа број додатих слогова.

Алгоритам уметања новог слога имплементиран у `TableScan` оператору функционише тако што тражи прво слободно место за нови слог, али почевши од позиције тренутног слога тог `TableScan` објекта. Након што се оператор иницијализује, позициониран је на почетку табеле, то јест пре првог слога. Ово значи да ће уметање првог слога у листи нових слогова увек кретати од почетка. Предност овог приступа је то што ће обрисани слогови брзо бити поново попуњени, па се простор максимално искоришћава. Мана овог приступа је то што уметање првог новог слога може да потраје, јер у најгорем случају мора да се прође кроз све слокове табеле да се пронађе празно место. Други начин имплементације алгоритма је да се уметање нових слогова увек врши на крају. Предност је добра брзина уметања, јер се прескаче претрага за слободно место. Мана је то што се све више и више простора троши на обрисане слокове.

Имплементирано решење је компромис ова два алгоритма, где се за сваку табелу памти позиција последњег уметнутог слога и нови слогови се умећу од те позиције. Памћење позиција последње уметнутих слогова важи само док је систем упаљен и ресетује се приликом гашења система. Задржава предност брзине уметања, а након рестарта система места обрисаних слогова могу поново бити попуњена. Такође, ресетује се при поништавању трансакције.

7.2.4.2 Алгоритам планирања UPDATE наредбе

Стабло планова за UPDATE наредбе се може састојати само од `TablePlan` класе плана, али испред њега може стајати и `SelectReadOnlyPlan` класа плана у случају да слогови који требају бити ажурирани морају да испуне неки услов филтрирања. Ове две (или један) класе плана се претварају у своје пратеће релационе операторе који знају да поставе нове вредности. Враћа број ажурираних слогова.

За разлику од INSERT наредбе, UPDATE наредба може да садржи изразе који нису константни, то јест који спомињу колоне табеле која се ажурира.

7.2.4.3 Алгоритам планирања DELETE наредбе

Стабло планова за DELETE наредбе се може састојати само од `TablePlan` класе плана, али испред њега може стојати и `SelectReadOnlyPlan` класа плана у случају да не требају да се обришу сви слогови из табеле већ само они који испуњавају услов филтрирања. Враћа број обрисаних слогова.

Брисање неког слога само означава место где се тај слог налазио као слободно за уметање новог слога, уместо да ради компресију датотеке и заправо изврши физичко брисање.

7.2.4.4 Алгоритам планирања CREATE TABLE наредбе

CREATE TABLE наредба је специјална врста наредбе јер не модификује слокове обичних табела, већ слокове каталошких табела (секција 4.1). Додаје један слог у каталошку табелу која памти све постојеће табеле. Додаје слог за сваку колону нове табеле у каталошку табелу која памти све постојеће колоне. Пошто не утиче на слокове обичних табела, увек враћа нула за број слогова на које је утицала.

Менаџер метаподатака табела интерно конструише три TableScan објекта: први користи да провери јединственост имена нове табеле, други користи да уметне слог за нову табелу у tablecatalog каталошку табелу, а трећи користи да уметне слокове нових колона у fieldcatalog каталошку табелу. Последица овакве имплементације је та да UpdatePlanner апстрактна класа заправо не врши семантичку проверу пре позивања алгоритма планирања CREATE TABLE наредбе, него реагује на грешке и трансформише их, ако се десе.

Глава 8

Клијентско-серверска архитектура

Постоје два главна начина како систем управљања релационим базама података може радити: локално, без мрежне инфраструктуре (енг. *embedded connection*) и као сервер.

Карактеристике система у локалном режиму рада:

- ради у истом процесу оперативног система као и апликација која га користи,
- само једна апликација може да користи ту инстанцу система,
- не захтева мрежну конекцију,
- захтева мање ресурса,
- понаша се као библиотека која разуме унутрашњости релационог модела података.

Карактеристике система у серверском режиму рада:

- ради као засебан процес оперативног система,
- више различитих апликација и корисника може да приступи тој инстанци система,
- захтева мрежну конекцију,
- захтева више ресурса,
- понаша се као пружилац услуге управљања релационим моделом података.

LBDB систем подржава само серверски режим рада.

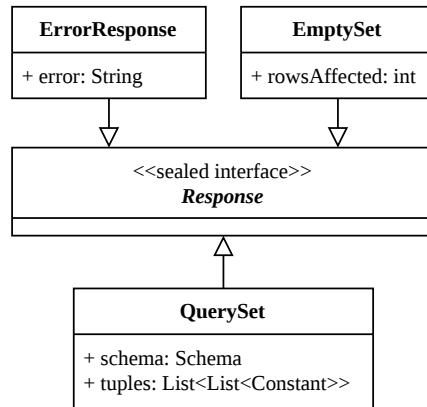
8.1 Комуникација са клијентима

Код серверског режима рада, претпоставља се да је клијент на удаљеном рачунару и нема приступ ресурсима рачунара на ком се покреће сервер. Последица овог је да сва комуникација мора да се врши кроз мрежу, преко неког протокола комуникације.

8.1.1 Одговори система

Све врсте одговора које сервер може дати за неку наредбу коју је клијент задао представљене су *Java record* конструктом и имплементирају *Response sealed interface*.

- *QuerySet* представља одговор на *read-only* наредбе, чији је резултат скуп слогова. Један слог је представљен листом константи. Да би систем знао како да пошаље слокове преко мреже, а касније и како да направи табеларни приказ, потребно је послати и пропратну шему која описује послате слокове.
- *EmptySet* представља одговор на модификационе наредбе. Садржи само податак о томе на колико слогова је утицано.
- *ErrorResponse* се враћа клијенту када се деси грешка приликом парсирања, приликом планирања или приликом извршавања наредбе.



Слика 29: Структура Response хијерархије

8.1.2 Протокол серијализације

Преко мреже је могуће слати само низ бајтова. Пошто Response објекти нису подразумевано представљени низовима бајтова, за њих се мора дефинисати начин серијализације и десеријализације.

Протокол серијализације Response објеката је инспирисан *RESP (REdis Serialization Protocol)*⁴ протоколом. Сваком типу се додељује један бајт који га јединствено представља. '*' за QuerySet, '_' за EmptySet и '-' за ErrorResponse.

Бројчане вредности се претварају у бајтове byte типа који се састоји од једног бајта, у бајтове short типа који се састоји од 2 бајта или у бајтове int типа који се састоји од 4 бајта. String вредности се претварају у бајтове уз *UTF-8 (Unicode Transformation Format)* кодирање.

EmptySet и ErrorResponse је тривијално серијализовати и десеријализовати, јер се састоје од само једне вредности.

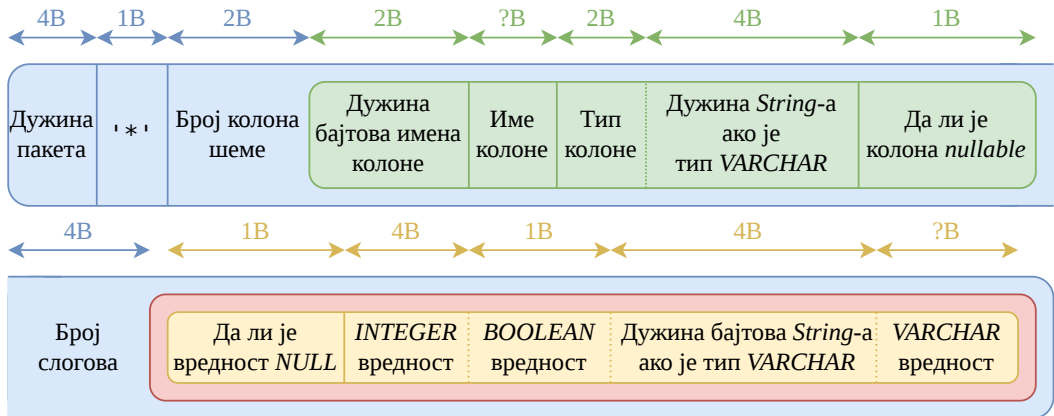
QuerySet садржи доста вредности које имају специфичан формат, па је алгоритам серијализације комплекснији (свака вредност је подразумевано записана у бајтовима):

- записује се јединствени идентификатор QuerySet типа: '*'
- записује се количина колона шеме резултујуће табеле као short
- за сваку колону се записује:
 - дужина бајтова имена, као short
 - кодирано име колоне
 - тип колоне, као short, ако је тип VARCHAR записује се и његова дужина, као int
 - да ли је колона nullable, као byte
- записује се количина слогова

⁴<https://redis.io/docs/latest/develop/reference/protocol-spec/>

- за сваку вредност сваког слога се записује:
 - 0 ако је *NULL*, 1 ако није *NULL*, као *short*
 - бројну вредност, као *int* ако је тип *INTEGER*, бројну вредност, као *byte* ако је тип *BOOLEAN*, дужина *String*-а, као *int* тип заједно са кодираним *String*-ом ако је тип *VARCHAR*

На крају серијализације се рачуна дужина бајтова и она се ставља пре свих бајтова, да би се приликом десеријализације знало колико бајтова да се прочита. Алгоритам десеријализације за све типове функционише исто, али у супротном смеру.



Слика 30: Структура QuerySet пакета

На слици плаво представља цео пакет, зелено представља део пакета који се понавља за сваку колону шеме, црвено представља део пакета који се понавља за сваки слог, а жуто представља део пакета који се понавља за сваку вредност слога.

8.2 Серверски слој

Класа *LBDBServer* садржи *main* функцију серверске апликације система. Чита и валидира аргументе командне линије: порт на ком ће сервер радити и путања где ће се чувати датотеке система. Покреће инстанцу сервера са прослеђеним аргументима. Сва логика која подржава серверске операције се налази у *Server* класи.

8.2.1 Руковођење клијентима

Прва функционалност сервера је обрада клијентских конекција и наредби које оне шаљу. Приликом покретања сервера, инстанцира се серверски *socket* који прихвата конекције на одређеном порту и отвара клијентски *socket* за сваког клијента.

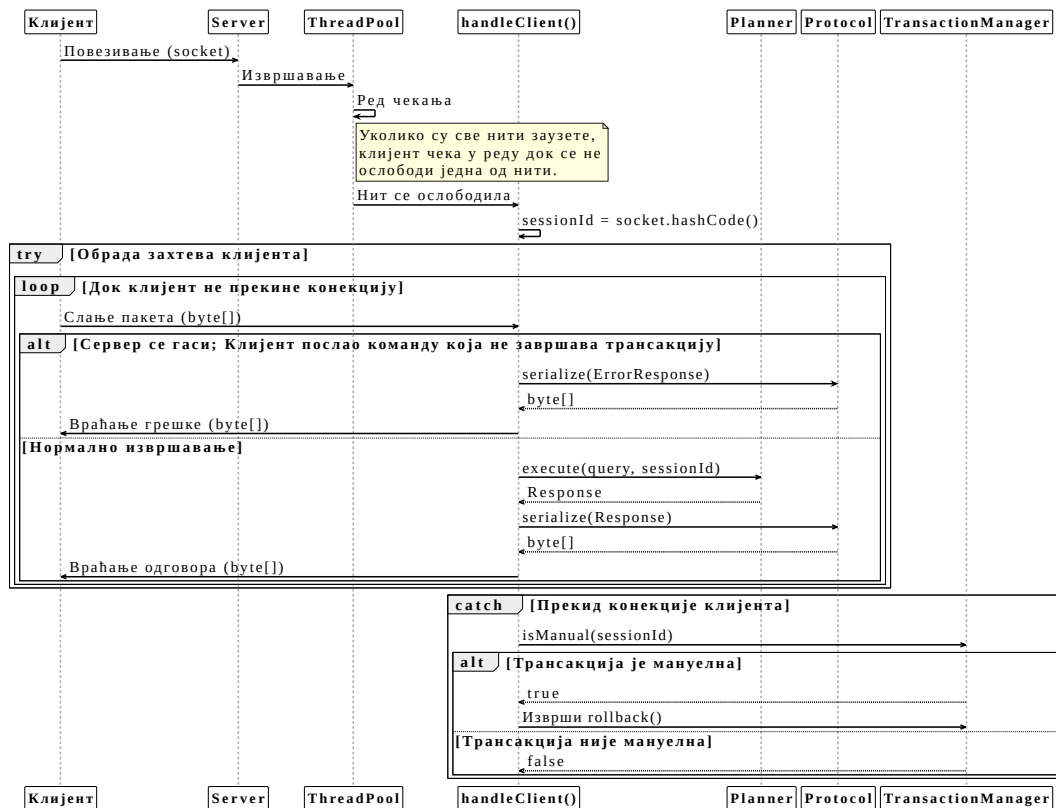
Пошто су клијенти независни, не би требало да чекају једни друге и због тога се обрађују у засебним нитима. Зарад лакшег управљања, постоји *ThreadPool* са 8 фиксних, стално постојећих нити, које чим заврше са обрадом једног захтева поново постају слободне.

8.2.1.1 Обрада једног клијента

Обрада захтева клијената се врши кроз `handleClient()` функцију. Сваки клијентски `socket` се кодира у јединствени број сесије, тако што се израчуна хеш вредност његове конекције. Овим се омогућава мапирање клијента на његову тренутну трансакцију. Ако број сесије клијента не постоји у систему, додељује се нова (прва) трансакција за тог клијента.

Клијентске трансакције могу да раде у режиму где се састоје од једне наредбе (*autocommit*) или у режиму где се састоје од више наредби. У режиму где се трансакције састоје од више наредби, потребно је почети их са `START TRANSACTION` наредбом, а завршити са `COMMIT` или `ROLLBACK` наредбама. Ово је главни разлог зашто је потребно јединствено идентификовати сесије клијената, да би њихова трансакција могла да перзистира кроз више наредби.

У случају прекида конекције, сервер ће извршити `ROLLBACK` тренутне трансакције клијента. У случају да се сервер гаси, повезани клијенти могу да пошаљу само наредбе које завршавају трансакције, али више о томе у опису гашења сервера.



Слика 31: Дијаграм секвенце обраде клијента

8.2.2 Писање контролних тачака

Друга функционалност сервера је одређивање када (али не и како) ће мирна контролна тачка бити писана. Приликом покретања сервера стартује се и нит која на сваких 10 минута започиње писање мирне контролне тачке. Да би се мирна контролна тачка записала, ниједна трансакција не сме бити активна у систему (секција 3.2.1).

Када прође 10 минута од последњег записа мирне контролне тачке, сервер позива алгоритам записа који чека да се све трансакције заврше и зауставља обраду нових.

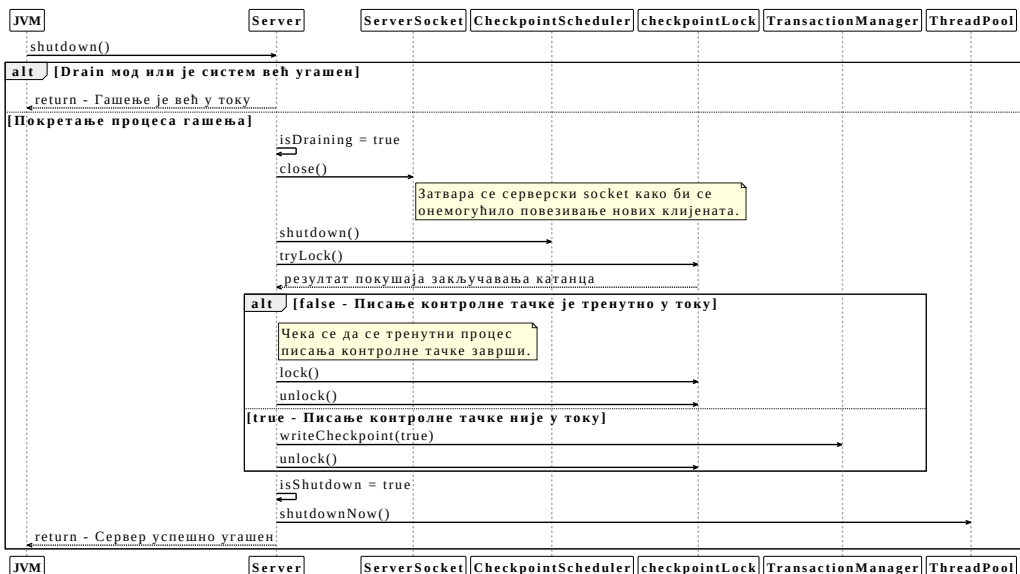
8.2.3 Гашење система

Трећа функционалност је безбедно гашење система. Безбедно гашење се иницира слањем SIGINT сигнала на Unix оперативним системима или слањем CTRL_C (или сличног) сигнала на Windows оперативном систему. Најчешће, ово се мапира на гашење прозора где је сервер покренут, или слањем сигнала преко CTRL + C пречице.

Безбедно гашење се састоји из три корака:

- престајање прихватања нових наредби, сем наредби завршетка трансакције
- чекање да се заврше све трансакције
- записивање мирне контролне тачке

Након што је гашење иницирано, улази се у drain мод, где се не прихватају конекције нових клијената, али старим клијентима је дозвољено да заврше своје трансакције преко COMMIT или ROLLBACK наредби. У drain моду, само ове наредбе су дозвољене. Ако је сервер већ у процесу писања мирне контролне тачке у тренутку слања захтева за гашење, неће се писати још једна.



Слика 32: Дијаграм секвенце безбедног гашења сервера

8.3 Клијентски слој

Класа `LBDBClient` садржи `main` функцију клијентске апликације система. Чита и валидира аргумент командне линије: порт на којем се налази сервер на који се клијент повезује. Сва логика слања наредби се налази у овој класи.

Клијентска апликација пружа корисницима терминал где се наредбе могу уписивати. Терминал подржава аутоматско завршавање кључних речи (енг. *auto complete*) притиском `TAB` тастера и навигацију историје команди. Терминал прати и колико времена се извршавала свака наредба.

Пакети које шаље серверу су серијализовани тако да је на првом месту дужина текста наредбе у бајтовима, а затим и кодирани текст наредбе. Пакете које прима од сервера десеријализује тако што чита прва 4 бајта која представљају дужину пакета у бајтовима, а затим користи алгоритам десеријализације који је описан у протоколу комуникације (секција 8.1.2).

Подржава испис све три врсте `Response` објеката, где се `ErrorResponse` и `EmptySet` тривијално приказују јер садрже само једну вредност. `QuerySet` објекти су табеле и њихово приказивање се ради алгоритмом штампања табела.

8.3.1 Штампање табела

Алгоритам штампања табела има задатак да форматира сва имена колона и све вредности слогова из `QuerySet` објекта. У том објекту стоји шема табеле, па ово није захтеван задатак. Користи `StringBuilder` за ефикасно креирање `String`-ова. За лепши приказ користи специјалне *Unicode* карактере као што су: `┌`, `┐`, `└`, `┘`, `├`, `┤`, `┬`, `┴`, `┴`, `┴`.

tableid	tablename	slotsize
2	tablecatalog	112
3	fieldcatalog	121
5	department	164
6	professor	173
7	student	173
8	course	172
9	enrollment	20

Листинг 5: Пример исписане табеле `SELECT * FROM tablecatalog`; наредбе

8.3.2 Масовно покретање наредби

`LBDB` пакет пружа још једну врсту клијентске апликације: `BulkExecutor`. Ова клијентска апликација функционише слично као и обична клијентска апликација, али уместо пружања интеракције са системом преко терминала, редом извршава све `SQL` наредбе из неке датотеке. Ово ради у ручно започетој трансакцији и ако бар једна наредба не успе са извршавањем, јавља грешку и врши *rollback*. Корисна је за попуњавање табела или за тестирање система.

Пошто је за коректно функционисање система потребно много комплексних функционалности и алгоритама, потребно је извршити интензивно тестирање истих да би се доказала правилна и ефикасна имплементација. Слојеви од којих се систем састоји су тестирани изоловано, с тим да се слојеви вишег апстракционог нивоа не тестирају одвојено од слојева нижег апстракционог нивоа.

9.1 Организација тестова

Све тестове у систему подржава *JUnit*⁵ библиотека. *JUnit* садржи разне конфигурационе параметре, а за тестирање *LBDB* система су најбитнији параметри који омогућавају дефинисање чистача и параметри који омогућавају паралелно покретање тестова.

```
junit.jupiter.extensions.autodetection.enabled = true
junit.jupiter.execution.parallel.enabled = true
junit.jupiter.execution.parallel.mode.default = concurrent
```

Листинг 6: Конфигурациони параметри *JUnit* библиотеке

Систем прати стандардну дефиницију структуре директоријума изворног кода *Maven*⁶ система за управљање зависностима. Више о њему у о прегледу система (секција 10.1). По *Maven*-у, тестови се налазе унутар `src/test/java` директоријума, а конфигурациони параметри *JUnit* библиотеке се налазе унутар `src/test/resources` директоријума. Тестови су груписани по истим модулима као и главни изворни код.

9.1.1 Тестно окружење

Да би се постигло коректно и униформно тестирање свих функционалности, потребно је пружити им одговарајуће тестно окружење. Пошто већина функционалности захтева рад са датотекама, главна дужност тестног окружења је да изолије директоријуме где ће се ове датотеке налазити. Тиме се постиже да тест *A* који креира на пример три табеле не може да утиче на тест *B* који креира две табеле где се имена табела поклапају.

`TestUtils` је помоћна класа која пружа имплементацију ове изолације, али пружа и додатне помоћне методе које олакшавају тестирање:

- провера постојања датотека,
- добављање приватних поља путем *Java* рефлексије.

⁵<https://junit.org/>

⁶<https://maven.apache.org/>

9.1.2 Систем изолације директоријума на диску

Први од два начина покретања тестова је у оквиру директоријума који се налазе на физичком диску. Предности овог начина покретања су лаки преглед генерисаних датотека зарад отклањања грешака и незахтевно покретање док је мана брзина јер је приступ физичком диску спор.

Да би се постигла изолација и кроз итерације покретања истих тестова, потребно је очистити старе директоријуме. *JUnit* омогућава конфигурацију чистача, то јест функције која се извршава пре свих тестова. *GlobalCleanup* класа садржи ову логику.

9.1.3 Систем изолације директоријума у радној меморији

Други од два начина покретања тестова је у оквиру директоријума који се налазе у радној меморији. Пошто је *LBDB* систем компатибилан са *java.nio.file API*-јем, руковођење директоријумима у радној меморији се врши преко *Jimfs* библиотеке. Предност овог начина покретања је брзина тестова, док су мане тежак приступ датотекама зарад отклањања грешака и велика потрошња радне меморије.

У оквиру *TestUtils* класе се подешава начин покретања тестова, где је покретање у радној меморији подразумевано подешено. *TestUtils* дефинише једну инстанцу *Jimfs* имплементације *Filesystem* класе која се користи за све тестове. Нема потребе за чишћењем преко *GlobalCleanup* класе јер се радна меморија сама чисти када се процес у ком су покренути тестови заврши.

9.2 Функционални тестови

Функционални тестови потврђују да ли су алгоритми и структуре података коректни, то јест да ли имају смислено и тачно понашање.

За подсистем релационих оператора, постоји помоћна класа *QueryTestUtils* која пружа додатне помоћне методе за тестирање овог подсистема:

- иницијализација и попуњавање једне табеле са 250 слогова
- иницијализација и попуњавање две табеле са по 250 слогова

За подсистем планирања, постоји помоћна класа *PlanTestUtils* која пружа додатне помоћне методе за тестирање овог подсистема:

- покретање **само** провере *SELECT* наредбе,
- покретање **само** провере модификационих наредби,
- прављење стабла планова за *SELECT* наредбе,
- покретање модификационих наредби,
- креирање нових трансакција, корисно за проверу изолације,
- иницијализација три празне или три попуњене табеле, где прве две имају исту шему као код помоћне класе за тестирање релационих оператора, а трећа садржи само једно *Integer* поље и корисна је за тестирање спајања табела самих са собом.

9.3 Тестирање перформанси

Тестови перформансе имају задатак да потврде претпоставке из текста рада. За коректно извршавање тестова перформанси, потребно је пружити механизме покретања који врше додатну изолацију. Тестови перформансе увек морају да се извршавају серијски, у засебним процесима и на диску. `TestUtils` већ пружа могућност избора извршавања на диску, док се серијско извршавање и извршавање у засебним процесима конфигурише преко *Maven* система и *JUnit* библиотеке. Битна напомена је да је време извршавања зависно од хардвера, док остале метрике нису, али време извршавања и даље може показати корисне увиде. Сваки тест се покреће десет пута и приказане вредности су медијалне да би се избегао шум.

Да би *Maven* инстанцирао нов *JVM* процес за сваки тест, неопходно је да сваки тест стоји у засебној класи. Пошто сви тестови перформансе једног дела система структурално изгледају исто, најбољи начин да се ово обезбеди је да се дефинише једна апстрактна класа у којој стоји логика теста и по једна класа наследница за сваку жељену комбинацију параметара теста. Листинг 7 показује *JUnit* анотације које се везују за апстрактну класу и који су неопходни за серијско покретање.

```
@Execution(ExecutionMode.SAME_THREAD)
@EnabledIfSystemProperty(named = "benchmark", matches = ".*")
```

Листинг 7: Подешавања апстрактне класе теста перформансе

9.3.1 Тестови перформансе алгоритама смене бафера

Постоје четири типа тестова који имају функцију тестирања алгоритама смене бафера под различитим околностима. Типови су описани наредбом која се извршава и количном слогама у табелама. Типови тестова који модификују податке (суфикс *_M*) имају додатну логику покретања две `UPDATE` наредбе која ће учинити бафере обе табеле “прљавим” и тиме дати могућност избора на основу тог податка.

```
public enum BufferTestType {
    HOT_BUFFERS("SELECT * FROM table1, table2;", 1000, 50),
    CONTINUOUS_READS("SELECT * FROM table1;", 50000, 0),
    HOT_BUFFERS_M("SELECT * FROM table1, table2;", 1000, 50),
    CONTINUOUS_READS_M("SELECT * FROM table1;", 50000, 0);

    BufferTestType(String query, int numRecords1, int numRecords2) { ... }
}
```

Листинг 8: Типови тестова за алгоритме смене бафера

```
mvn test -Dgroups="[TIP PO ENUMERACIJI]" -Dbenchmark
```

Листинг 9: Покретање тестова перформансе алгоритама смене бафера одређеног типа

Табела 5: Резултати тестирања алгоритама смене бафера за *HOT_BUFFERS*

Тип теста	Алгоритам	Читања	Писања	Погоци	Време (ms)
<i>HOT_BUFFERS</i>	<i>LRU</i>	337	1	13007	229
<i>HOT_BUFFERS</i>	<i>FIFO</i>	337	1	13006	219
<i>HOT_BUFFERS</i>	<i>Clock</i>	1792	1	11552	227
<i>HOT_BUFFERS</i>	<i>First unmodified</i>	13342	1	2	238
<i>HOT_BUFFERS</i>	<i>LRM</i>	13342	1	2	252
<i>HOT_BUFFERS</i>	<i>Naive</i>	13342	1	2	236

Табела 6: Резултати тестирања алгоритама смене бафера за *CONTINUOUS_READS*

Тип теста	Алгоритам	Читања	Писања	Погоци	Време (ms)
<i>CONTINUOUS_READS</i>	<i>LRU</i>	16669	1	3	162
<i>CONTINUOUS_READS</i>	<i>FIFO</i>	16669	1	3	162
<i>CONTINUOUS_READS</i>	<i>Clock</i>	16669	1	3	164
<i>CONTINUOUS_READS</i>	<i>First unmodified</i>	16671	1	1	157
<i>CONTINUOUS_READS</i>	<i>LRM</i>	16671	1	1	162
<i>CONTINUOUS_READS</i>	<i>Naive</i>	16671	1	1	161

Табела 7: Резултати тестирања алгоритама смене бафера за *HOT_BUFFERS_M*

Тип теста	Алгоритам	Читања	Писања	Погоци	Време (ms)
<i>HOT_BUFFERS_M</i>	<i>LRU</i>	3505	3317	64991	1046
<i>HOT_BUFFERS_M</i>	<i>FIFO</i>	3488	3291	65010	1082
<i>HOT_BUFFERS_M</i>	<i>Clock</i>	10640	2036	57855	1103
<i>HOT_BUFFERS_M</i>	<i>First unmodified</i>	13412	3542	55083	1118
<i>HOT_BUFFERS_M</i>	<i>LRM</i>	68481	3671	14	1232
<i>HOT_BUFFERS_M</i>	<i>Naive</i>	68481	3671	14	1170

Табела 8: Резултати тестирања алгоритама смене бафера за *CONTINUOUS_READS_M*

Тип теста	Алгоритам	Читања	Писања	Погоци	Време (ms)
<i>CONTINUOUS_READS_M</i>	<i>LRU</i>	166695	161509	45	1751
<i>CONTINUOUS_READS_M</i>	<i>FIFO</i>	166695	162112	45	1709
<i>CONTINUOUS_READS_M</i>	<i>Clock</i>	166695	95856	45	1586
<i>CONTINUOUS_READS_M</i>	<i>First unmodified</i>	166609	174945	131	1814
<i>CONTINUOUS_READS_M</i>	<i>LRM</i>	166735	175019	5	1816
<i>CONTINUOUS_READS_M</i>	<i>Naive</i>	166735	175019	5	1799

У табелама 5, 6, 7, 8 стоје вредности метрика: броја читања са диска, броја уписа на диск, број погодака бафера који је већ у меморији и време извршавања.

Резултати тестирања перформанси за тип теста где се ради упит који захтева задржавање истих бафера у меморији су представљени табелом 5. Ови подаци јасно показују поделу између алгоритама који успешно препознају радни скуп унутрашње петље и оних који потпуно заказују. *LRU* и *FIFO* остварују оптималне резултате са минималним бројем читања јер успешно задржавају блокове мање табеле у меморији. Насупрот њима, *Naive*, *LRM* и *First Unmodified* показују најгоре перформансе јер константно избацују потребне податке пре него што се они поново искористе, док је *Clock* позициониран између њих са умереним бројем промашаја.

Резултати тестирања перформанси за тип теста где ради упит који захтева секвенцијално читање су представљени табелом 6. У овом сценарију сви испитивани алгоритми показују практично идентично понашање јер величина табеле драстично превазилази капацитет бафер пула. Ниједна стратегија не може да задржи податке за поновну употребу, па сви алгоритми завршавају са истим бројем читања.

Резултати тестирања перформанси за тип теста где се прво модификују подаци па се ради упит који захтева задржавање истих бафера у меморији су представљени табелом 7. Увођење модификације података додатно наглашава разлике у ефикасности, где *LRU* и *FIFO* поново постижу најбоље резултате са најмање читања јер успешно чувају радни скуп унутрашње петље. *First Unmodified* донекле успешно искоришћава чињеницу да су неки бафери “прљави”, али даје лошије резултате јер не избацавањем “прљавих” бафера превише сужава расположиви простор у меморији. *Naive* и *LRM* увек избацују погрешне бафере. *Clock* заузима средину јер успешно смањује број уписа на диск, али уз цену већег броја читања.

Резултати тестирања перформанси за тип теста где се прво модификују подаци па се ради упит који захтева секвенцијално читање су представљени табелом 8. При масовном скенирању измењених података број читања остаје исти за све стратегије јер се свака страница мора повући са диска, али се кључна разлика уочава у броју уписа. *Clock* алгоритам се овде показује као најефикаснији.

Генерално, у свим тестним типовима, број бафера у систему је изабран да буде 15 јер се тиме постиже да алгоритми морају да раде под притиском и изаберу најбоље бафере за смену. Један слог табеле `table1` је дужине 1240 бајтова, један слог табеле `table2` је дужине 916 бајтова, док је величина блока у систему подешена на 4096 бајтова. Ово значи да у једном блоку стаје три слога табеле `table1` или четири слога табеле `table2`.

Анализом података резултата тестирања, тврдње из секције 2.2.1 су потврђене. *LRU* је убедљиво победник за генерални случај, али постоје и ситуације где не мора бити најбржи.

9.3.2 Тестови перформансе *RBO* технике у планирању *SELECT* наредби

Постоје два типа тестова који имају функцију тестирања алгоритма планирања *SELECT* наредбе. Типови су описани наредбом која се извршава и количином слогова у табели. Код првог типа теста, прво се ставља табела чија је величина слога већа. Код другог типа теста, прво се ставља табела чија је величина слога мања.

```
public enum ProductOrderTestType {
    BIG_SMALL("SELECT * FROM big, small;", 1000),
    SMALL_BIG("SELECT * FROM small, big;", 1000);

    ProductOrderTestType(String query, int numRecords) { ... }
}
```

Листинг 10: Типови тестова за производ великих и малих табела

```
mvn test -Dgroups="[TIP IZ ENUMERACIJE]" -Dbenchmark
```

Листинг 11: Покретање тестова перформансе производа велике и мале табеле са *LRU* алгоритмом избора смене бафера

```
mvn test -Dgroups="[TIP IZ ENUMERACIJE]" -Dbenchmark -Dnaive
```

Листинг 12: Покретање тестова перформансе производа велике и мале табеле са *Naive* алгоритмом избора смене бафера

Табела 9: Резултати тестирања *RBO* технике за тип *BIG_SMALL*

Тип теста	Алгоритам смене бафера	$B(T_r) = B(T_l) + (RPB(T_l) * B(T_l) * B(T_d))$	Читања	Време (ms)
<i>BIG_SMALL</i>	<i>LRU</i>	2338	338	4926
<i>BIG_SMALL</i>	<i>Naive</i>	2338	2342	4845

Табела 10: Резултати тестирања *RBO* технике за тип *SMALL_BIG*

Тип теста	Алгоритам смене бафера	$B(T_r) = B(T_l) + (RPB(T_l) * B(T_l) * B(T_d))$	Читања	Време (ms)
<i>SMALL_BIG</i>	<i>LRU</i>	547094	338	5099
<i>SMALL_BIG</i>	<i>Naive</i>	547094	2342	5367

У табелама 9, 10 стоје вредности метрика: предвиђени број читања формулом, заправи број читања и време извршавања. Додатно, стоји и алгоритам избора смене бафера јер коришћењем *LRU* алгоритма блокови мање табеле константно остају у меморији, док се коришћењем *Naive* алгоритма не постиже ова оптимизација.

На основу измерених података, тврдње из секције 7.2.3.1 су потврђене. Алгоритам планирања *SELECT* наредбе увек бира јефтинију путању и *RBO* техника оптимизације има видљив утицај.

Глава 10

Преглед система

У оквиру овог поглавља су објашњени разноврсни детаљи система који нису везани за саме функционалности система.

10.1 Изградња и покретање

Систем користи *Maven* за: аутоматизацију компилације, изградњу артефаката (апликација које се покрећу) и за руковођење зависностима. У *Maven* екосистему, изворни код прати стриктно дефинисану структуру и налази се унутар `src/main` директоријума.

За коректно функционисање *Maven* апликација, потребно је дефинисати `pom.xml` датотеку у којој се налазе све неопходне инструкције потребне *Maven*-у.

По инструкцијама `pom.xml` датотеке *LBDB* система, клијентске апликације и серверска апликација се граде одвојено, у три различита артефакта. Ово омогућава једноставно одвојено покретање. Након изградње, артефакти се могу пронаћи унутар `target` директоријума под именима: `LBDBServer.jar`, `LBDBClient.jar` и `BulkExecutor.jar`.

```
mvn clean package -Dmaven.test.skip=true
```

Листинг 13: Изградња свих артефаката система, без покретања тестова

```
mvn test
```

Листинг 14: Покретање свих функционалних тестова у систему

10.1.1 Руковођење зависностима

Клијентске апликације и серверска апликација деле код за:

- протокол комуникације,
- дефиницију свих кључних речи (због *auto complete* функционалности клијентске апликације)
- дефиницију константи,
- дефиницију шеме и типа вредности због коректног исписа.

Преостале зависности и код нису дељени.

10.1.1.1 Зависности сервера

Зависности сервера су следеће:

- `datasketches-java` за *Java* имплементацију *HyperLogLog* структуре података⁷,
- `annotations` пружа додатне *Java* анотације попут `@NotNull`⁸.

⁷<https://datasketches.apache.org/>

10.1.1.2 Зависности клијента

Зависности обичног клијента су следеће:

- `jline-reader`, `jline-terminal` и `jline-terminal-jna` пружају имплементацију терминала и омогућавају системски агностичну подршку за *UTF-8* испис⁹,
- `net.java.dev.jna:jna` за приступ *native* инструкцијама оперативног система¹⁰.

Зависности `BulkExecutor` клијента су исте као и зависности обичног клијента, са тиме да `jline` терминал није искоришћен.

10.1.1.3 Зависности тестног окружења

Ове зависности се користе у тестном окружењу и нису део артефакта:

- `junit-jupiter-engine` и `junit-jupiter-params` за покретање и дефинисање тестова,
- `jimfs` је имплементација система датотека у радној меморији¹¹,
- `mockito-core` и `mockito-junit-jupiter` за прављење објеката који имају празну имплементацију а неопходни су за позивање функција и метода¹².

10.2 Системска конфигурација

У оквиру система постоји и конфигурациона класа `LBDBSettings` преко које је могуће поставити параметре избора алгоритама или вредности за одређене операције. Подразумеване вредности су добар избор за ненадгледану иницијализацију система и могу се видети у следећем блоку кода:

```
public class LBDBSettings {
    public boolean UNDO_ONLY_RECOVERY = true;
    public int BLOCK_SIZE = 4096;
    public int BUFFER_POOL_SIZE = 128;
    public BufferStrategy bufferStrategy = BufferStrategy.LRU;
    public String LOG_FILE = "log_file";
    public QueryPlannerType queryPlannerType = QueryPlannerType.BETTER;
    public UpdatePlannerType updatePlannerType = UpdatePlannerType.BASIC;
}
```

Листинг 15: Код системске класе `LBDBSettings`

⁸<https://github.com/JetBrains/java-annotations>

⁹<https://github.com/jline/jline3>

¹⁰<https://github.com/java-native-access/jna>

¹¹<https://github.com/google/jimfs>

¹²<https://github.com/mockito/mockito>

Редом, параметри означавају:

1. алгоритам опоравка система,
2. величина једног блока у бајтовима где један блок представља најмању јединицу интеракције са диском,
3. количина бафера са којим систем располаже,
4. алгоритам избора бафера који ће бити смењен,
5. путања до датотеке где се чувају подаци потребни за опоравак система и поништавање трансакција,
6. имплементација планера за операције упита; подржана само BETTER имплементација,
7. имплементација планера за операције модификације; подржана само BASIC имплементација.

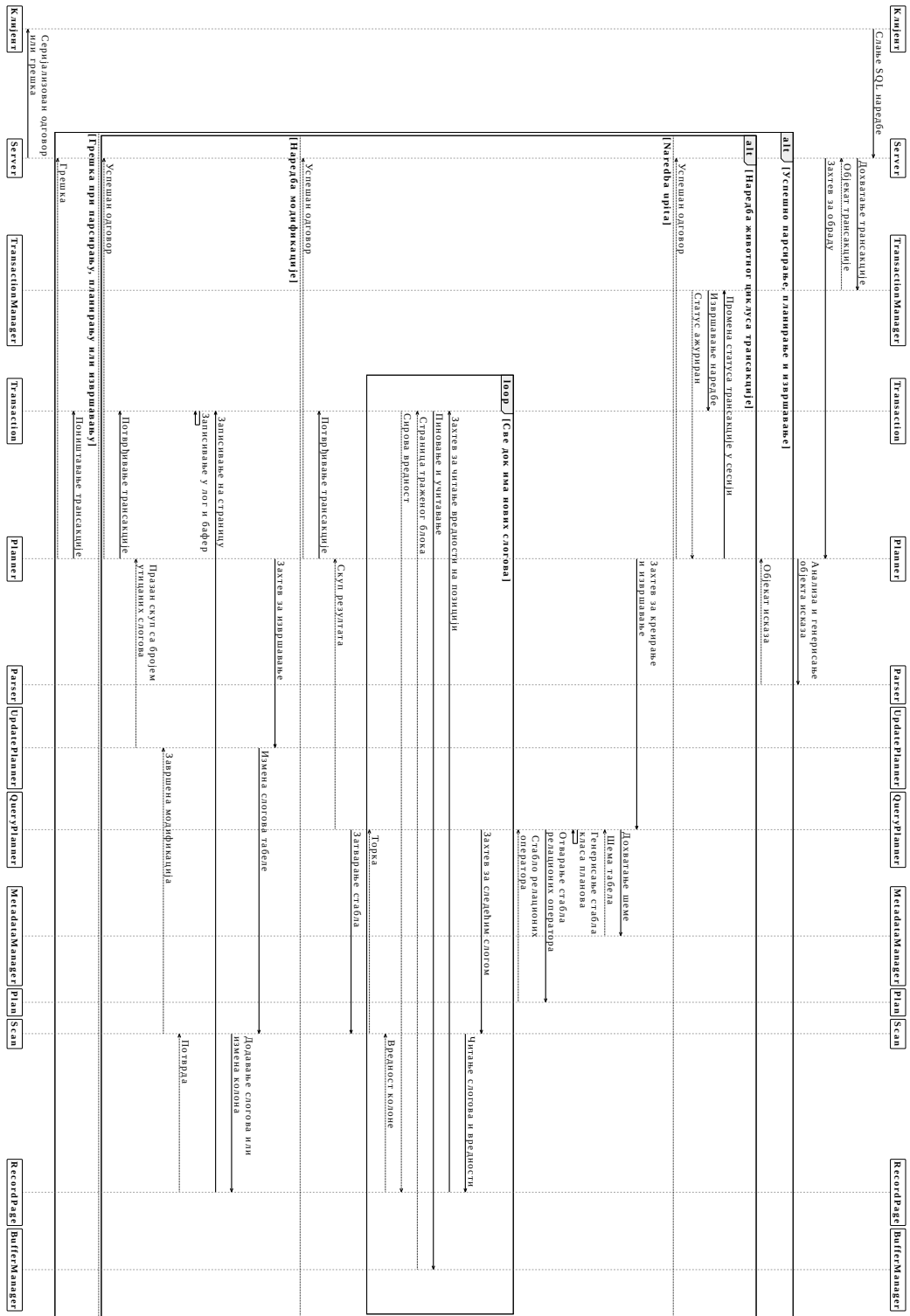
10.3 Интеграција са *GitHub* платформом

*GitHub*¹³ платформа омогућава покретање тестова (енг. *Continuous Integration, CI*) и изградњу и објављивање апликација (енг. *Continuous Delivery, CD*) у оквиру њених сервера. *LBDB* систем користи ову могућност и дефинише специјалну *GitHub* датотеку за *CI*. У оквиру ње се дефинише *Windows* и *Ubuntu Linux* окружење за тестирање. Резултат тестова стоји у оквиру беца у *README.md* датотеци која се налази у *LBDB* репозиторијуму.

10.4 Пример функционисања целокупног система

Дијаграм секвенце (слика 33) представља генерално понашање свих слојева *LBDB* система. Описани су случајеви за *SELECT* наредбу, за наредбе управљања животним циклусом трансакција и за наредбе модификације табела. Специфичности попут алгоритма прављења стабла планова или алгоритма поништавања трансакција нису обрађени јер би дијаграм био превелик, а њихово објашњење је свакако дато у поглављима где су дефинисани.

¹³<https://github.com/>



Слика 33: Дијаграм секвенце функционисања целог система

11.1 Мотивација

Системи за управљање базама података (СУБП) су ме интересовали од друге године основних академских студија, након слушања предмета “Базе података”. СУБП-ови су основа већине информационих система који се користе у данашњици и разумевање само *SQL* језика није довољно да би се схватило њихово интерно функционисање. Због њихове комплексности, схватио сам да би их најбоље разумео тако што имплементирам свој СУБП систем. Научио сам много из области управљања датотекама, изолације података и обраде декларативних програмских језика као што је *SQL*.

11.2 Ограничења система

Иако је *LBDB* СУБП функционалан и пружа задовољавајућу подршку *SQL* језика, за неке делове имплементације се може рећи да им фали још дораде. Одлучио сам да направим пресек код ових ствари, да би постигао функционалну имплементацију у догледном временском периоду, макар она не била савршена.

11.2.1 Имплементиран је само подскуп *SQL*-а

Спецификација *SQL* језика дефинисана *ISO/IEC 9075* стандардом¹⁴ покрива велику количину наредби. *LBDB* систем имплементира само најосновније наредбе потребне за примитивно управљање табелама и слововима.

Аргументација не имплементирања разних група наредби је следећа:

- Иако се брисање табела своди на брисање слогова у каталошким табелама и брисање датотека тих табела, систем опоравка и потврде трансакција није довољно напредан да прати промене које се дешавају ван бафера: креирање и брисање датотека. На пример, ако се у истој трансакцији креира и обрише табела, након потврде трансакције датотека табеле ће остати на диску. Ово се дешава јер процес потврде записује све бафере (секција 3.1.2.2.2), а бар један бафер ће бити додељен новој обрисаној датотеци и његово писање на диск ће увек рекреирати датотеку.

Додавање нових блокова на крају датотека је исте природе као и креирање и брисање датотека, али поништавање те акције је лакше имплементирати пошто ће датотека и даље постојати након опоравка.

¹⁴https://en.wikipedia.org/wiki/ISO/IEC_9075

- Не постоје операције над целим базама података (`CREATE DATABASE ...`) јер нису неопходне за функционисање имплементације релационог модела.
- Не постоје погледи (енг. *views*). Иако у *Database Design And Implementation* [1] књизи постоји опис имплементације погледа, одлучио сам да их избадим јер се нису слагали са свим семантичким проверама планера. Потребно је продубити и евентуално променити начине на који планер проверава упите да би се лако проверавали и погледи.
- `GROUP BY`, `DISTINCT`, `ORDER BY` и остали делови `SELECT` наредбе који захтевају агрегацију података нису подржани јер не постоји имплементација материјализоване обраде. *Database Design And Implementation* [1] у поглављу 13 описује материјализовану обраду, па ћу је истражити за следећу итерацију *LBDB* система.
- Остатак *SQL* језика је превише комплексан за имплементацију у оквиру оваквог пројекта, али је вредно истражити га да би се схватило како модерни СУБП-ови функционишу [9].

11.2.2 Ограничење на слоге фиксне дужине

LBDB систем за управљање датотекама и слоговима ограничава слоге на фиксну, унапред дефинисану величину и ређа их серијски једне до других. Предност овог начина управљања слоговима је једноставност имплементације и лако рачунање позиција слогова (што даље омогућава лако брисање, уметање, ...). Уз неке промене (омогућавање *NULL* вредности), преузет је директно из *Database Design And Implementation* [1] књиге.

Презентују се две велике мане:

- чување вредности различитих дужина у оквиру исте колоне (*String*, то јест *VARCHAR* тип) је немогуће, јер је увек потребно знати унапред величину свих вредности. Да би се могле сместити све вредности до те дужине, финална величина колоне ће увек бити максимална.
- укупна величина слога (у бајтовима) не сме да буде већа од системски дефинисане величине једног блока (исто у бајтовима) јер слогови не могу да се простиру кроз више од једног блока.

Slotted Page Architecture (SPA) представља архитектуру која решава ове проблеме и модерни СУБП-ови је интензивно користе¹⁵. Концепти на које се ослања су први пут уведени у оквиру *SystemR RSS (Relational Storage System)* система [10].

У оквиру *SPA*, вредности слогова, па ни сами слогови, немају предефинисану позицију. Блокови се исто мапирају на странице. Једна страница се састоји од две компоненте: заглавље и вредности. У заглављу стоје специјални бројеви који означавају позиције (енг. *slots*) вредности. Заглавље се увек налази на почетку странице и расте у десно,

¹⁵<https://www.postgresql.org/docs/current/storage-page-layout.html>

а вредности се увек налазе на крају странице и расту у лево. Страница се сматра попуњеним ако се заглавље преклопи са вредностима.

Ако се страница попуни на тај начин да сви слогови који јој логички припадају немају довољно простора за све своје вредности, тој страници се додељује страница преливања (енг. *overflow page*) и тиме се избегава ограничење величине слога на величину блока. Позиција странице преливања се исто налази у заглављу. Могуће је увезивати више страница преливања. Ако се страница попуни и треба да се дода нови слог који јој не припада логички, прави се нова страница уместо странице преливања.

У заглављу такође стоји и позиција следећег логичког слога. Слогови се идентификују преко *slot* вредности па је реорганизација, брисање и додавање слогова могућа манипулацијом само *slot* вредности. Ово знатно олакшава проблеме створене страницама преливања, јер се слогови не прате преко физичке позиције, већ преко логичке позиције. Рупе и фрагментација страница се решавају компакцијом (*PostgreSQL VACUUM* наредба¹⁶).

11.2.3 Споро рачунање статистичких метаподатака

Статистички метаподаци система се чувају у меморији и приступ њима је брз. Проблеми овог начина су то што се не скалира ефикасно са бројем табела и то што се рачунање метаподатака мора радити изнова (при сваком покретању система, на сваких 100 позива). Бољи приступ је чување статистичких метаподатака у засебним каталожним табелама. *LBDB* не подржава овај начин зато што је:

- одржавање ажурности тих табела захтевно
- читање из тих табела потребно радити брзо што даље захтева имплементацију *read uncommitted* изолационог нивоа трансакција

Модерни СУБП-ови имплементирају и цео део споменутог *SQL* стандарда у ком су дефинисани специјални погледи и табеле метаподатака [1]. Поред тога постоје и комплексни хистограми који садрже разне податке из више табела и додатно убрзавају упите [11].

11.2.4 Недостајућа имплементација индексних структура података

Индекси представљају специјалну структуру података која омогућава знатно бржу претрагу података. Реализују се преко једне од *BTree* варијанти или као *Hash* индекс.

Иако су индекси круцијални за ефикасан СУБП, одлучио сам да их не имплементирам јер желим да им се посветим у оквиру следеће итерације *LBDB* система. Ипак, подржано је креирање метаподатака везаних за индексе у оквиру менаџера метаподатака, али сматрам да ово није вредно спомињати ван овог поглавља јер не утиче на даљи систем.

¹⁶<https://www.postgresql.org/docs/current/sql-vacuum.html>

11.2.5 Неоптимални планер

Планер *LBDB* система не користи скоро ниједну напредну технику планирања. Брзина извршавања упита доста зависи од редоследа табела у упиту, услов филтрирања се примењује након уланчавања свих табела уместо изоловано по табели, селективност и процене броја *NULL* и јединствених вредности се не користе, итд.

Поглавља 14 и 15 из [1] описују имплементацију планера који интензивно користи *RBO* технике планирања, али је ово остављено за следећу итерацију *LBDB* система.

11.2.6 Уланчавање чланова предиката је могуће само коњункцијом

Предикати се могу састојати само од чланова уланчаних логичком операцијом коњункције (*AND*). Негација израза (*NOT*) и уланчавање операцијом дисјункције (*OR*) нису подржани иако је лако додати обраду логичких операција јер нисам био сигуран како се уклапају у напредне технике планирања. Када завршим са истраживањем напредног планера, биће ми лакше да убацим и недостајуће логичке операције.

11.3 Разлике између основне имплементације и *LBDB* система

Табела 11: Најзначајнија проширења у имплементацији *LBDB* система

Основна имплементација из [1]	Имплементација <i>LBDB</i> система
<i>Integer</i> и <i>String</i> типови	Подржан и <i>Boolean</i> тип
<i>Naive</i> алгоритам смене бафера	Подржани и <i>FIFO</i> , <i>LRU</i> , <i>Clock</i> , <i>First unmodified</i> , <i>LRM</i> алгоритми смене бафера
<i>Undo only</i> алгоритам опоравка	Подржан и <i>undo redo</i> алгоритам опоравка
-	Подржане <i>NULL</i> вредности
-	Апроксимација броја јединствених вредности колоне табеле преко <i>HyperLogLog</i> структуре
Константни и идентификациони изрази	Подржани и аритметички изрази и чланови са свим операторима поређења
Само оператори селекције, пројекције и производа	Подржани и оператори генерализоване пројекције, преименовања и уније
Просто рачунање редукционог фактора	Рачунање редукционог фактора на основу алгоритма истраживачког рада <i>SystemR</i> [7]
-	Израз заменског члана
-	Детаљна семантичка провера свих наредби
-	Подржана <i>EXPLAIN</i> наредба
-	Редукција израза за време планирања
-	Протокол комуникације, серверски и клијентски слој, контролисано гашење система и периодично писање мирне контролне тачке
-	Имплементација система за тестирање функционалности
Старија <i>Java</i> верзија	Модерна <i>Java</i> верзија и њене могућности попут <i>sealed interface</i> , <i>record</i> , <i>Optional</i> , <i>stream API</i> -ја, ...
-	<i>Maven</i> за аутоматизацију компилације

Списак слика

Слика 1	Упрошћени дијаграм слојевите архитектуре система	1
Слика 2	Изглед <i>log</i> датотеке и смер итерације	4
Слика 3	Изглед једног блока попуњеног слоговима	8
Слика 4	Операције, њихови <i>log</i> типови и структура <i>log</i> типова	11
Слика 5	Хијерархија константи	19
Слика 6	Хијерархија израза	20
Слика 7	Хијерархија имплементације релационих оператора	23
Слика 8	Пример наредбе у <i>SQL</i> језику где се виде све компоненте виртуелне машине	26
Слика 9	Пример позива <i>getValue()</i> у конкретном стаблу релационих оператора	27
Слика 10	Дијаграм секвенце <i>next()</i> у конкретном стаблу релационих оператора ..	28
Слика 11	Граматика <i>Parse</i> синтаксичке категорије	30
Слика 12	Граматика <i>ParseUpdate</i> синтаксичке категорије	31
Слика 13	Граматика <i>ParseSelect</i> синтаксичке категорије	31
Слика 14	Граматика <i>ParseInsert</i> синтаксичке категорије	32
Слика 15	Граматика <i>ParseDelete</i> синтаксичке категорије	32
Слика 16	Граматика <i>ParseCreateTable</i> синтаксичке категорије	33
Слика 17	Граматика <i>ParsePredicate</i> синтаксичке категорије	33
Слика 18	Граматика <i>ParseExpression</i> синтаксичке категорије	34
Слика 19	Граматика <i>ParseEvaluable</i> синтаксичке категорије	36
Слика 20	Хијерархија имплементације планова	38
Слика 21	<i>Flowchart</i> дијаграм рачунице редукционог фактора члана	40
Слика 22	Структура планера	46
Слика 23	<i>Flowchart</i> дијаграм <i>BetterQueryPlanner</i> алгоритма планирања	48
Слика 24	Пример стабла планова након уније 3 наредбе	49
Слика 25	Пример стабла планова након филтрирања и пројекције	49
Слика 26	Пример стабла планова након производа 3 табеле	50
Слика 27	Пример стабла планова након квалификације колона табеле	51
Слика 28	Пример комплетног стабла планова	51
Слика 29	Структура <i>Response</i> хијерархије	56
Слика 30	Структура <i>QuerySet</i> пакета	57
Слика 31	Дијаграм секвенце обраде клијента	58
Слика 32	Дијаграм секвенце безбедног гашења сервера	59
Слика 33	Дијаграм секвенце функционисања целог система	70

Списак листинга

Листинг 1	Подржани типови колона	7
Листинг 2	Evaluatable интерфејс	21
Листинг 3	Приоритети аритметичких операција у систему	35
Листинг 4	Пример резултујуће табеле EXPLAIN наредбе	52
Листинг 5	Пример исписане табеле SELECT * FROM tablecatalog; наредбе	60
Листинг 6	Конфигурациони параметри JUnit библиотеке	61
Листинг 7	Подешавања апстрактне класе теста перформансе	63
Листинг 8	Типови тестова за алгоритме смене бафера	63
Листинг 9	Покретање тестова перформансе алгоритама смене бафера одређеног типа	63
Листинг 10	Типови тестова за производ великих и малих табела	66
Листинг 11	Покретање тестова перформансе производа велике и мале табеле са LRU алгоритмом избора смене бафера	66
Листинг 12	Покретање тестова перформансе производа велике и мале табеле са Naive алгоритмом избора смене бафера	66
Листинг 13	Изградња свих артефаката система, без покретања тестова	67
Листинг 14	Покретање свих функционалних тестова у систему	67
Листинг 15	Код системске класе LBDBSettings	68

Списак табела

Табела 1	Различити изолациони нивои трансакција	14
Табела 2	Шема <i>tablecatalog</i> табеле	17
Табела 3	Шема <i>fieldcatalog</i> табеле	17
Табела 4	Пример карактеристика табела за рачунање броја блокова плана производа	43
Табела 5	Резултати тестирања алгоритама смене бафера за <i>HOT_BUFFERS</i>	64
Табела 6	Резултати тестирања алгоритама смене бафера за <i>CONTINUOUS_READS</i>	64
Табела 7	Резултати тестирања алгоритама смене бафера за <i>HOT_BUFFERS_M</i> ..	64
Табела 8	Резултати тестирања алгоритама смене бафера за <i>CONTINUOUS_READS_M</i>	64
Табела 9	Резултати тестирања <i>RBO</i> технике за тип <i>BIG_SMALL</i>	66
Табела 10	Резултати тестирања <i>RBO</i> технике за тип <i>SMALL_BIG</i>	66
Табела 11	Најзначајнија проширења у имплементацији <i>LBDB</i> система	74

Списак коришћених скраћеница

Скраћеница	Значење
СУБП	Систем за управљање базама података
SQL	<i>Structured Query Language</i>
AST	<i>Abstract Syntax Tree</i>
ACID	<i>Atomicity, Consistency, Isolation, Durability</i>
MVCC	<i>Multi Version Concurrency Control</i>
FIFO	<i>First In First Out</i>
LRU	<i>Least Recently Used</i>
LRM	<i>Least Recently Modified</i>
RAII	<i>Resource Acquisition Is Initialization</i>
RBO	<i>Rule-Based Optimisation</i>
HBO	<i>Heuristics-Based Optimisation</i>
CBO	<i>Cost-Based Optimisation</i>
JDBC	<i>Java Database Connectivity</i>
SPA	<i>Slotted Page Architecture</i>
RSS	<i>Relational Storage System</i>
CI	<i>Continuous Integration</i>
CD	<i>Continuous Delivery</i>
У/И	Улаз/Излаз
API	<i>Application Programming Interface</i>
EOF	<i>End Of File</i>
UTF-8	<i>Unicode Transformation Format</i>

Скраћеница**Значење***RESP**REdis Serialization Protocol*

Списак коришћених појмова

Појам	Објашњење
Блок	Најмања јединица интеракције са датотеком и диском.
Страница	Сирови бајтови једног блока учитани у радну меморију.
Бафер	Омотач око странице у меморији који садржи додатне метаподатке за управљање стањем.
Пиновање	Означавање бафера од стране трансакције како се он не би избацио из радне меморије докле год је у употреби.
<i>Log</i> секвенца	Уникатни идентификатор записаног <i>log</i> -а на основу кога се одржава редослед операција.
Мирна контролна тачка	Ознака у <i>log</i> датотеци након које систем за опоравак не мора да гледа у прошлост јер нема активних трансакција
Дељени катанац	Механизам закључавања блока који омогућава вишенитно читање истих података без модификација.
Ексклузивни катанац	Механизам закључавања блока који гарантује искључиви приступ подацима зарад модификације.
Прљаво читање	Проблем конкурентности где једна трансакција чита непотврђене измене друге трансакције.
Фантомско читање	Проблем конкурентности где се током обраде опсега појаве нови слогови уметнути од стране друге трансакције.
Слог	Најмања инстанца података која описује један ред из табеле.
Шема	Дефиниција атрибута, типова, дужина и ограничења колона за једну табелу.
Каталожка табела	Системска табела базе у којој се перзистирају метаподаци потребни за функционисање система.
<i>HyperLogLog</i>	Пробабилистичка структура података искоришћена за процену броја јединствених вредности.
Виртуелна табела	Скуп резултујућих слогова након примене релационих оператора који нису нужно перзистирани на диску.

Израз	Аритметичка комбинација константи, оператора и колона чијом евалуацијом над неким слогом добијамо константну нумеричку вредност.
Члан	Објект који пореди два израза на основу неког оператора поређења. Евалуацијом добијамо да ли је резултат поређења истинит или не.
Предикат	Логичка комбинација чланова и чијом евалуацијом добијамо константну логичку вредност.
Проточна обрада	Извршавање стабла оператора обрадом слогова један по један, без чувања међуре­зултата у меморији.
Материјализована обрада	Чување свих (или делимичних) израчунатих међуре­зултата на диску или у меморији, потребно за операције сортирања и агрегације.
Редукциони фактор	Статистички број који означава за колико ће се пута смањити количина излазних слогова под дејством одређеног предиката.
Исказ	Највиша синтактичка категорија која описује потпуну <i>SQL</i> операцију и садржи све неопходне податке да би се иста извршила (енг. <i>Statement</i>).
<i>Pratt parsing</i>	Техника <i>recursive descent</i> парсирања за ефикасну валидацију израза и обраду аритметичких приоритета.
<i>Generics</i>	Механизам <i>Java</i> програмског језика за писање агностичног кода којег различити типови користе на исти начин
Пуноквалификујуће име	Име колоне састављено од имена табеле и саме колоне како би се избегла двосмисленост.
Парцијални евалуатор	Компонента која статички своди изразе са тривијалним операцијама на константе већ у тренуцима креирања плана упита.
Протокол серијализације	Скуп правила конвертовања структура података у линеаран низ бајтова за пренос кроз мрежу.
Уграђени режим	Режим рада базе података где она постоји у оквиру истог оперативног процеса као и клијентска апликација која је користи.
Серверски режим	Режим рада где се база података понаша као сервер, примајући наредбе са мреже од више независних клијената.
<i>Socket</i>	Пар <i>IP</i> адресе и порта који, уз додатну логику, омогућава комуникацију преко мреже
Зависност	Софтверска библиотека од које зависи изградња и успешан рад система.
<i>Native</i> инструкције	Инструкције специфичног оперативног система

Литература

- [1] E. Sciore, *Database Design and Implementation: Second Edition*. in Data-Centric Systems and Applications. Springer Cham, 2020.
- [2] T. Haerder and A. Reuter, “Principles of transaction-oriented database recovery,” *ACM computing surveys (CSUR)*, vol. 15, no. 4, pp. 287–317, 1983.
- [3] P. A. Bernstein and N. Goodman, “Multiversion concurrency control—theory and algorithms,” *ACM Transactions on Database Systems (TODS)*, vol. 8, no. 4, pp. 465–483, 1983.
- [4] P. Flajolet, É. Fusy, O. Gandouet, and F. Meunier, “Hyperloglog: the analysis of a near-optimal cardinality estimation algorithm,” *Discrete mathematics & theoretical computer science*, no. Proceedings, 2007.
- [5] E. F. Codd, “A relational model of data for large shared data banks,” *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.
- [6] V. R. Pratt, “Top down operator precedence,” in *Proceedings of the 1st annual ACM SIGACT-SIGPLAN symposium on Principles of programming languages*, 1973, pp. 41–51.
- [7] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price, “Access path selection in a relational database management system,” in *Proceedings of the 1979 ACM SIGMOD international conference on Management of data*, 1979, pp. 23–34.
- [8] E. Wong and K. Youssefi, “Decomposition—a strategy for query processing,” *ACM Transactions on Database Systems (TODS)*, vol. 1, no. 3, pp. 223–241, 1976.
- [9] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. McGraw-Hill Education, 2020.
- [10] M. M. Astrahan *et al.*, “System R: relational approach to database management,” *ACM Trans. Database Syst.*, vol. 1, no. 2, pp. 97–137, June 1976.
- [11] P. B. Gibbons, Y. Matias, and V. Poosala, “Fast incremental maintenance of approximate histograms,” *ACM Trans. Database Syst.*, vol. 27, no. 3, pp. 261–298, Sept. 2002.